

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Multidisciplinary Project (CO3107)

Project:

YOLO: HOME

CC03 - Group 1

Advisor:	Nguyễn Lê Duy Lai	
Students:	Trần Minh Hiếu	- 2252216 (Leader)
	Nguyễn Nhật Khôi	- 2252379 (Member)
	Nguyễn Anh Khoa	- 2252352 (Member)
	Nguyễn Quang Phú	- 2252621 (Member)
	Hà Thế Bình	- 2152435 (Member)

HO CHI MINH CITY, MAY 2025

Contents

<i>List of Figures</i>	6
<i>List of Tables</i>	9
1 Workload Distribution	10
2 Abstraction	12
3 Description	13
4 Requirements	14
4.1 Functional Requirements	14
4.1.1 Smart Door with Face Recognition and Password	14
4.1.2 Voice-Controlled Fan System	15
4.1.3 Auto Light Control with Sensor	15
4.1.4 Monitoring Air Quality	15
4.1.5 Fire Detection	16
4.1.6 Light Management Control by App	16
4.1.7 Fan Management Control by App	16
4.2 Non-functional Requirements	17
4.2.1 Performance Requirements	17
4.2.2 Security Requirements	17
4.2.3 Usability Requirements	18
4.2.4 Maintainability Requirements	18
5 Devices Used in Yolo:Home	19

5.1	Yolo:Bit Controller	19
5.2	Remote Control	20
5.3	IR Sensor	20
5.4	RC Servo Motor	21
5.5	DHT20 Temperature and Humidity Sensor	22
5.6	Mini Fan Module	23
5.7	4 LED RGB Module	23
5.8	LCD 16x2 Display	24
5.9	Light Sensor	25
5.10	PIR Motion Sensor	25
5.11	Summary Table of Device Applications	26
6	System and Design Analysis	27
6.1	Use-case Table	27
6.2	Yolo:Bit Module	28
6.2.1	UC001: Smart Door with Face Recognition and Password	28
6.2.2	UC002: Voice-Controlled Fan System	31
6.2.3	UC003: Auto Light Control with Sensor	34
6.2.4	UC004: Monitoring Air Quality	37
6.2.5	UC005: Fire Detection	39
6.3	Application Module	41
6.3.1	UC006: Light Control via Web App	41
6.3.2	UC007: Fan Control via Web App	44
6.3.3	UC008: Monitoring Sensors by Application	47
6.4	Sequence Diagram for Our Systems	49
6.4.1	Module 1.1: Face Detection	49
6.4.2	Module 1.2: Password Entrance	51
6.4.3	Module 2: Voice Recognition	53
6.4.4	Module 3: Auto-Light Controlled with Sensor	55
6.4.5	Module 4: Air Quality Monitoring Module	57
6.4.6	Module 5: Fire Detection Module	59
6.4.7	Module 6: Light Control via Web App	61

6.4.8	Module 7: Fan Control via Web App	63
6.4.9	Module 8: Monitoring Sensors by Web App	65
7	Implementation	67
7.1	YOLO	67
7.2	Data	67
7.3	Data Preprocessing and Augmentation	68
7.3.1	Motivation and Reasoning	68
7.3.2	Some Advantages of Preprocessing and Augmentation	68
7.3.3	Implementation Overview	68
7.3.4	Code Snippet for Preprocessing	69
7.3.5	Detailed Explanation of Code and Image Transformations	69
7.3.6	Visual Example of Augmented Images	70
7.4	Model Training and Evaluation	71
7.4.1	Training Configuration	71
7.4.2	Model Architecture Overview	73
7.4.3	Training Progress and Metrics	74
7.4.4	Evaluation and Diagnostic Curves	78
7.4.5	Conclusion	89
7.5	Testing and Inference	90
7.5.1	Model YOLOv10	90
7.5.1.1	Quantitative Evaluation	90
7.5.1.2	Qualitative Inference Results	90
7.5.2	Model YOLOv11	91
7.5.2.1	Quantitative Evaluation	91
7.5.2.2	Qualitative Inference Results	92
7.5.3	Model YOLOv12	93
7.5.3.1	Quantitative Evaluation	93
7.5.3.2	Qualitative Inference Results	93
7.5.4	Comparison of YOLOv10, YOLOv11, and YOLOv12	95
7.5.4.1	Detailed Comparison	95
7.5.4.2	Conclusion and Recommendation	95

8	System detailed design	96
8.1	System Interaction Overview	96
8.2	Cloud-Based AI Inference with Oracle Cloud	98
8.3	Supabase Cloud Database Integration	98
8.4	Frontend Technologies and Design Rationale	99
8.5	Backend Technologies and Architecture	100
8.6	Design Justification and Architectural Fit	101
9	Deployment	103
9.1	Github Repository, Ohstem Code and Video Demo	103
9.1.1	Part 1: Yolo:Bit Module	103
9.1.2	Part 2: Web Application Module	103
9.1.3	Part 3: Fire and Smoke Detection Model with AI (YOLOv10, YOLOv11, YOLOv12)	103
9.1.4	Part 4: Demonstration	103
9.2	YoloBit module	104
9.2.1	Monitoring Air Quality	104
9.2.2	Cipher Lock Door	105
9.2.3	IOT Dashboard	106
9.2.4	Voice AI	107
9.2.5	Face AI	109
9.3	Web Application	111
9.3.1	Home Page (Landing Page)	111
9.3.2	Authentication Pages	112
9.3.3	Home Page (Dashboard)	114
9.3.4	Fan and Light Control Page	116
9.3.5	Temperature Page	118
9.3.6	Humidity Page	119
9.3.7	Light Intensity Page	120
9.3.8	Data Logging Page	121
9.3.9	Fire Detection Page	122
10	Conclusion	126

11 Future Developments	127
12 Special Thanks	128

List of Figures

5.1	Yolo:Bit Controller	19
5.2	Remote Control	20
5.3	IR Sensor	20
5.4	RC Servo Motor	21
5.5	DHT20 Sensor	22
5.6	Mini Fan Module	23
5.7	4 LED RGB Module	23
5.8	LCD 16x2 Display	24
5.9	Light Sensor	25
5.10	PIR Motion Sensor	25
6.1	Face Recognition Example	30
6.2	Voice Control Example	33
6.3	Auto Light Control Example	36
6.4	Monitoring Air Quality Example	38
6.5	Fire Detection Example	40
6.6	Sequence Diagram of the Face Detection Module	49
6.7	Sequence Diagram of the Password Entrance Module	51
6.8	Sequence Diagram of the Voice Recognition Module	53
6.9	Sequence Diagram of the Auto-light Controlled Module	55
6.10	Sequence Diagram of the Air Quality Monitoring Module	57
6.11	Sequence Diagram of the Fire Detection Module	59
6.12	Sequence Diagram of the Light Control via Web App Module	61
6.13	Sequence Diagram of the Fan Control via Web App Module	63
6.14	Sequence Diagram of the Monitoring Sensors by Web App Module	65

7.1	Augmentation Techniques Comparison	71
7.2	Training progress for YOLOv10 across 10 epochs.	74
7.3	Training progress for YOLOv11 across 10 epochs.	75
7.4	Training progress for YOLOv12 across 10 epochs.	75
7.5	YOLOv10 Confusion Matrix	78
7.6	YOLOv11 Confusion Matrix	79
7.7	YOLOv12 Confusion Matrix	79
7.8	YOLOv10 Precision–Recall Curve	80
7.9	YOLOv10 Precision vs. Confidence Curve	80
7.10	YOLOv10 Recall vs. Confidence Curve	81
7.11	YOLOv10 F1 Score vs. Confidence Curve	81
7.12	YOLOv10 Label Correlogram	82
7.13	YOLOv11 Precision–Recall Curve	83
7.14	YOLOv11 Precision vs. Confidence Curve	84
7.15	YOLOv11 Recall vs. Confidence Curve	84
7.16	YOLOv11 F1 Score vs. Confidence Curve	85
7.17	YOLOv11 Label Correlogram	86
7.18	YOLOv12 Precision–Recall Curve	87
7.19	YOLOv12 Precision vs. Confidence Curve	87
7.20	YOLOv12 Recall vs. Confidence Curve	88
7.21	YOLOv12 F1 Score vs. Confidence Curve	88
7.22	YOLOv12 Label Correlogram	89
7.23	YOLOv10 Inference Result	91
7.24	YOLOv11 Inference Result	92
7.25	YOLOv12 Inference Result	94
8.1	Detailed System Architecture: Microbit to Cloud Dashboard via MQTT and Adafruit IO	96
9.1	Display information on the LCD character screen	104
9.2	Display information on the LCD character screen	105
9.3	Cipher Lock Door	106
9.4	IOT Dashboard	106

9.5	YoloBit code for Voice AI	108
9.6	YoloBit code for Voice AI	110
9.7	Home page: introduction, team profiles, testimonials	111
9.8	Home page: call-to-action.	112
9.9	Authentication pages: secure login with interactive components.	113
9.10	Authentication pages: user registration with interactive components.	114
9.11	Home Page: comprehensive dashboard with device controls.	115
9.12	Home Page: comprehensive dashboard with environmental monitoring. . . .	116
9.13	Fan and Light Control page: management of fan operation, speed, and LED light color.	117
9.14	Temperature page: real-time monitoring with detailed statistical insights. . .	118
9.15	Humidity page: real-time moisture readings and historical trend analysis. . .	119
9.16	Light Intensity page: monitoring ambient brightness with rich statistics. . . .	120
9.17	Data Logging page: centralized management of all sensor events and activities.	122
9.18	Fire Detection page: selection between Image Detection and Video Stream Detection functionalities.	123
9.19	Image Detection: detecting fire by uploading an image and analyzing it with the remote detection server.	123
9.20	Video Stream Detection: using a laptop camera to simulate real-time fire detection via video streaming.	124

List of Tables

1.1	Workload Distribution Summary (see full details in the detailed Google Sheet)	11
5.1	Summary Table of Device Applications	26
6.1	Summary of Use Cases	27
6.2	Use Case 1 - Smart Door with Face Recognition and Password	30
6.3	Use Case 2 - Voice-Controlled Fan System	33
6.4	Use Case 3 - Auto Light Control with Sensor	36
6.5	Use Case 4 - Monitoring Air Quality	38
6.6	Use Case 7 - Fire Detection	40
6.7	Use Case 6 - Light Control via Web App	43
6.8	Use Case 7 - Fan Control via Web App	46
6.9	Use Case 8 - Monitoring Sensors via Web App	48
7.1	Comparison of Performance Metrics across model	95

Chapter 1

Workload Distribution

Objective/Task	Details	Assigned Members
Kickoff Meeting	Online meeting to plan the project, finalize the topic, identify necessary devices, and assign roles based on members' expertise.	All members
Familiarization with Adafruit	Create initial dashboard with basic functionalities.	Khoi
User Requirement Analysis	Identify system users, write user requirements, and prioritize them.	Khoa, Phu, Hieu, Binh
Functional Requirements	Define system requirements (both functional and non-functional) and create diagrams including class diagrams, use-case diagrams, sequence diagrams, etc.	Khoa, Phu, Hieu, Binh
IoT Gateway Implementation	Implement IoT Gateway according to documentation (Chapters 3, 4, 5).	Khoi
Database Design	Identify collected data, choose suitable database technology (SQL/NoSQL), design schema, and test database connectivity.	Khoa, Phu, Hieu, Binh
Microbit Sensor Integration	Implement connection between Microbit and sensors following documentation (Chapters 5-10).	Khoi
UI/UX Design	Create UI/UX using Figma, select frontend framework, and begin frontend development.	All members
Device Control from Server	Implement device communication using MQTT or WebSockets.	Khoi, Binh

Backend API Implementation	Implement backend APIs to connect frontend, database, and control server.	Hieu
AI Model Implementation	Train AI models for voice and facial recognition; optimize models to work efficiently on embedded hardware (Edge AI).	Khoa, Phu
System Performance Tuning	Optimize API performance, including SQL query tuning and caching.	All members
Final System Testing	Full system testing with real devices; debugging and performance evaluation.	All members

Table 1.1: Workload Distribution Summary (see full details in the [detailed Google Sheet](#))

Chapter 2

Abstraction

The rapid development of technology in the era of **Industry 4.0** has led to the emergence of numerous innovative applications, among which the concept of smart homes has gained significant attention. By integrating **Artificial Intelligence (AI) with the Internet of Things (IoT)**, smart home systems are becoming increasingly intelligent, capable of automation, remote control, and environmental monitoring. This fusion, often referred to as **AIoT (Artificial Intelligence of Things)**, enables smart homes to not only interact with users more naturally but also to learn and adapt to user preferences over time.

This report presents **Yolo:Home**, an end-to-end AIoT platform designed to demonstrate the fundamental and advanced functionalities of a smart home system. The platform incorporates essential features such as environmental monitoring, a cipher lock door, voice control, and facial recognition. Additionally, Yolo:Home introduces two advanced capabilities: a computer vision-based fire detection system and a web application for centralized management of smart devices.

Yolo:Home serves not only as a demonstration of modern smart home technologies but also as a flexible platform for future extensions and educational purposes. Through this project, we aim to bridge the gap between theoretical knowledge and practical implementation, equipping learners with hands-on experience in **AIoT development**.

Chapter 3

Description

Smart homes have become a highly relevant and popular topic within STEM education, providing students with practical exposure to real-world technology applications. In this project, we develop **Yolo:Home**, a smart home platform that combines conventional automation features with **cutting-edge AI and IoT technologies**. By integrating these advanced capabilities, the platform enhances user interaction and operational intelligence, bringing the concept of a modern smart home closer to reality.

Yolo:Home supports core functionalities such as environmental monitoring and automated device control, while also incorporating innovative features such as voice recognition for hands-free operation and facial recognition for personalized access control. Furthermore, Yolo:Home is equipped with a fire detection system using computer vision techniques, enhancing safety through real-time hazard detection. A dedicated web application provides centralized monitoring and control, enabling users to manage their smart home remotely.

A key technical innovation introduced in this project is a new approach to programming the **Yolo:Bit** microcontroller. Instead of relying on traditional sequential programming, the Yolo:Bit system leverages an event-driven architecture based on **MicroPython**. This design allows the system to respond dynamically to real-time events, enhancing flexibility and scalability for future feature development.

This report provides a comprehensive overview of the **Yolo:Home** project, detailing the technologies, hardware components, and system architecture that underpin the platform. Each core functionality is presented alongside the relevant technologies, highlighting the interdisciplinary nature of the project and its alignment with modern AIoT trends.

Chapter 4

Requirements

4.1 Functional Requirements

4.1.1 Smart Door with Face Recognition and Password

- The system shall allow the user to **register credentials**, including **face ID** and **password**, and log in via a mobile application.
- The system shall allow the user to **change credentials**.
- The system shall evaluate **password strength** and display a warning if the password is too weak.
- The system shall **capture and store facial images** using the **door camera**.
- The system shall **authenticate users** using the **facial recognition module** before unlocking the door.
- The system shall **activate the door camera** when a person approaches the door.
- The system shall **log all access attempts**, including both **failed** and **successful** attempts.
- The system shall allow users to **enter a password via remote control** to unlock the door.
- The system shall **verify passwords** against those stored in the **database**.
- The system shall limit the number of consecutive **failed access attempts** before triggering security actions.
- The system shall **send an alarm notification** to the user in case of **multiple failed attempts** within a short period.

- The system shall **lock door access** for a set period after repeated incorrect attempts, and after sending a warning.
- The system shall **ignore all access attempts when offline**.

4.1.2 Voice-Controlled Fan System

- The system shall allow users to **control the fan using voice commands**.
- The system shall maintain a predefined list of **supported voice commands**.
- When a voice command is recognized, the system shall **send a control signal** to the **Ohstem server**.
- The system shall **display a confirmation message** in the **IoT dashboard** when a command is successfully executed.
- If a voice command is not recognized, the system shall **display an error message** and be ready for new commands.
- When a voice command is recognized, the system shall **display a corresponding message**.
- The system shall **notify the user if the voice control module is offline**.

4.1.3 Auto Light Control with Sensor

- The system shall **automatically control lights** based on **ambient light intensity**.
- The system shall allow users to **configure the light intensity threshold**.
- The system shall provide a **manual override option** using **remote control**.
- The system shall **log all light sensor readings and actions**.
- The system shall **indicate manual override status**.
- The system shall **notify the user if the light sensor is disconnected**.
- If the light sensor is disconnected, the system shall **switch to manual mode**.

4.1.4 Monitoring Air Quality

- The system shall **collect and display air quality parameters**.
- If air quality drops below a safe threshold, the system shall **send an alert**.

- The system shall allow users to **configure air quality thresholds**.
- The system shall **log all air quality readings**.
- The system shall **log all notifications and warnings** with **timestamps**.
- The system shall allow users to **export configuration files**.

4.1.5 Fire Detection

- The system shall detect fire incidents and trigger an immediate alarm.
- The system shall notify the user via a web application in case of fire detection.
- The system shall log fire incidents with timestamps for later review.
- The system shall activate emergency response mechanisms upon detecting fire.

4.1.6 Light Management Control by App

- The user can turn the light on or off via the web application.
- The user can change the light's color and adjust the light's brightness.
- The led can automatically turn on/off based on a light sensor reading or on a user-defined schedule.
- The user can configure these thresholds or time intervals.
- The application's dashboard must display the current led status (on/off, color, brightness).
- Any change in the light's state (on/off, color, brightness) should be updated in real time on the application.
- The user can observe and monitor the light data and status.
- Any changes to the light's state must be synchronized across all user devices (web apps).

4.1.7 Fan Management Control by App

- The user can turn the fan on or off via the web application.
- The user can change the fan's speed (low, medium, high, or via a slider).
- The system can allow the fan to run automatically based on temperature or humidity (DHT20 sensor).

- The application dashboard must show the current fan status (on/off, speed).
- Any change in the fan's state (on/off, speed) should be updated in real time on the application.
- The user can observe and monitor the fan data and status.
- Any changes to the fan's state must be synchronized across all user devices (web apps).

4.2 Non-functional Requirements

4.2.1 Performance Requirements

- The system shall process **facial recognition authentication** in **3 seconds**.
- The system shall process **voice commands** within **1 second**.
- The fan, light, and door modules shall respond within **2 seconds** after receiving control commands.
- The system shall process **sensor readings every 4 seconds**.
- The system shall **immediately update the dashboard** after receiving data.
- The system shall support up to **20 connected devices** simultaneously.
- The response time for toggling on/off or adjusting settings (light/fan) should be less than 1 second under normal network conditions.

4.2.2 Security Requirements

- The system shall **encrypt all biometric data and passwords**.
- The system shall **encrypt all communications** between the server and client devices.
- The system should allow adding more lights/fans or other IoT devices without disrupting the service.
- The system should allow adding multiple lights or additional fans by simply registering more MQTT topics on Yolo:Bit and configuring them on the dashboard.

4.2.3 Usability Requirements

- The system shall allow **multiple authenticated users** to use it concurrently.
- New users shall be able to **learn the system within 10 minutes** after reading the manual.
- The system shall support **English and Vietnamese languages**.
- The user interface should update smoothly and continuously (no noticeable lag).
- The user interface for led/fan control should be intuitive (clear on/off buttons, sliders for fan speed, color picker for light, etc.).

4.2.4 Maintainability Requirements

- The system shall **recover within 1 minute** after a power outage.
- The **facial recognition module** shall work in **low-light conditions**.
- The system shall allow **new IoT devices to be added without restart**.
- The system shall support **automatic software updates** with minimal downtime.
- The system should allow adding more lights/fans or other IoT devices without disrupting the service.
- The system should allow adding multiple lights or additional fans by simply registering more MQTT topics on Yolo:Bit and configuring them on the dashboard.

Chapter 5

Devices Used in Yolo:Home

This chapter introduces the hardware devices used in the Yolo:Home project, providing an overview of each device's function and its general applicability to a smart home system.

5.1 Yolo:Bit Controller

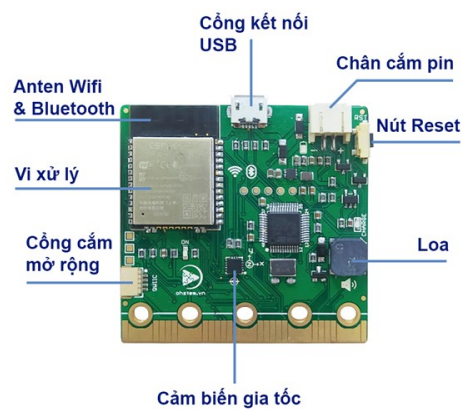


Figure 5.1: Yolo:Bit Controller

Description: The **Yolo:Bit** is the **central controller** of the smart home system. It acts as the core processing unit, collecting sensor data, executing control logic, and facilitating communication between devices.

Application in Smart Home:

- Integrates sensors and actuators into a cohesive system.
- Executes automation rules based on sensor readings.

- Interfaces with the IoT Dashboard to enable remote monitoring and control.

5.2 Remote Control



Figure 5.2: Remote Control

Description: The **remote control** allows users to **manually control** smart home devices, offering an alternative to voice control or automated processes.

Application in Smart Home:

- **Control devices** such as lights, fans, and locks manually.
- **Override automatic actions** if needed.
- **Input security credentials** for systems like smart door locks.

5.3 IR Sensor

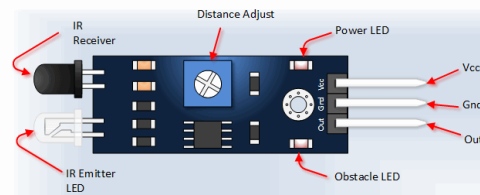


Figure 5.3: IR Sensor

Description: The **IR sensor** receives infrared signals from the remote control.

Application in Smart Home:

- **Receives manual commands** from remote control.
- **Facilitates secure access** through remote password entry.
- **Acts as a bridge** between remote control and smart devices.

5.4 RC Servo Motor



Figure 5.4: RC Servo Motor

Description: The **RC servo motor** controls physical movements such as door locking and unlocking.

Application in Smart Home:

- Automates door locks.
- Controls window blinds or curtains.
- Operates mechanical switches.

5.5 DHT20 Temperature and Humidity Sensor

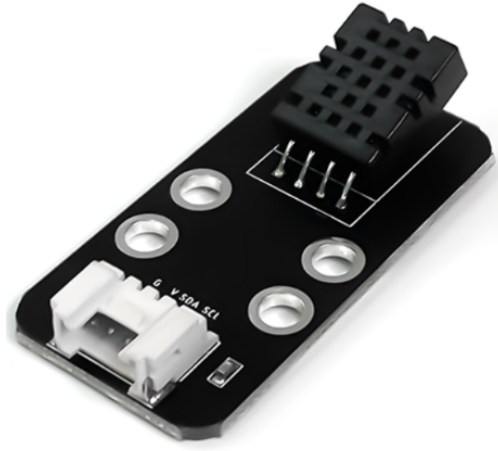


Figure 5.5: DHT20 Sensor

Description: The **DHT20 sensor** measures environmental parameters such as **temperature** and **humidity**.

Application in Smart Home:

- Monitors indoor climate.
- Triggers air conditioning or ventilation systems.
- Provides data for historical analysis.

5.6 Mini Fan Module



Figure 5.6: Mini Fan Module

Description: The **mini fan module** is an actuator used to control airflow.

Application in Smart Home:

- Provides localized cooling in specific rooms.
- Can be controlled automatically based on temperature sensors.
- Supports manual or voice control.

5.7 4 LED RGB Module



Figure 5.7: 4 LED RGB Module

Description: The **4 LED RGB Module** provides customizable lighting through individually addressable LEDs.

Application in Smart Home:

- Ambient lighting control.
- **Visual notifications** (e.g., door unlocked, sensor failure).
- **Color-based alerts** (e.g., red for fire detection).

5.8 LCD 16x2 Display

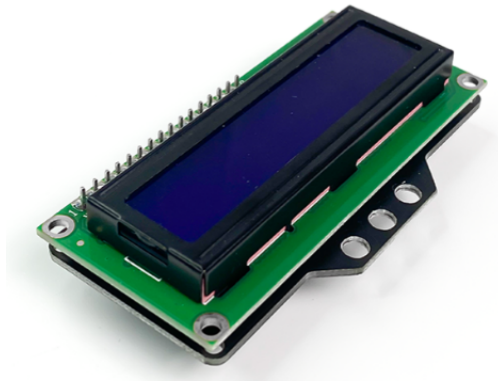


Figure 5.8: LCD 16x2 Display

Description: The **LCD 16x2** is a display module capable of showing textual data.

Application in Smart Home:

- Displays real-time sensor readings.
- Shows system status and notifications.
- **Provides user feedback** during manual operations.

5.9 Light Sensor



Figure 5.9: Light Sensor

Description: The **light sensor** measures ambient light intensity.

Application in Smart Home:

- Automates indoor lighting.
- Contributes to energy saving.
- Provides environmental data for analysis.

5.10 PIR Motion Sensor



Figure 5.10: PIR Motion Sensor

Description: The **PIR sensor** detects motion by sensing infrared radiation.

Application in Smart Home:

- **Triggers security alerts** when motion is detected.
- **Activates cameras or lights** when someone enters a room.
- **Detects room occupancy** for smart automation.

5.11 Summary Table of Device Applications

Device	General Applications in Smart Home
Yolo:Bit Controller	Core controller, processing hub, system communication
Remote Control	Manual device control, password entry, system override
IR Sensor	Receives remote commands, facilitates secure inputs
RC Servo Motor	Controls door locks, window blinds, mechanical actuators
DHT20 Sensor	Measures temperature, humidity, climate monitoring
Mini Fan Module	Provides cooling, supports automatic/remote/voice control
4 LED RGB Module	Lighting control, visual notifications, alerts
LCD Display	Displays sensor data, status, alerts
Light Sensor	Detects ambient light for automatic lighting control
PIR Sensor	Motion detection for security, automation, room monitoring

Table 5.1: Summary Table of Device Applications

Chapter 6

System and Design Analysis

6.1 Use-case Table

ID	Use-case Name	Description
UC001	Smart Door with Face Recognition and Password	The user authenticates with the system to open the door.
UC002	Voice-Controlled Fan System	The user uses the voice to control the fan.
UC003	Auto Light Control with Sensor	The light would be toggled by the state of the sensor.
UC004	Monitoring Air Quality	The air quality would be displayed for monitoring.
UC005	Monitoring Sensors by Application	The user can monitor and display the information of the sensors through the team application.
UC006	Controlling Sensors by Application	The user can control the sensors through the team application.
UC007	Fire Detection	The system detects fire and announces to the user.

Table 6.1: Summary of Use Cases

6.2 Yolo:Bit Module

6.2.1 UC001: Smart Door with Face Recognition and Password

Use Case ID	UC001
Use Case Name	Smart Door with Face Recognition and Password
Created by	Phu
Date Created	05/03/2025
Primary actor	User
Secondary actors	Face Recognition Module YoloBit System Remote Control RC Servo Motor IR sensor Ohstem Server IoT Dashboard
Description	The smart door system allows the homeowner to unlock the door using either face recognition or a password entered via remote control. The system ensures access by authenticated users and logs unsuccessful attempts.
Trigger	- The user approaches the door, or - The user presses a button on the remote to enter a password.
Precondition	- The face recognition model is trained and operational. - The password is already set for the authenticated user. - The remote control, camera, and IR sensor are functioning correctly. - The YoloBit system is powered on.
Postcondition	- If authentication is successful, the door opens. - If authentication fails, unsuccessful attempts are logged for security purposes.

Normal Flow	1. The user approaches the door. 2. The user clicks a button to activate the face recognition module. 3. The camera activates and performs face recognition. If the face is recognized (confidence > 0.7): • A greeting message is published to Ohstem Server. • Ohstem Server forwards the message to the Information Box on the IoT Dashboard. • The IoT Dashboard displays the greeting message. If face recognition fails: • An error message is sent to Ohstem Server. • Ohstem Server forwards the error message to the Information Box on the IoT Dashboard. • The IoT Dashboard displays the error message and the system returns to Step 3.
Alternative Flow	A1: Password Entry Flow - The user uses the Remote Control. - The user enters the password. - The IR sensor receives and verifies the password from YoloBit: • If valid, YoloBit saves the password and confirms via IoT dashboard notification. • If invalid, YoloBit denies the password change and displays an error message. A2: Password Change Flow - The user uses the Remote Control. - The user enters “ADMIN + ADMIN” as a command to initiate password change. - The system prompts the user to enter a new password using the remote control keypad. - The IR sensor verifies the new password from YoloBit: • If valid, YoloBit saves the new password and confirms via IoT dashboard notification. • If invalid, YoloBit denies the password change and displays an error message.
Exception	E1: System Offline or Not Operational If the system is offline or not operational, access is denied, and no authentication occurs.

Notes and Issues	- Ensure that face recognition data is securely stored to comply with privacy regulations. - Logs of failed attempts must be accessible to homeowners. - Notifications should be sent in case of repeated failed attempts or unauthorized access. - Password change functionality should enforce strong password policies.
------------------	---

Table 6.2: Use Case 1 - Smart Door with Face Recognition and Password



Figure 6.1: Face Recognition Example

6.2.2 UC002: Voice-Controlled Fan System

Use Case ID	UC002
Use Case Name	Voice-Controlled Fan System
Created by	Phu
Date Created	05/03/2025
Primary actor	User
Secondary actors	Voice Control Submodule (AI Model) Ohstem Server Mini Fan Module IoT Dashboard
Description	The voice-controlled fan system allows the user to turn the fan on or off using voice commands. The system recognizes the user's command, processes it through the Ohstem server, and sends signals to control the mini fan module. Feedback messages are displayed in the information box on the IoT Dashboard to confirm actions or notify the user of unrecognized commands.
Trigger	The user sends a voice command to control the fan.
Precondition	- The voice control submodule must be operational and able to recognize commands. - The Ohstem server must be connected and functional. - The mini fan module must be powered on and ready to receive signals. - The information box in the IoT Dashboard must be functional for displaying feedback messages.
Postcondition	- If the command is recognized, the fan is turned on or off as per the user's request, and a confirmation message is displayed. - If the command is not recognized, an error message is displayed in the information box.

Normal Flow	1. The user sends a voice command. 2. The voice control submodule recognizes the command. 3. The voice control submodule processes the command. 4. If the command is “Turn on the fan”: • The system triggers the action to turn on the fan. • The voice control submodule publishes signal 1 to V10 (Ohstem server). • The Ohstem server sends signal 1 to the mini fan module, turning on the fan. • A message (“The fan has been turned on”) is published to V13 (information box). 5. If the command is “Turn off the fan”: • The system triggers the action to turn off the fan. • The voice control submodule publishes signal 0 to V10 (Ohstem server). • The Ohstem server sends signal 0 to the mini fan module, turning off the fan. • A message (“The fan has been turned off”) is published to V13 (information box). 6. If the command is not recognized: • The voice control submodule publishes an error message (“I don’t understand”) to V13 (information box). • The information box displays the message for 10 seconds. • The information box displays: “Waiting for other commands”. • Continue at Step 1.
Alternative Flow	None
Exception	E1: System Offline or Not Operational If any component (voice control submodule, Ohstem server, mini fan module, or information box) is offline or not operational, no action can be performed, and no feedback will be provided.

Notes and Issues	- Ensure that voice commands are processed securely and efficiently by integrating robust speech-to-text algorithms in the voice control submodule. - Provide clear instructions for users about acceptable commands to minimize unrecognized input. - Logs of commands and system actions should be maintained for troubleshooting and performance tracking. - Notifications should alert users if any part of the system becomes unresponsive or disconnected.
-------------------------	---

Table 6.3: Use Case 2 - Voice-Controlled Fan System



Figure 6.2: Voice Control Example

6.2.3 UC003: Auto Light Control with Sensor

Use Case ID	UC003
Use Case Name	Auto Light Control with Sensor
Created by	Phu
Date Created	05/03/2025
Primary actor	Homeowner
Secondary actors	YoloBit System IoT Dashboard
Description	The system uses a light sensor to automatically turn lights on or off based on ambient light intensity. When the surrounding light drops below a predefined threshold , the lights are turned on. Conversely, when the ambient light exceeds the threshold, the lights are turned off. This ensures energy efficiency and convenience for the homeowner.
Trigger	The light sensor detects changes in ambient light intensity.
Precondition	- The YoloBit system must be powered on and connected to the light sensor, relay switch, and LED module. - The light sensor must be calibrated to detect accurate light intensity levels. - The relay switch must be operational to control the LED module.
Postcondition	- If ambient light is below the threshold, the lights are turned on. - If ambient light exceeds the threshold, the lights are turned off.

Normal Flow	1. The light sensor continuously monitors ambient light intensity. 2. The sensor sends real-time data to the YoloBit system for processing. 3. YoloBit compares the received light intensity value with a pre-defined threshold. For Low Light Intensity (Below Threshold): • YoloBit sends a signal to activate the relay switch. • The relay switch powers on the connected 4 LED RGB module, turning on the lights. For High Light Intensity (Above Threshold): • YoloBit sends a signal to deactivate the relay switch. • The relay switch powers off the connected 4 LED RGB module, turning off the lights. 4. Continue at Step 1.
Alternative Flow	A1: Manual Override by User (Triggered by Remote Control) - The user presses a button on the remote control to manually turn lights on or off, overriding automatic control. - The IR sensor receives signals from the remote control and sends them to YoloBit for processing. • If “Turn On” is selected, YoloBit activates the relay switch to power on the lights, regardless of ambient light conditions. • If “Turn Off” is selected, YoloBit deactivates the relay switch to power off the lights.
Exception	E1: Sensor Failure or Disconnection - If the light sensor fails or becomes disconnected, YoloBit detects missing data and sends an error message to an LCD display. - The system defaults to manual control via remote until the issue is resolved. E2: System Offline or Not Operational - If YoloBit or any connected components (relay switch or LED module) are offline or not operational, no automatic lighting adjustments can occur, and no feedback will be provided.

Notes and Issues	- Ensure that sensor calibration is performed regularly to maintain accuracy in detecting ambient light intensity. - Provide an option for users to adjust the predefined threshold through an IoT dashboard or remote control. - Logs of lighting activity should be maintained for troubleshooting and energy usage analysis. - Notifications should alert users if any component becomes unresponsive or disconnected.
-------------------------	--

Table 6.4: Use Case 3 - Auto Light Control with Sensor



Figure 6.3: Auto Light Control Example

6.2.4 UC004: Monitoring Air Quality

Use Case ID	UC004
Use Case Name	Monitoring Air Quality
Created by	Phu
Date Created	05/03/2025
Primary actor	User
Secondary actor	YoloBit System
Description	The monitoring air quality system allows the user to measure and display environmental data , including temperature, humidity, and light intensity. The system collects data from sensors, processes it through YoloBit, and displays the results on an LCD module for real-time monitoring .
Trigger	The user clicks the “Run” button to start the air quality monitoring process.
Precondition	- The YoloBit system must be powered on and connected to all sensors. - The LCD module must be operational and ready to display data. - Sensors must be calibrated to provide accurate readings.
Postcondition	- The system successfully displays temperature, humidity, light intensity, date, and time on the LCD module.
Normal Flow	1. The user clicks the “Run” button on the YoloBit system. 2. YoloBit sends a request to the DHT20 sensor to retrieve temperature and humidity data. 3. The DHT20 sensor responds with temperature and humidity values, which are sent back to YoloBit. 4. YoloBit sends a request to the Light Sensor to retrieve light intensity data. 5. The Light Sensor responds with light intensity values, which are sent back to YoloBit. 6. YoloBit combines temperature, humidity, and light intensity data into a single package and sends it to the LCD module. 7. YoloBit retrieves the current date and time and sends this information to the LCD module. 8. The LCD module displays all collected data (temperature, humidity, light intensity, date, and time) for user review.

Alternative Flow	A1: Sensor Failure or Missing Data (applies to all steps) - If any sensor (DHT20 or Light Sensor) fails or does not respond, YoloBit detects the missing data and sends an error message to the LCD module. - The LCD module displays an error message. A2: Data Out of Range (applies to Step 8) - If any collected data exceeds predefined thresholds (e.g., extreme temperature or light levels), YoloBit flags the abnormal readings and sends a warning message to the LCD module (e.g., “Warning: High Temperature”). - The LCD module displays both collected data and warning messages for user attention.
Exception	E1: System Offline or Not Operational - If YoloBit or any connected components (sensors or LCD module) are offline or not operational, no data can be collected or displayed, and no feedback is provided.
Notes and Issues	- Ensure that sensor data is securely transmitted between components without loss or corruption. - Provide calibration options for sensors to maintain accuracy. - Logs of collected environmental data should be stored for historical analysis or troubleshooting purposes. - Notifications should alert users if any sensor becomes unresponsive or disconnected.

Table 6.5: Use Case 4 - Monitoring Air Quality

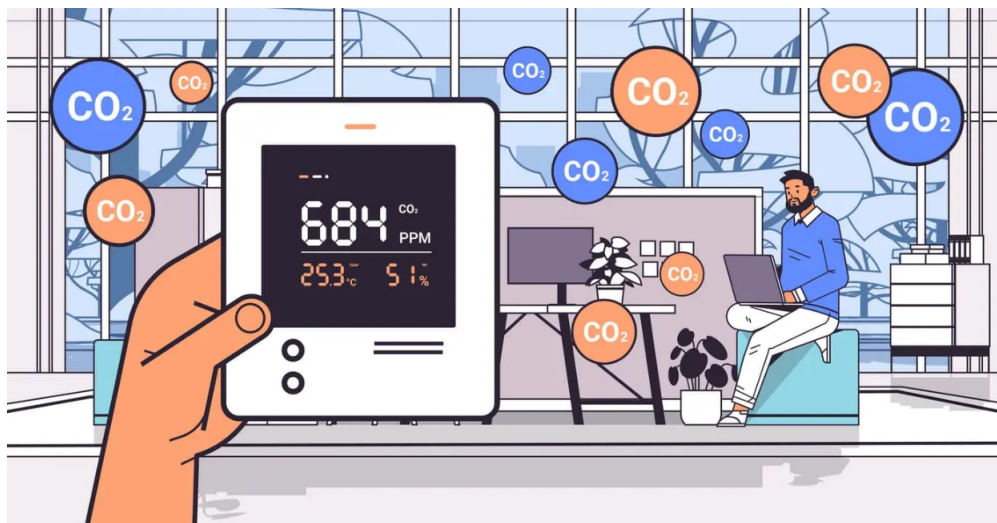


Figure 6.4: Monitoring Air Quality Example

6.2.5 UC005: Fire Detection

Use Case ID	UC007
Use Case Name	Fire Detection
Created by	Khoa
Date Created	05/03/2025
Primary actor	User
Secondary actor	YoloBit System
Description	The fire detection system monitors the rooms within the house using a camera and an AI fire detection model. The camera captures frames in real-time, and the AI model processes them to detect potential fires. If a fire is detected with a confidence score above a predefined threshold, the system signals an alert to the user.
Trigger	The system continuously captures and processes frames using a camera.
Precondition	- The YoloBit system must be powered on and connected to the camera.
Postcondition	- If a fire is detected, an alert is triggered, and a notification is sent to the user. - If no fire is detected, the system continues monitoring without sending alerts.
Normal Flow	1. The system continuously captures and processes frames using a camera. 2. The system processes each frame using the AI-based fire detection model. 3. The model evaluates the image and assigns a confidence score for fire detection. • If the confidence score is above the threshold, the system triggers a fire alert. • If the confidence score is below the threshold, the system continues monitoring without taking action. 4. Continue at Step 1.

Alternative Flow	A1: False Fire Detection (Step 4) - If the user receives an alert but determines that there is no fire, they can dismiss the alert via the IoT Dashboard. - The system logs the false alarm and continues monitoring. - Continue at Step 1.
Exception	E1: System Offline or Not Operational - If YoloBit or the camera is offline or not operational, no frame can be captured. E2: AI Model Failure - If the AI fire detection model fails to process the image, the system logs the failure and restarts the camera module.
Notes and Issues	- Ensure that the image is securely transmitted between components without losing quality or being corrupted. - The system should support threshold adjustments to the fire detection model to avoid false positives. - Notifications should alert users if any sensor becomes unresponsive or disconnected.

Table 6.6: Use Case 7 - Fire Detection



Figure 6.5: Fire Detection Example

6.3 Application Module

6.3.1 UC006: Light Control via Web App

Use Case ID	UC006
Use Case Name	Light Control via Web App
Created by	Binh
Date Created	13/03/2025
Primary actor	User
Secondary actor	Yolo:Home Application, OhStem server, Yolo:Bit, Light module(LED RGB)
Description	The user can control the light via the web application . The user can turn the light on or off and adjust the brightness. The system sends commands to the IoT server, which then communicates with Yolo:Bit to control the LED state. Feedback messages are displayed within the web app interface.
Trigger	The user runs the Yolo:Home Applications and sends a control command from the web application (turn on/off, adjust brightness).
Precondition	- The user is authenticated and logged into the application. - The light module is powered on, connected to the IoT system, and subscribed to the correct topic on the OhStem server. - Yolo:Bit is connected and ready to receive control signals from the server.
Postcondition	- The light is turned on/off or the color is updated based on the user's request. - The user is notified of the action through the web interface.

Normal Flow	1. The user logs into the web app and navigates to the light control panel. 2. The user sees the list of available lights and their status (on/off). 3. The user selects a light and can either: • Turn On/Off (toggle). • Change the color or brightness (using the color picker or slider). 3. The web app processes the corresponding command. 4. If the command is "Turn On". • The system triggers the action to turn on the light. • The command is forwarded to Yolo:Bit via OhStem Server. • Yolo:Bit sends a signal to activate the relay switch. • The relay switch powers on the connected 4 LED RGB module, turning on the lights. 5. If the command is "Turn Off". • The system triggers the action to turn off the light. • The command is forwarded to Yolo:Bit via OhStem Server. • Yolo:Bit sends a signal to activate the relay switch. • The relay switch powers on the connected 4 LED RGB module, turning off the lights. 6. If the command is "Change Color". • The user selects a new color using the color picker. • The system triggers the color change action. • The command is forwarded to Yolo:Bit via the OhStem Server. • Yolo:Bit sends the updated color signal to the LED RGB module, changing its color. 7. If the command is "Adjust Brightness". • The user selects the desired brightness using a slider. • The system processes the change and sends the command to Yolo:Bit via OhStem Server • Yolo:Bit adjusts the brightness of the LED RGB module accordingly.
---------------------------	---

Alternative Flow	None
Exception	E1: Network Error: - If the web app cannot connect to the IoT server, it will display “Connection Error,” and no changes will be made. E2: AI Model Failure - If the light does not respond to the command (due to wiring or hardware issues), the system will display “LED not responding.”

Table 6.7: Use Case 6 - Light Control via Web App

6.3.2 UC007: Fan Control via Web App

Use Case ID	UC007
Use Case Name	Fan Control via Web App
Created by	Binh
Date Created	13/03/2025
Primary actor	User
Secondary actor	Yolo:Home Application, OhStem server, Yolo:Bit, Fan module.
Description	The user can control the fan via the web application. The user can turn the fan on or off, and adjust the fan speed. The system sends commands to the IoT server, which then communicates with Yolo:Bit to control the fan module. Feedback messages are displayed within the web app interface.
Trigger	The user sends a control command from the web application (turn on/off, adjust speed).
Precondition	- The user is authenticated and logged into the application. - The fan module is powered on, connected to the IoT system, and subscribed to the correct topic on the OhStem server. - Yolo:Bit is connected and ready to receive control signals from the server.
Postcondition	- The fan is turned on or off, and the speed is adjusted based on the user's request. - The user is notified of the action through the web interface.

Normal Flow	1. The user logs into the web app and navigates to the fan control panel. 2. The user sees the list of available fans and their current status (on/off). 3. The user selects a fan and can either: • Turn On/Off (toggle). • Adjust the fan speed (low, medium, high). 4. The system sends the control command to the YoloBit system via the OhStem server. 5. If the command is "Turn On": • The system triggers the action to turn on the fan. • The command is forwarded to Yolo:Bit via OhStem server. • Yolo:Bit sends a signal to the fan module to power on the fan. 6. If the command is "Turn Off": • The system triggers the action to turn off the fan. • The command is forwarded to Yolo:Bit via OhStem server. • Yolo:Bit sends a signal to the fan module to power off the fan. 7. If the command is "Adjust Speed": • The user selects the desired speed (low, medium, high). • The system sends the speed adjustment command to Yolo:Bit via OhStem server. • Yolo:Bit adjusts the fan speed accordingly.
Alternative Flow	If the user wants to adjust the fan speed via a slider: • The user selects the desired speed using a slider. • The system sends the speed adjustment command to Yolo:Bit via the OhStem server. • Yolo:Bit processes the command and adjusts the fan speed accordingly. • The web interface updates and displays the current fan speed.

Exception	E1: Network Error: - If the web app cannot connect to the IoT server, it will display “Connection Error,” and no changes will be made. E2: AI Model Failure: - If the fan does not respond to the command (due to wiring or hardware issues), the system will display “Fan not responding.”
------------------	---

Table 6.8: Use Case 7 - Fan Control via Web App

6.3.3 UC008: Monitoring Sensors by Application

Use Case ID	UC008
Use Case Name	Monitoring Sensors by Application
Created by	Binh
Date Created	13/03/2025
Primary actor	User
Secondary actor	Yolo:Home Application, OhStem server, Yolo:Bit, Sensor modules.
Description	The user can view real-time data collected by the sensors, such as temperature, humidity, light intensity, through the web app interface. The system retrieves the sensor data from the IoT server and displays it to the user.
Trigger	The user navigates to the "Sensor Monitoring" page within the web application.
Precondition	- The user is authenticated and logged into the application. - Yolo:Home system is operational and connected to all sensors. - All sensors are calibrated and transmitting accurate data.
Postcondition	- The user sees the most recent sensor data displayed in real-time on the web application. - The system continues to update the data periodically, reflecting the latest readings from the sensor.
Normal Flow	1. The user logs into the web application and navigates to the "Sensor Monitoring" page. 2. The web app sends a request to the OhStem server to retrieve the latest sensor data. 3. The OhStem server processes the request and forwards the sensor data (e.g., temperature, humidity) from the Sensor Module. 4. The web app receives the sensor data and displays it in real-time on the interface. 5. The system periodically updates the sensor data to ensure the user sees the most recent values.
Alternative Flow	None

Exception	E1: Network Error: - If the web app cannot connect to the OhStem server or the sensor data is unavailable, the system displays a "Connection Error" message to the user. E2: Sensor Malfunction: - If the sensor data is corrupted or unavailable, the system displays an error message like "Sensor data not available" and attempts to retrieve the data again.
------------------	---

Table 6.9: Use Case 8 - Monitoring Sensors via Web App

6.4 Sequence Diagram for Our Systems

6.4.1 Module 1.1: Face Detection

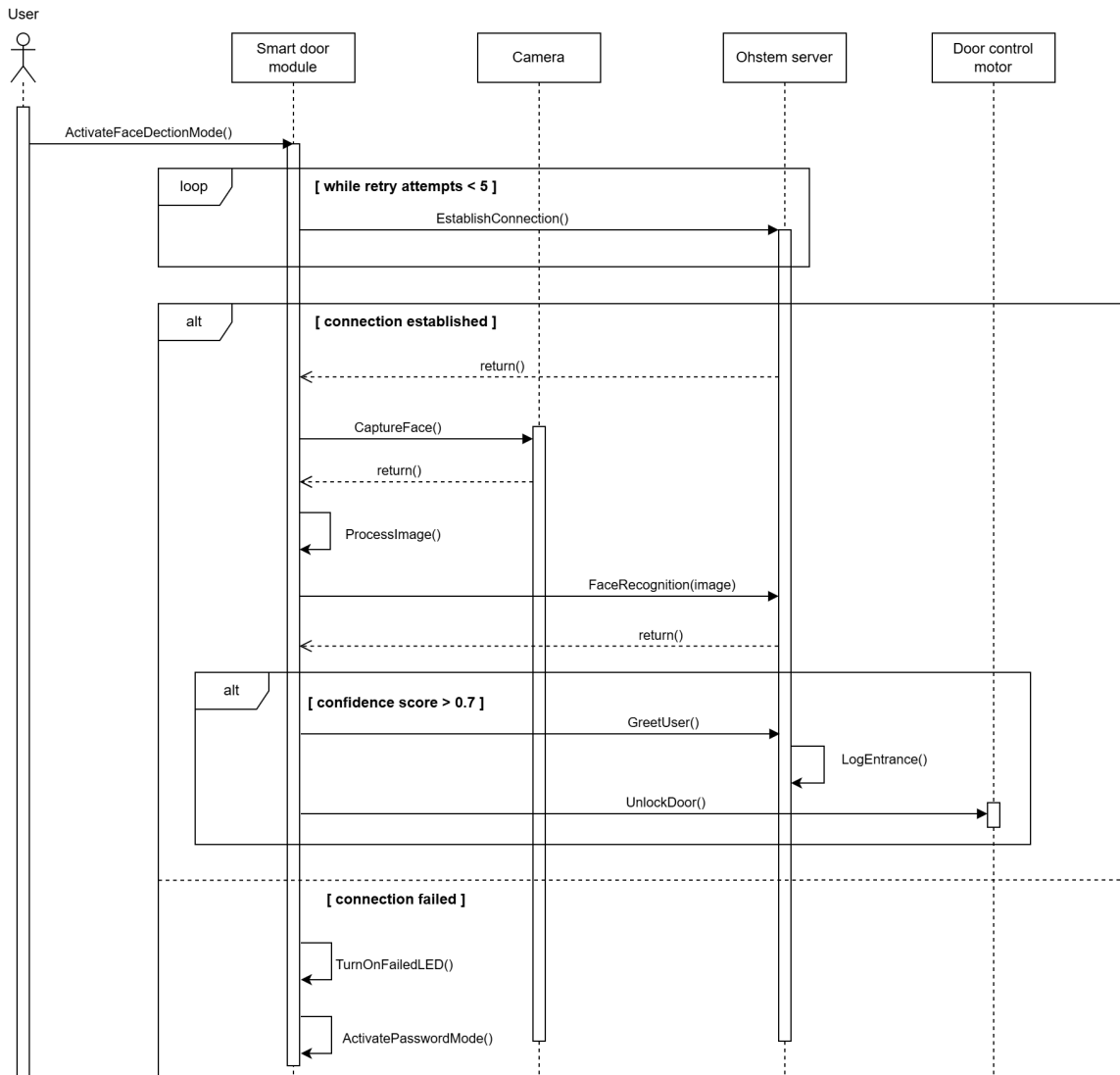


Figure 6.6: Sequence Diagram of the Face Detection Module

The sequence diagram illustrates the process of a smart door system using face detection for user authentication. The interaction involves the user, the smart door module, the camera, the Ohstem server, and the door control motor. The process is as follows:

- The user initiates the system by activating face detection mode (`ActivateFaceDetectionMode()`).
- The smart door module attempts to establish a connection with the camera in a loop, retrying up to 5 times if necessary (`EstablishConnection()`).
- If the connection is successfully established:
 - The camera captures the user's face (`CaptureFace()`), and the image is returned to the smart door module.
 - The smart door module processes the image (`ProcessImage()`) and sends it to the Ohstem server for face recognition (`FaceRecognition(image)`).
 - The server returns a confidence score.
 - If the confidence score is greater than 0.7, The system greets the user (`GreetUser()`), unlocks the door (`UnlockDoor()`), and logs the entrance (`LogEntrance()`).
 - If the confidence score is not sufficient or face recognition fails, The system activates an alternative password mode (`ActivatePasswordMode()`) and turns on a failure indicator LED (`TurnOnFailedLED()`).
- If the connection fails after 5 attempts, the system also activates password mode and failure indication.

This diagram demonstrates conditional flows for successful face recognition, failed recognition, and connection failures, ensuring robust user authentication.

6.4.2 Module 1.2: Password Entrance

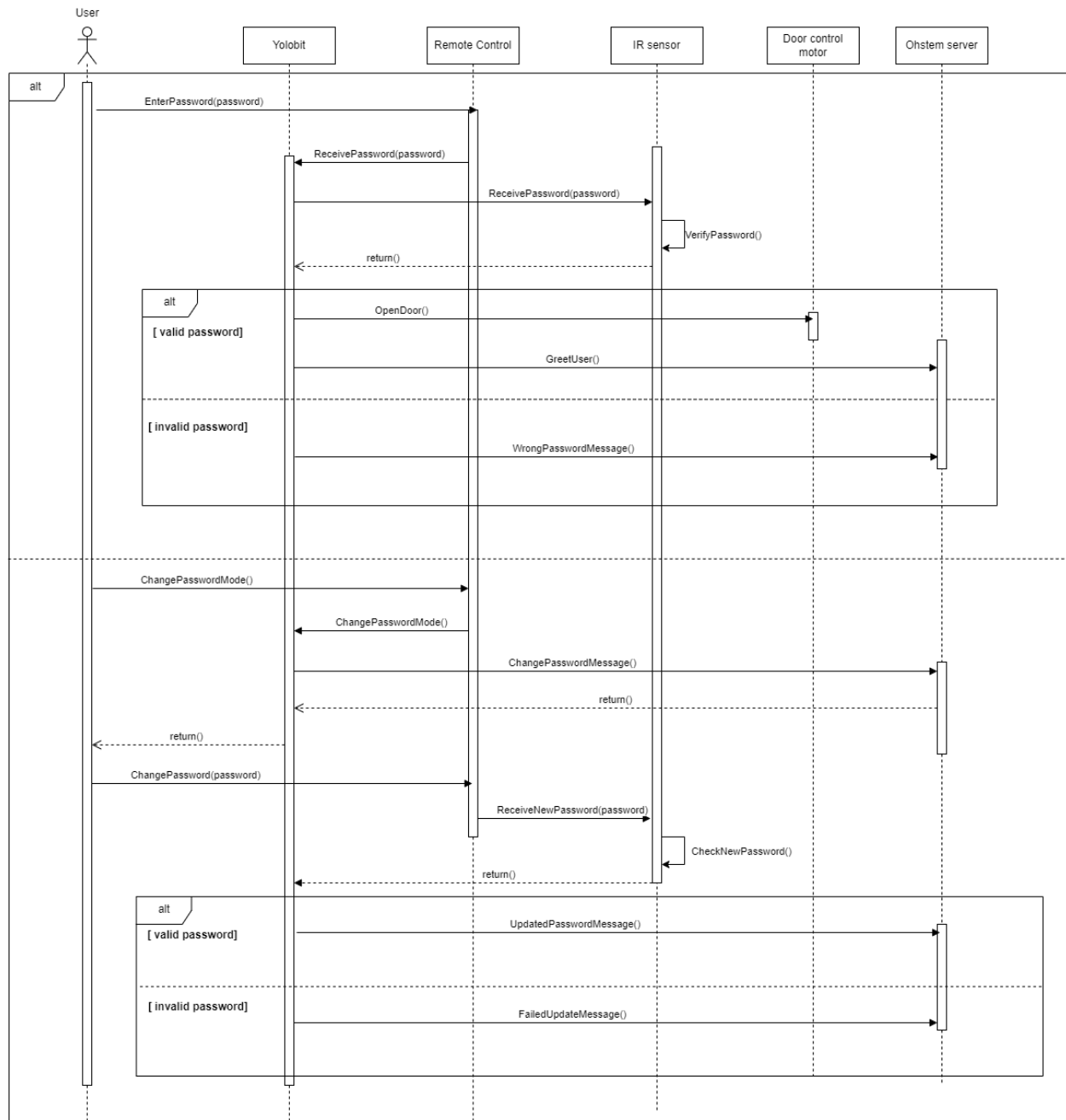


Figure 6.7: Sequence Diagram of the Password Entrance Module

The sequence diagram illustrates the interaction between the **user**, the **YoloBit**, the **remote control**, the **IR sensor**, the **door control motor**, and the **Ohstem server** to handle password-based door access and password management. The process is as follows:

- The user enters a **password** using `EnterPassword(password)`.
- The **YoloBit** receives the password via `ReceivePassword(password)` and forwards it to the **remote control**, which then sends it to the **IR sensor** via `ReceivePassword(password)`.
- The **IR sensor** verifies the password using `VerifyPassword()`.
- Based on the verification result:
 - If the **password is valid**:
 - * The **IR sensor** triggers the **door control motor** to open the door via `OpenDoor()`.
 - * The system greets the user with `GreetUser()`.
 - If the **password is invalid**, The system sends a `WrongPasswordMessage()` to indicate access denial.
- The user can initiate password change using `ChangePasswordMode()`.
- The **YoloBit** processes the mode change via `ChangePasswordMode()` and forwards it to the **IR sensor**, which then sends a `ChangePasswordMessage()`.
- The user enters a **new password** using `ChangePassword(password)`, which is received via `ReceiveNewPassword(password)`.
- The **IR sensor** verifies the new password via `CheckNewPassword()`.
- Based on the verification result:
 - If the **new password is valid**, The system sends an `UpdatedPasswordMessage()` to confirm the update.
 - If the **new password is invalid**, The system sends a `FailedUpdateMessage()` indicating the password update failure.

The diagram demonstrates a secure authentication process for granting access and managing password changes effectively.

6.4.3 Module 2: Voice Recognition

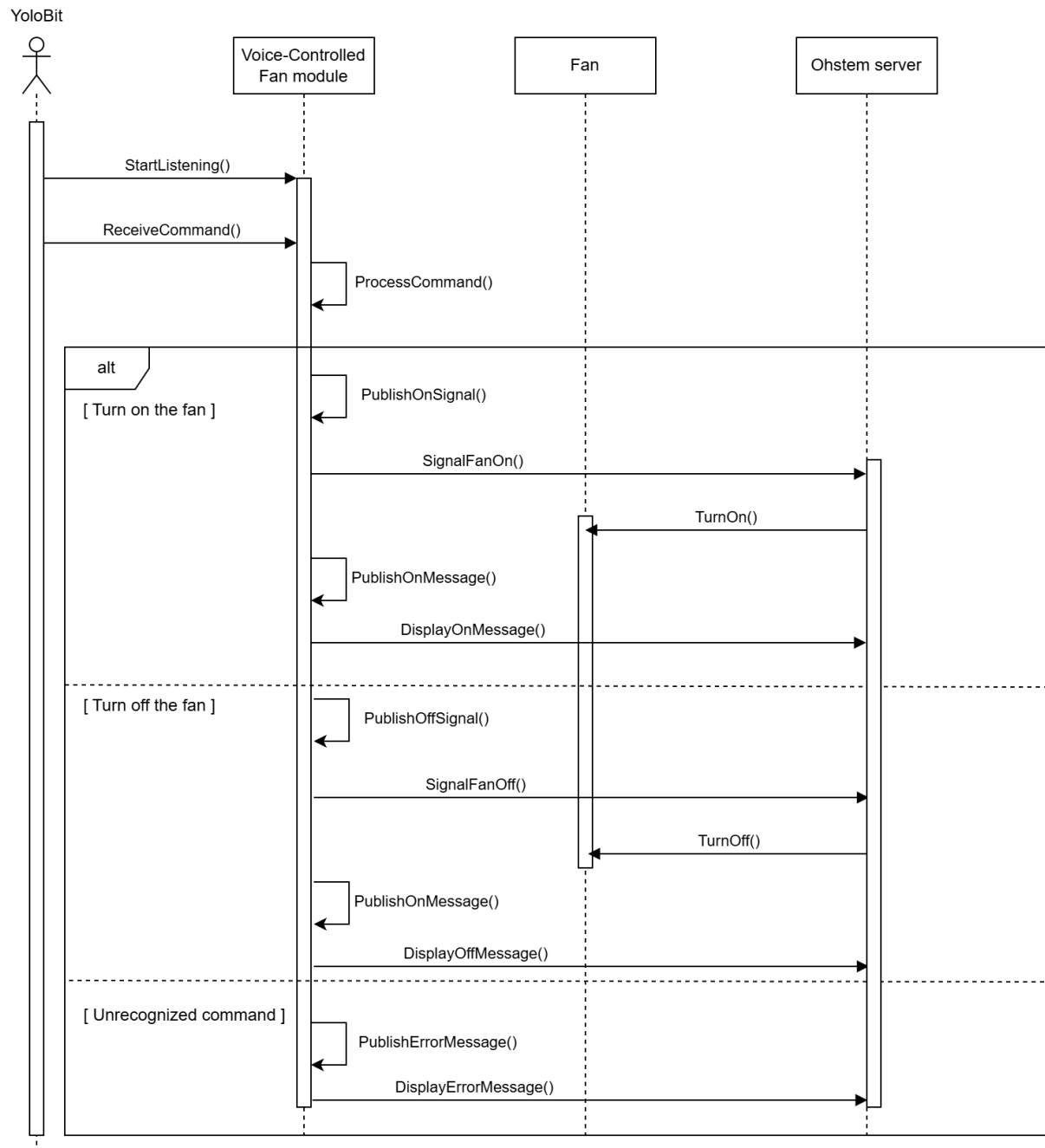


Figure 6.8: Sequence Diagram of the Voice Recognition Module

The sequence diagram illustrates the **interaction between the YoloBit user, a voice-controlled fan module, the fan, and the Ohstem server** for controlling the fan. The process is as follows:

- The user **initiates the interaction** by starting the listening process (`StartListening()`) via the voice-controlled fan module.
- The **voice-controlled fan module** receives a command (`ReceiveCommand()`) and processes it (`ProcessCommand()`).
- Based on the command:
 - **If the command is to “Turn on the fan”**, the module publishes an “on” signal (`PublishOnSignal()`), signals the fan to turn on (`SignalFanOn()`), and executes the `TurnOn()` function. The module also publishes a message (`PublishOnMessage()`) and displays feedback (`DisplayOnMessage()`).
 - **If the command is to “Turn off the fan”**, the module publishes an “off” signal (`PublishOffSignal()`), signals the fan to turn off (`SignalFanOff()`), and executes the `TurnOff()` function. Similarly, it publishes a message (`PublishOffMessage()`) and displays feedback (`DisplayOffMessage()`).
 - If an unrecognized command is received, an error message is published (`PublishErrorMessage()`) and displayed (`DisplayErrorMessage()`).

The diagram demonstrates conditional flows for recognized commands (turning on/off) and unrecognized commands, ensuring proper communication between components.

6.4.4 Module 3: Auto-Light Controlled with Sensor

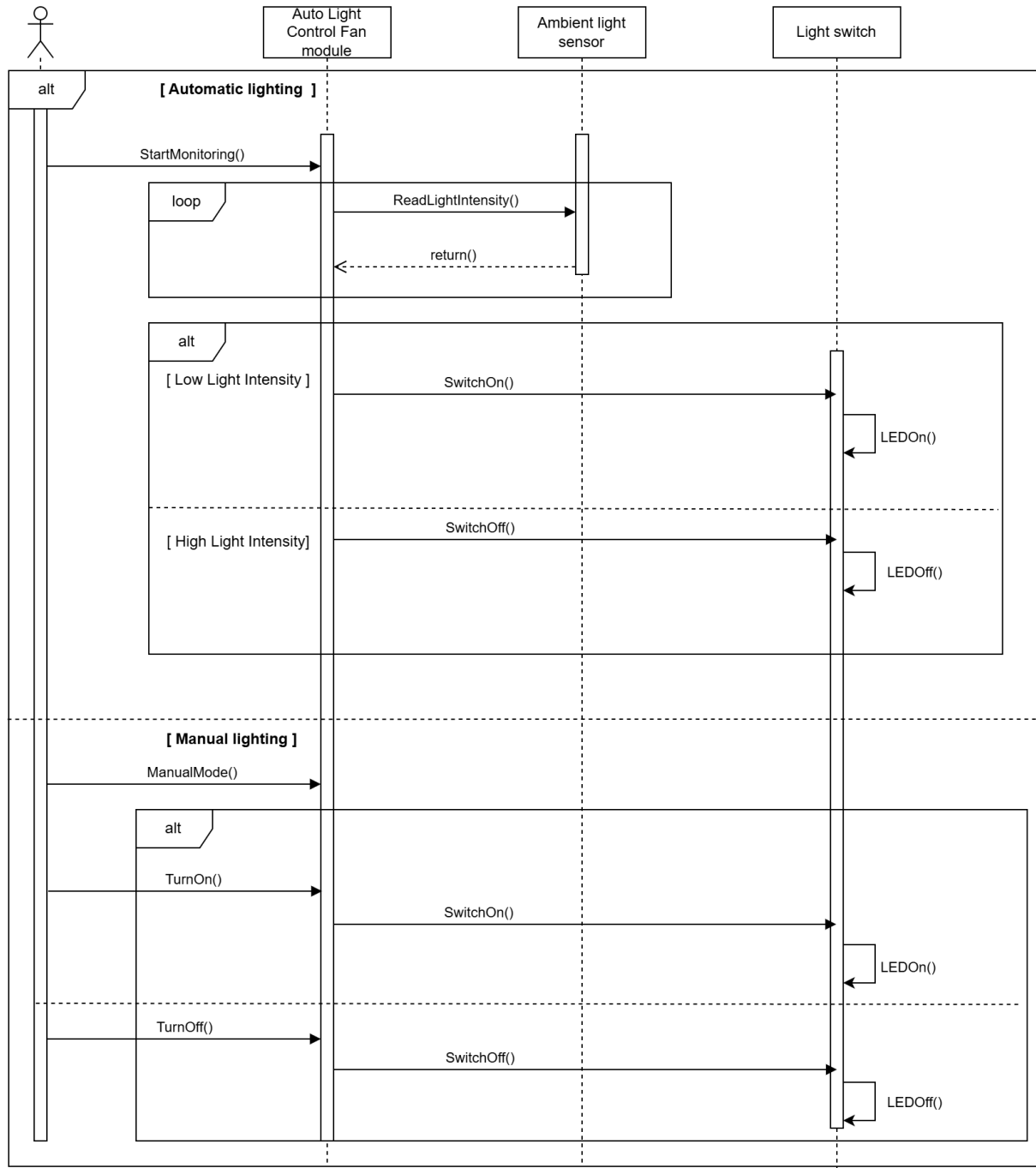


Figure 6.9: Sequence Diagram of the Auto-light Controlled Module

The sequence diagram illustrates the interaction between the **user**, the **auto light control module**, the **ambient light sensor**, and the **light switch** to manage automatic and manual lighting modes. The process is as follows:

- The user initiates the **automatic lighting** process by calling `StartMonitoring()` on the **Auto Light Control Fan module**.
- The **Auto Light Control module** enters a loop where it continuously calls `ReadLightIntensity()` on the **ambient light sensor**.
- The **ambient light sensor** returns the **light intensity** value.
- Based on the returned value:
 - If the **light intensity** is **low**, the **Auto Light Control module** calls `SwitchOn()`, which sends a signal to the **light switch** to turn on the LED (`LEDOn()`).
 - If the **light intensity** is **high**, the **Auto Light Control module** calls `SwitchOff()`, which sends a signal to the **light switch** to turn off the LED (`LEDOff()`).
- The user can also initiate **manual lighting** mode by calling `ManualMode()`.
- In manual mode, the user can choose to turn the light **on** or **off**:
 - If the user selects `TurnOn()`, the system calls `SwitchOn()`, activating the LED (`LEDOn()`).
 - If the user selects `TurnOff()`, the system calls `SwitchOff()`, deactivating the LED (`LEDOff()`).

The diagram demonstrates the system's ability to handle both **automatic** and **manual lighting control**, ensuring flexibility and efficient light management.

6.4.5 Module 4: Air Quality Monitoring Module

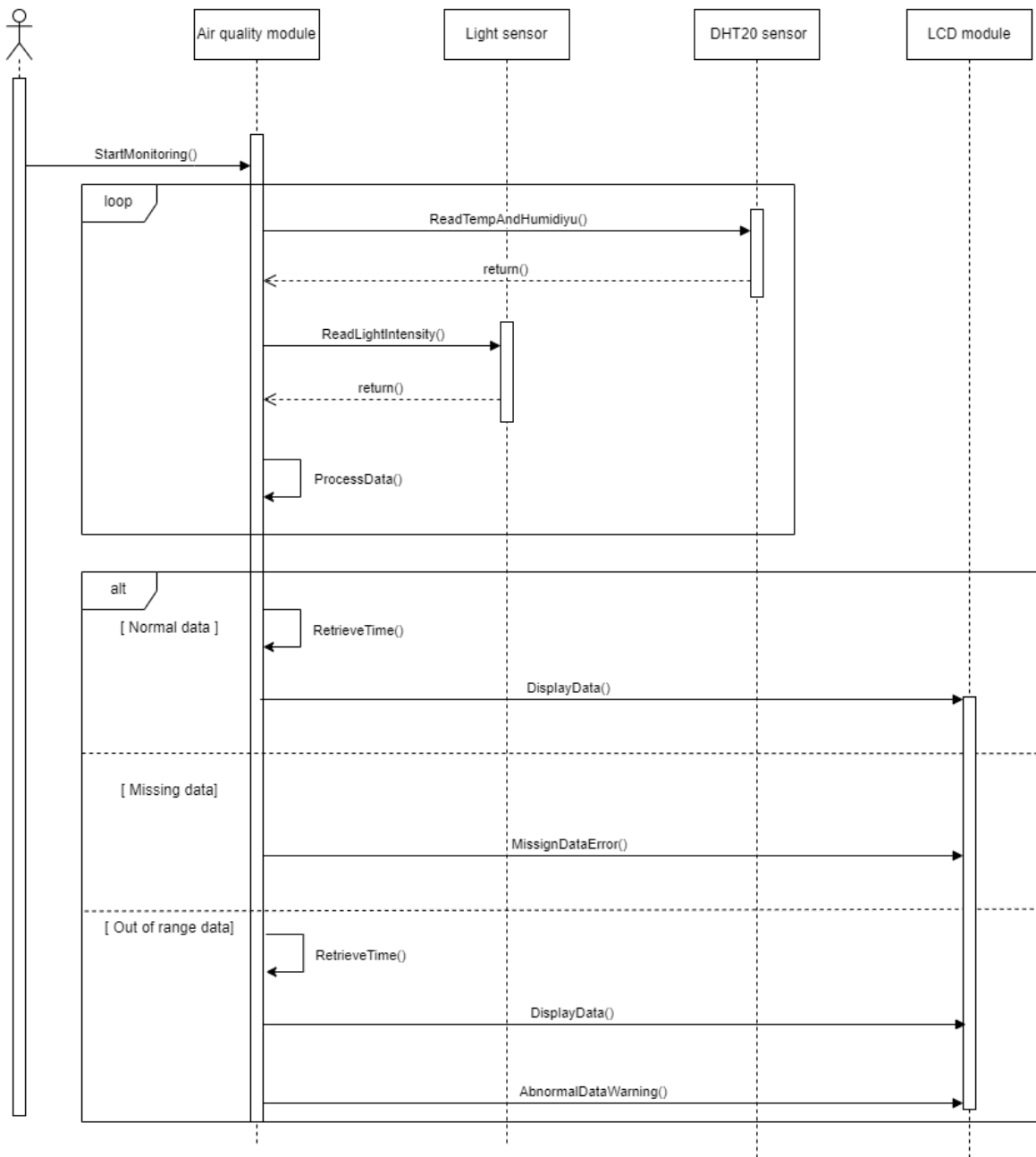


Figure 6.10: Sequence Diagram of the Air Quality Monitoring Module

The sequence diagram illustrates the interaction between the **user**, the **air quality module**, the **light sensor**, the **DHT20 sensor**, and the **LCD module** to monitor environmental conditions and display the results. The process is as follows:

- The user initiates the **monitoring process** by calling `StartMonitoring()` on the **Air Quality Module**.
- The **Air Quality Module** enters a loop where it continuously:
 - Calls `ReadTempAndHumidity()` on the **DHT20 sensor** to obtain temperature and humidity data.
 - Calls `ReadLightIntensity()` on the **Light Sensor** to measure ambient light levels.
 - Processes the collected data using `ProcessData()`.
- Based on the processed data, the system handles three possible cases:
 - **Normal Data:**
 - * Calls `RetrieveTime()` to get the current timestamp.
 - * Calls `DisplayData()` on the **LCD module** to show the recorded environmental values.
 - **Missing Data:**
 - * Calls `MissingDataError()` on the **LCD module** to display an error message indicating unavailable sensor readings.
 - **Out of Range Data:**
 - * Calls `RetrieveTime()` to get the current timestamp.
 - * Calls `DisplayData()` on the **LCD module** to show the recorded values.
 - * Calls `AbnormalDataWarning()` on the **LCD module** to display a warning about detected abnormal environmental conditions.

The diagram demonstrates how the system continuously monitors environmental parameters and displays real-time updates while handling missing or out-of-range data effectively.

6.4.6 Module 5: Fire Detection Module

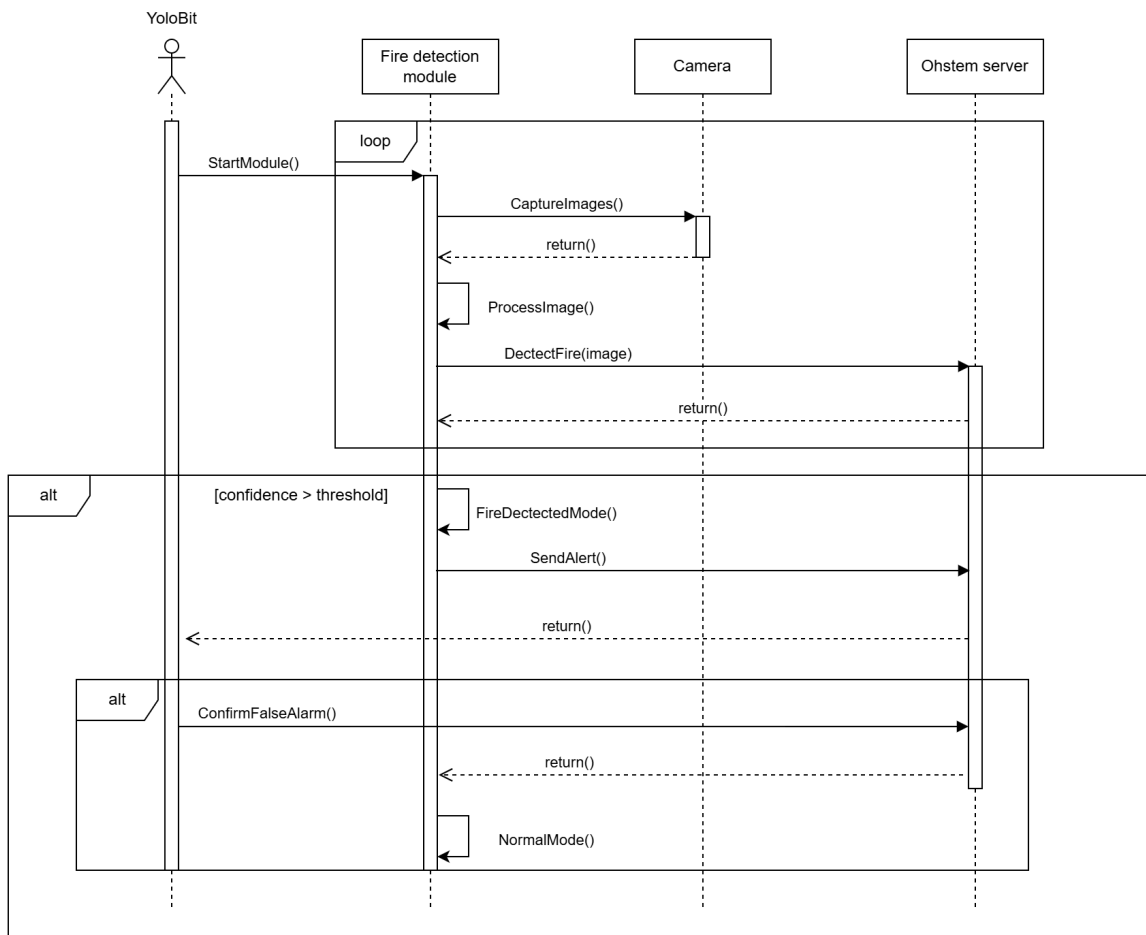


Figure 6.11: Sequence Diagram of the Fire Detection Module

The sequence diagram illustrates the interaction between the **YoloBit user**, the **fire detection module**, the **camera**, and the **Ohstem server** to detect and respond to fire incidents. The process is as follows:

- The user initiates the **fire detection module** by calling `StartModule()`.
- The **fire detection module** enters a loop where it continuously:
 - Calls `CaptureImages()` on the **camera** to take an image.
 - Receives the captured image and processes it using `ProcessImage()`.
 - Calls `DetectFire(image)` to analyze the image for signs of fire.
- If the **confidence** of fire detection is **above the threshold**:
 - The **fire detection module** enters `FireDetectedMode()`.
 - Calls `SendAlert()` to notify the **Ohstem server** of the fire event.
- If a **false alarm** is confirmed:
 - Calls `ConfirmFalseAlarm()` to verify the detection.
 - Returns to `NormalMode()` if no fire is present.

The diagram demonstrates how the system continuously monitors fire hazards, processes image data, and issues alerts when necessary, while allowing for false alarm verification to prevent unnecessary alerts.

6.4.7 Module 6: Light Control via Web App

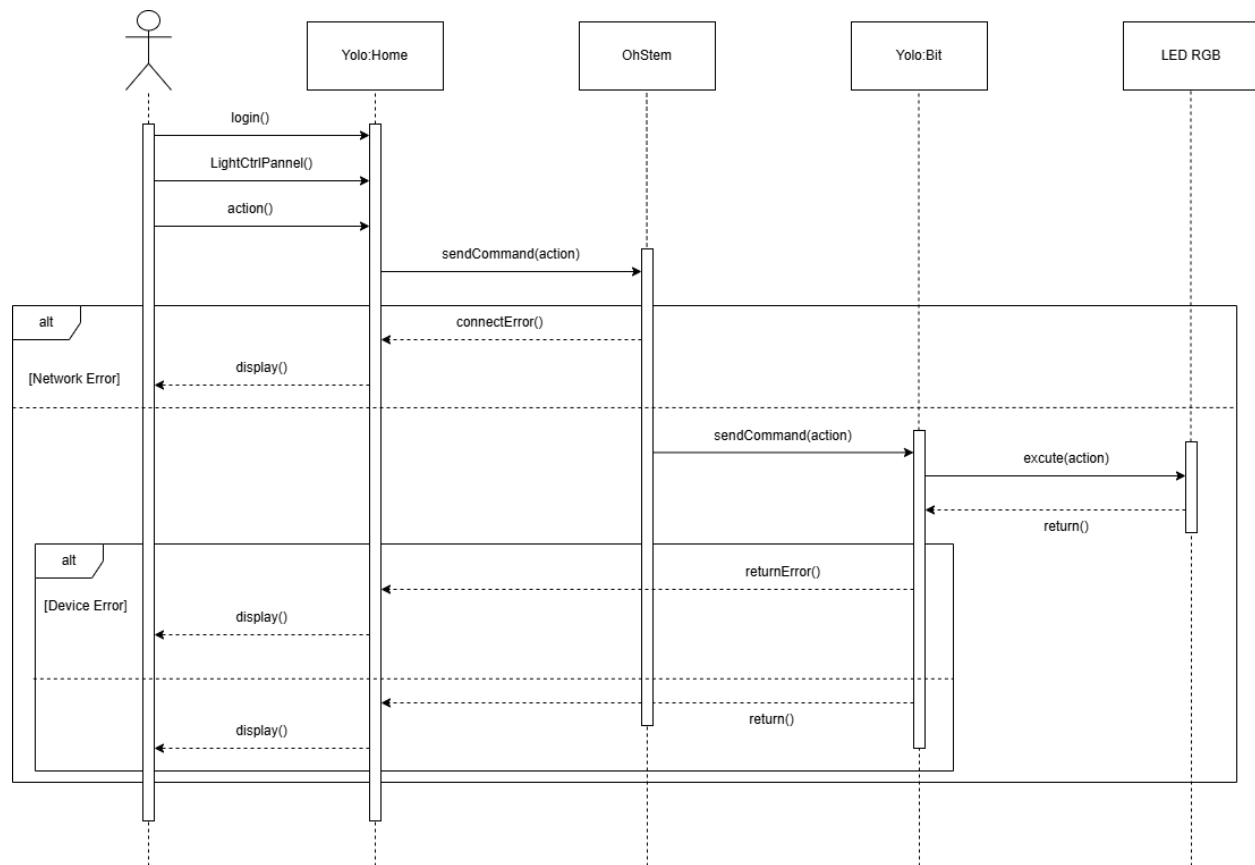


Figure 6.12: Sequence Diagram of the Light Control via Web App Module

The sequence diagram illustrates how to control a light module through a web or mobile interface involving the User, the Yolo:Home(Web Application), OhStem(IoT Server), Yolo:Bit, and the Light module(LED RGB). The process is as follows:

- The user access YoloHome application and performs the login() action. Once logged in successfully, the user then navigates to LightCtrlPanel() within YoloHome.
- The user sees the list of available lights and their status (on/off/color/brightness) and controls for adjusting them.
- The User choose a light and select a specific action() from the panel:
 - Calls toggleON() to turn On the light
 - Calls toggleOFF() to turn Off the light.
 - Calls setColor(newColor) to change the light's color.
 - Calls setBrightness(level) to adjust the light's brightness

- YoloHome sends the command to the OhStem server via `sendCommand(action)`. The server (OhStem) receives this request and forwards it to the YoloBit device, again using `sendCommand(action)`.
- YoloBit executes the command `execute(action)` on the LED RGB hardware. Depending on the action, it may turn the LED on or off, change the LED color, or adjust the brightness.
- Once the LED RGB responds, YoloBit returns the outcome (success or error) to OhStem, which in turn sends the result back to YoloHome. YoloHome updates the user interface accordingly and `display()` to display the present status of the light to the User.
- If there is a network issue between YoloHome and OhStem, `sendCommand(action)` fails, and YoloHome displays an error message (“Network Error”).
- If YoloBit or the LED RGB fails to respond or encounters an issue, the error is propagated back (`returnError()`). YoloHome displays an error message (“LED not responding” or similar) to inform the user.

The diagram demonstrates how the system provides real-time feedback to the user while effectively handling and reporting any network or device errors, ensuring a reliable and seamless control experience.

6.4.8 Module 7: Fan Control via Web App

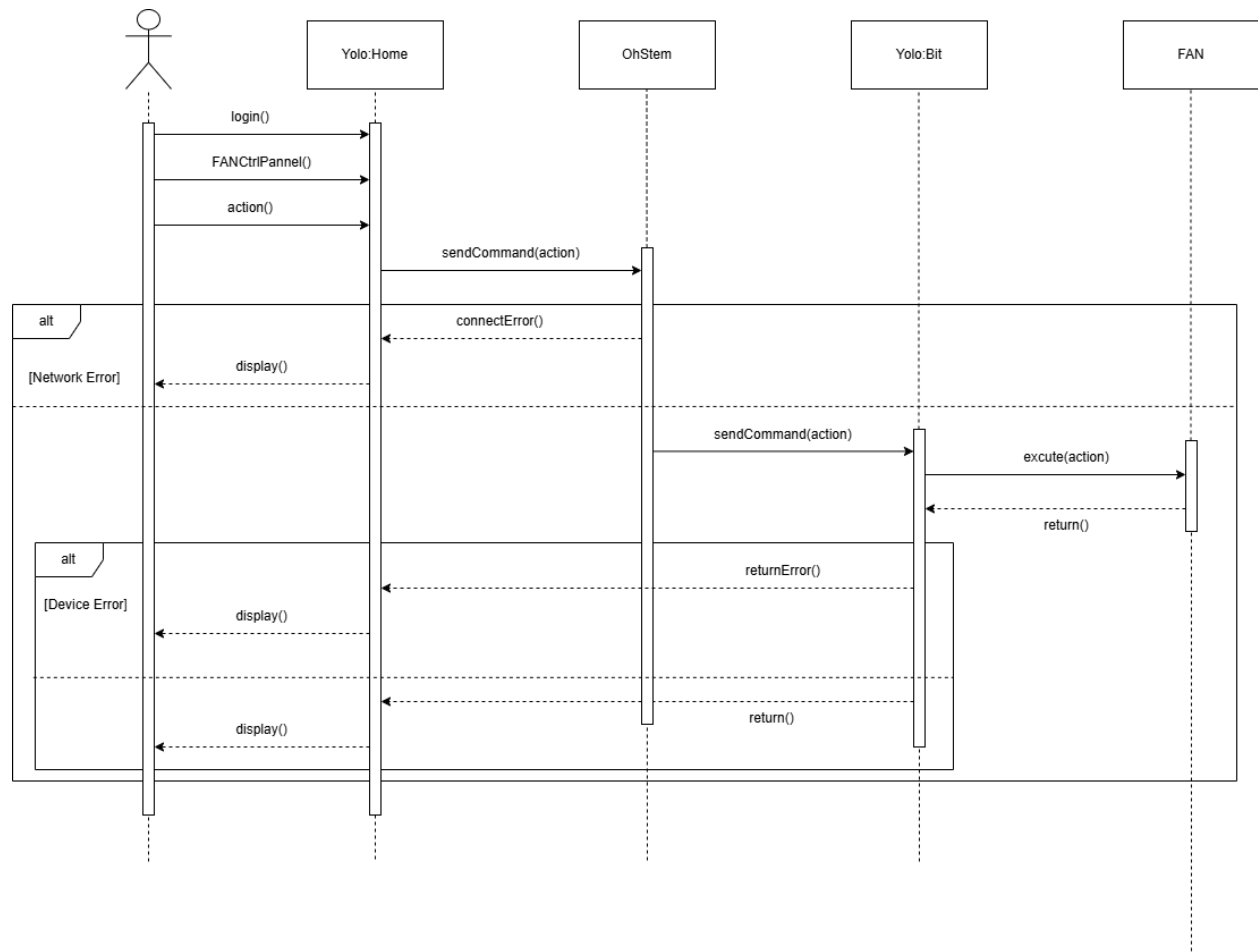


Figure 6.13: Sequence Diagram of the Fan Control via Web App Module

The sequence diagram illustrates how to control a fan module through a web or mobile interface involving the User, the Yolo:Home(Web Application), OhStem(IoT Server), Yolo:Bit, and the Fan module. The process is as follows:

- The user access YoloHome application and performs the login() action. Once logged in successfully, the user then navigates to FanCtrlPanel() within YoloHome.
- The user sees the list of available Fans and their status (on/off/speed) and controls for adjusting them.
- The User choose a fan and select a specific action() from the panel:
 - Calls toggleON() to turn On the Fan
 - Calls toggleOFF() to turn Off the Fan.
 - Calls setSpeed(level) to adjust the fan's speed.

- The user selects a specific action to control the fan
- YoloHome sends the command to the OhStem server via `sendCommand(action)`. The server (OhStem) receives this request and forwards it to the YoloBit device, again using `sendCommand(action)`.
- YoloBit executes the command `execute(action)` on the Fan, turning it on/off or adjusting the speed, and then returns the result back to OhStem.
- OhStem relays the result to YoloHome, which updates the user interface to reflect the fan's new status
- If there is a network error between YoloHome and OhStem, YoloHome displays an error message ("Network Error") and the command is not executed.
- If YoloBit or the Fan fails to respond, the system returns an error, and YoloHome displays a "Device Error" message to inform the user.

the diagram demonstrates how the system provides real-time feedback to the user while effectively handling and reporting any network or device errors, ensuring a reliable and seamless control experience.

6.4.9 Module 8: Monitoring Sensors by Web App

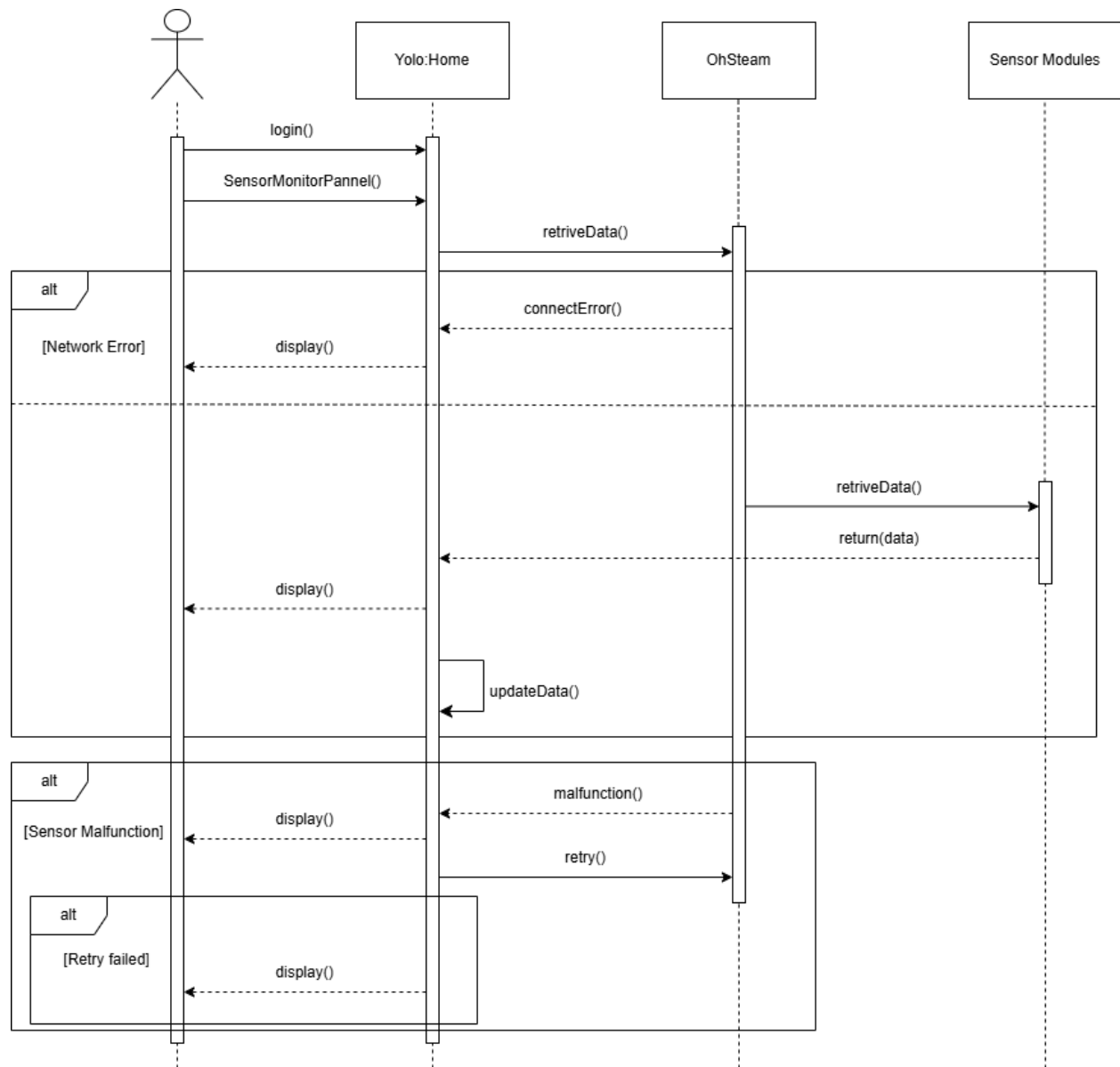


Figure 6.14: Sequence Diagram of the Monitoring Sensors by Web App Module

The sequence diagram illustrates how the user, via the YoloHome application, requests sensor data from various sensor modules through the OhSteam server, and how the system handles potential errors:

- The user access YoloHome application and performs the login() action. Once logged in successfully, the user then navigates to SensorMornitoringPannel() within YoloHome.
- YoloHome displays a dashboard for monitoring sensor data (e.g., temperature, humidity, etc.).

- YoloHome calls `retrieveData()` on the OhStem server. OhStem forwards the request to the appropriate sensor modules by also calling `retrieveData()`.
- The sensor modules gather the requested data and return it to OhStem, which then returns it to YoloHome.
- Upon receiving the data, YoloHome updates the dashboard via `updateData()`, allowing the user to view the latest sensor readings.
- If a network issue occurs between YoloHome and OhStem, a `connectError()` is returned. YoloHome calls `display()` to show an error message (“Network Error”) to the user.
- If a sensor module fails or does not respond, OhStem returns an error (`malfunction()`), and YoloHome displays a relevant error message (“Sensor Malfunction”) to inform the user.
- The user can initiate a `retry()` action (e.g., re-request data), prompting YoloHome to call `retrieveData()` again. If the retry attempt also fails, YoloHome displays a continued malfunction status or prompts the user to check the sensor hardware.

The diagram demonstrates how the system periodically or on-demand retrieves sensor data, updates the user interface in real time.

Chapter 7

Implementation

7.1 YOLO

You Only Look Once (YOLO) is a object detection first introduced 2015 and was revolutionary in the field of computer vision due its application of nerural network. Since YOLO introduction in 2015, many parties, were involved in the improvement of the model, leading the creation of YOLOv4, YOLOv5, YOLOv8,... One of the most noticable contributor to YOLO with their own spin is Ultralytics, who released YOLOv5, this particualr version of YOLO was ported Pytorch. In late 2024, Ultralytics announced YOLOv11 with an im-prove backbone, to ensure that our project perform the best, we will ultize the strength of YOLOv11.

In additon to working on their own version of YOLO model, Ultralytics has also developed a Python library that streamlines the training and deployment of YOLO models, including YOLOv11. This library provide both command-line and Python, to perform various tasks such as training, validation, prediction, and model export. The training and inferencing process is compacted as Ultralytics also provide a [tutorial notebook](#) allowing us to make small adjustment to their notebook to create a model for our own use cases instead of having to figure out the training process from scratch. Another feature of this library is that it allow us to export the mode in multiple deployment formats, making it highly compatible with the library used in the frontend of our project.

7.2 Data

To train an effective object detection model, especially one that's critical like detecting fires on home, we need access to high-quality, well-annotated. For this purpose, we will look into Roboflow, a platform designed to streamline the dataset management process for computer vision tasks. Roboflow allows users to export datasets in a variety of formats compatible with YOLO, Another part of its strength is that it is well intergrated with the previously mentioned Ultralytics. With the help of Roboflow, we able to identify and select

the `[amenazas__dataset]` that displayed good performance metrics on detecting fire without having to scavenge on the Internet.

7.3 Data Preprocessing and Augmentation

7.3.1 Motivation and Reasoning

In computer vision tasks such as fire detection, model performance heavily depends on the quality and quantity of the dataset. The original dataset from Roboflow consists of 1596 images for train, 521 images for validation and 219 images for test.

While the dataset is annotated and structured, the number of training samples is relatively small for training a deep learning model such as YOLOv11. To address this limitation and improve the model's generalization capability, we apply data augmentation techniques.

Data augmentation synthetically increases the size and diversity of the training data by applying a series of random transformations to existing images. This not only helps mitigate overfitting but also enhances robustness to various conditions (e.g., orientation, blurring).

7.3.2 Some Advantages of Preprocessing and Augmentation

- **Improved Generalization:** Augmentations introduce variability in the dataset, helping the model generalize better to unseen images.
- **Combating Overfitting:** With limited data, models tend to memorize training examples. Augmentation helps reduce this risk.
- **Robust Feature Learning:** Techniques like flipping or rotation force the model to focus on essential features that remain invariant.
- **Data Efficiency:** Effectively multiplies the available dataset without requiring manual labeling of new images.

7.3.3 Implementation Overview

We implement preprocessing and augmentation using the `albumentations` library.

1. Define paths for saving augmented images and labels.
2. Create necessary directories if they don't already exist.
3. Define augmentation pipelines such as **horizontal flipping**, **90-degree rotation**, and **Gaussian blur**.
4. For each image in the training set:

- Copy the original image and label to the new augmented dataset folder.
- Apply each defined augmentation.

7.3.4 Code Snippet for Preprocessing

Listing 7.1: Data Augmentation using Albumentations

```

1 # New augmented dataset paths
2 train_new_images_path = os.path.join(dataset_path, "train_new/images")
3 train_new_labels_path = os.path.join(dataset_path, "train_new/labels")
4
5 # Create new directories if they don't exist
6 os.makedirs(train_new_images_path, exist_ok=True)
7 os.makedirs(train_new_labels_path, exist_ok=True)
8
9 # Define augmentation transformations using A.Compose()
10 augmentations = {
11     "flip": A.Compose([
12         A.HorizontalFlip(p=1.0)
13     ], bbox_params=A.BboxParams(format="yolo", label_fields=["class_labels"])),
14
15     "rotate": A.Compose([
16         A.RandomRotate90(p=1.0)
17     ], bbox_params=A.BboxParams(format="yolo", label_fields=["class_labels"])),
18
19     "blur": A.Compose([
20         A.GaussianBlur(blur_limit=(3, 7), p=1.0)
21     ], bbox_params=A.BboxParams(format="yolo", label_fields=["class_labels"])),
22 }

```

7.3.5 Detailed Explanation of Code and Image Transformations

- **Step 1 - Define New Output Paths:**

The variables `train_new_images_path` and `train_new_labels_path` specify the locations where all augmented images and their corresponding YOLO-format label files will be stored. These are kept separate from the original training data to avoid overwriting.

- **Step 2 - Create Necessary Folders:**

`os.makedirs(...)` is used to ensure that the output directories exist. If they do not, they will be created. This guarantees that the program does not fail due to missing paths when saving augmented files.

- **Step 3 - Define Augmentation Strategies:**

The dictionary `augmentations` maps the name of each augmentation to a specific

transformation pipeline. These transformations are defined using `A.Compose()` from the `albumentations` library, with the following actions:

- **flip:** Applies a horizontal flip to the image and adjusts all bounding box coordinates accordingly.
- **rotate:** Applies a 90-degree clockwise rotation. This simulates camera orientations and boosts model robustness.
- **blur:** Adds Gaussian blur to the image, simulating foggy or low-quality camera feeds.

The use of `bbox_params` with `format="yolo"` ensures that all bounding box annotations are updated properly when images are transformed.

- **Step 4 - Iterate Over Original Training Images:**

The code reads all image files from the training set. For each image:

1. **Original Preservation:** The image and its label file are duplicated to the new dataset folder to preserve the original data.
2. **Augmentation Execution:** Each augmentation is applied using a helper function `apply_augmentation()`. This function:
 - Loads the image and its corresponding bounding boxes.
 - Applies the specified augmentation transformation.
 - Adjusts bounding boxes to match the new image.
 - Saves the transformed image and label file with a suffix indicating the augmentation type (e.g., `image_flip.jpg`, `image_blur.jpg`).

- **Goal of This Process:**

The augmented data increases both the size and variability of the training dataset. This helps the YOLOv11 model learn more robust features and become less sensitive to variations in angle, lighting, and clarity — which are all common in real-world fire detection environments.

7.3.6 Visual Example of Augmented Images

Below is a visual example showing how different augmentation techniques are applied to a single fire detection image. Each sub-image demonstrates how the fire instance and its bounding box are transformed under various conditions including flipping, rotation, blurring, and combinations of these augmentations.

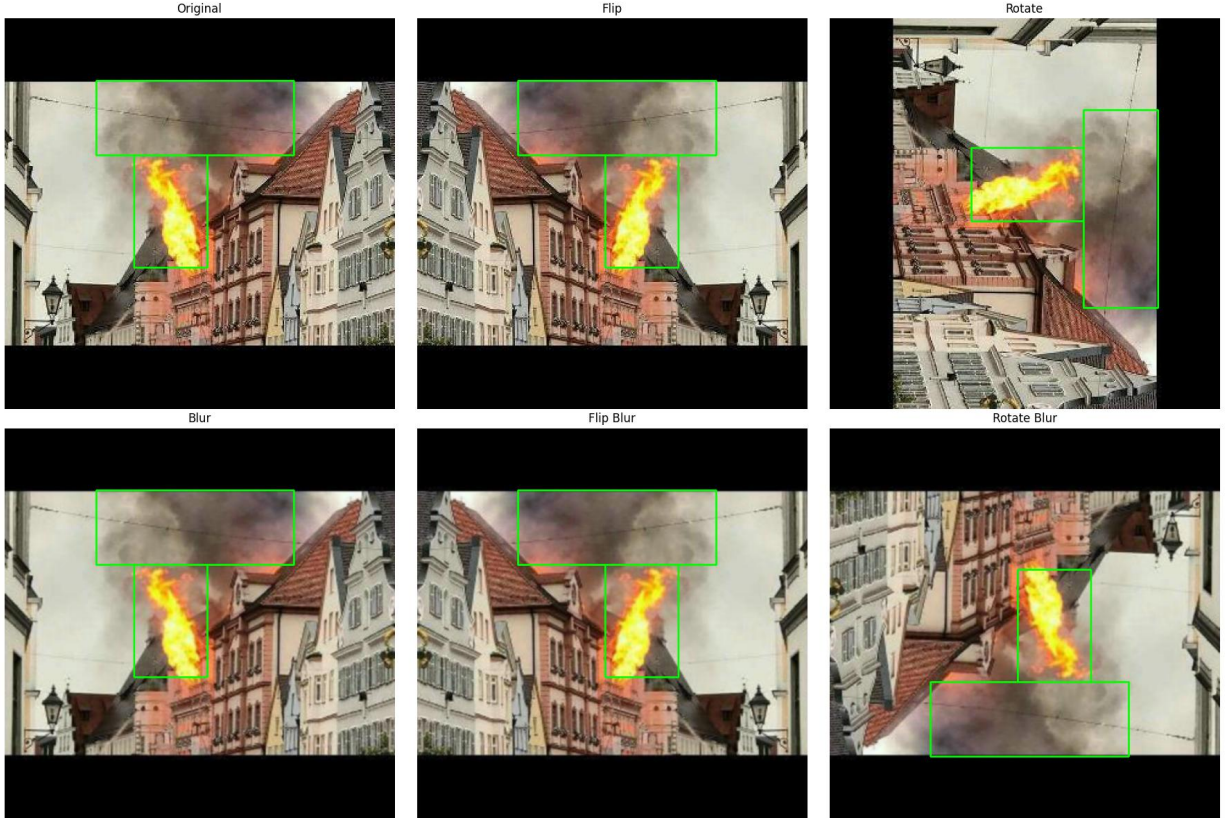


Figure 7.1: Comparison of augmentation techniques: Original, Flip, Rotate, Blur, Flip+Blur, and Rotate+Blur. Each image retains proper bounding box alignment, showcasing the effectiveness of annotation-preserving transformations.

7.4 Model Training and Evaluation

7.4.1 Training Configuration

The fire detection model was trained using the **YOLOv11n**(Nano) architecture provided by the Ultralytics YOLOv8 framework, version 8.3.93. In addition to **YOLOv11n**, we also tinkered with other version of YOLO such as: **YOLOv10n** and **YOLOv12n** to evaluate their relative performance the fire detection dataset.

The training process was executed on a **Tesla P100-PCIE-16GB GPU** with CUDA support and **PyTorch 2.5.1+cu121**, utilizing mixed precision (AMP) to accelerate performance and reduce memory usage.

The model was trained on the augmented dataset, consisting of:

- **Train set:** 6384 images (augmented from the original 1596)
- **Validation set:** 521 images

Training Parameters:

- **Model:** YOLOv11n, YOLOv10n, YOLOv12n, (yolo11n.pt, pretrained on COCO with 80 classes)
- **Transfer Learning:** Yes (COCO weights, overridden to 1 class)
- **Epochs:** 10
- **Image size:** 640×640
- **Batch size:** 16
- **Optimizer:** AdamW (auto-selected), lr=0.002, momentum=0.9
- **Loss functions:** Box loss, Classification loss, and Distribution Focal Loss (DFL)
- **Augmentation:** RandAugment, Mosaic, Horizontal Flip (0.5), Translate (0.1), Scale (0.5), CLAHE, Blur, MedianBlur, AutoErase, etc.
- **AMP:** Enabled (Automatic Mixed Precision)
- **Freeze:** The last DFL conv layer was frozen (model.23.dfl.conv.weight)
- **Logging:** TensorBoard integration enabled

Due to the slight difference between the architecture of the training pipeline of each model, there is a small deviation between the training command between YOLOv12, YOLOv10, and YOLOv11. To put it simply, the training command for YOLOv11 can be directly run from the terminal while the other models requires the user to write a small Python script to initiate the training process. The training commands used were:

- **Model Version: v11**
 - !yolo train model=yolo11n.pt data=.../data.yaml epochs=10 imgsz=640
 - Trained with interactive terminal
- **Model Version: v10**
 - model = YOLO('yolov10n.pt').to(device)
 - model.train(data="/kaggle/working/dataset_folder/data.yaml", epochs=10, imgsz=640, batch=16)
 - Trained with Python script
- **Model Version: v12**
 - model = YOLO('yolov12n.pt')
 - model.train(data="/kaggle/working/dataset_folder/data.yaml", epochs=10, imgsz=640, batch=16)
 - Trained with Python script

7.4.2 Model Architecture Overview

The YOLOv11n backbone follows a highly efficient and lightweight design with **181 layers** and approximately **2.59 million parameters** (6.4 GFLOPs). It was designed for real-time applications and mobile deployment. The architecture consists of the following key components:

- **C3k2 Blocks:** Lightweight C3 modules for feature extraction at various scales. For example:
 - [32, 64, 1, False, 0.25] in shallow layers for speed
 - [384, 256, 1, True] in deeper blocks for semantics
- **SPPF (Spatial Pyramid Pooling - Fast):** Enhances the receptive field at no additional computational cost. It is applied after the final backbone convolutional block with a kernel size of 5.
- **C2PSA Block:** A channel-wise attention block that enables the model to focus on more informative spatial and channel features before detection.
- **Concat and Upsample Operations:** Used to build a feature pyramid and enable multi-scale detection through top-down and lateral connections.
- **Detect Head:** The final head predicts bounding boxes at 3 scales using strides of 8, 16, and 32 pixels. Anchors are eliminated; prediction is anchor-free. The channels are set to [64, 128, 256].
- **Parameter Summary:**
 - **Total layers:** 181
 - **Trainable params:** 2,590,019
 - **Frozen params:** 16 (1 layer frozen)
 - **FLOPs:** 6.4 GFLOPs

Layer-by-Layer Highlights:

- Initial convolutional blocks downsample the input from 3 to 256 channels through a stack of Conv and C3k2 modules.
- The middle backbone applies the SPPF block and C2PSA attention module to enrich context.
- The head connects features across 3 layers (16, 19, 22) for multi-scale detection using the final Detect layer.

This design balances accuracy and efficiency, making YOLOv11n highly suitable for real-time fire detection, especially in edge devices or embedded systems.

7.4.3 Training Progress and Metrics

Trainings

During training process of the YOLO models, both loss values and evaluation metrics were monitored through out the 10 epochs. The model was trained on augmented data using the Ultralytics YOLOv8 framework with automatic mixed precision (AMP) and AdamW optimizer. The evolution of training and validation performance for YOLOv10, 11 and 12 is shown in Figure 7.2, 7.3 and 7.4 respectively.

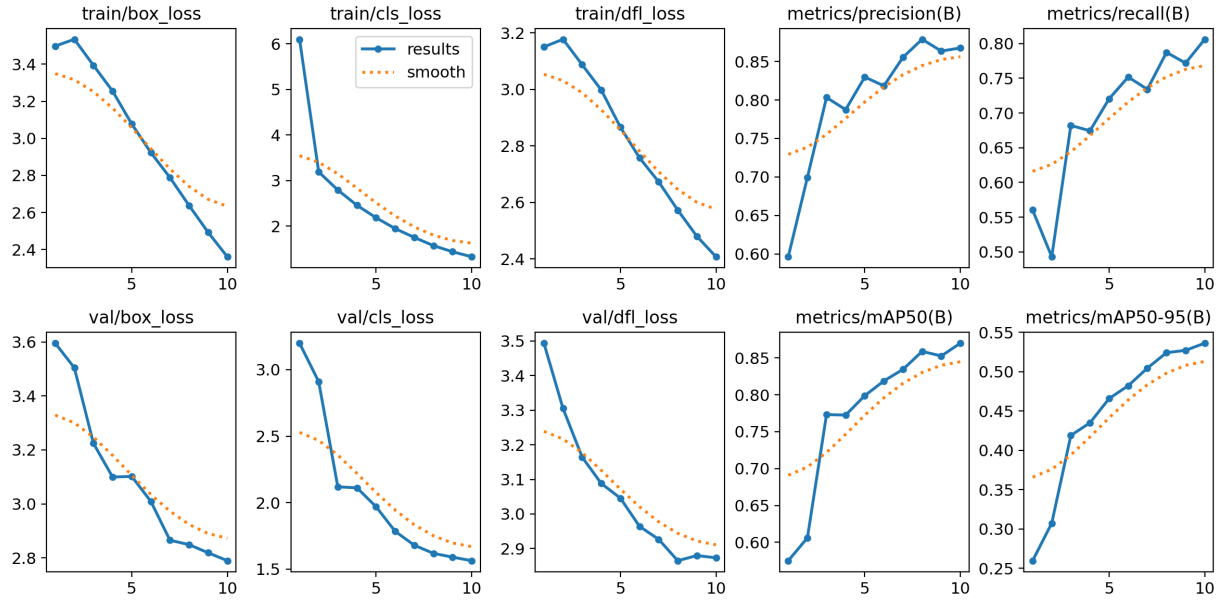


Figure 7.2: Training progress for **YOLOv10** across 10 epochs.

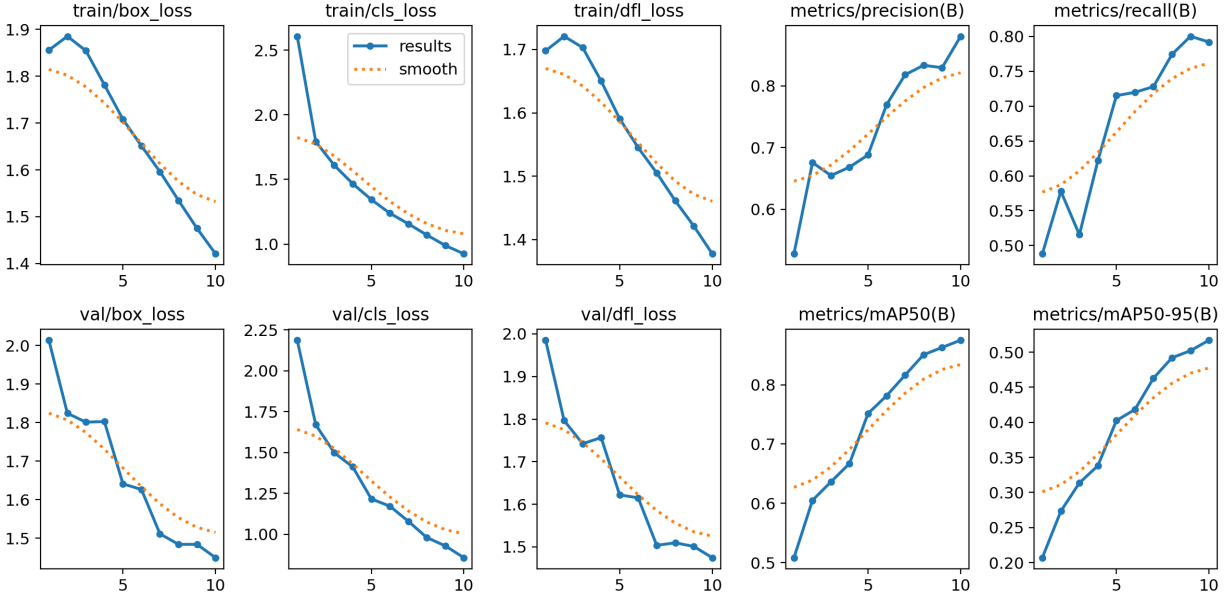


Figure 7.3: Training progress for **YOLOv11** across 10 epochs.

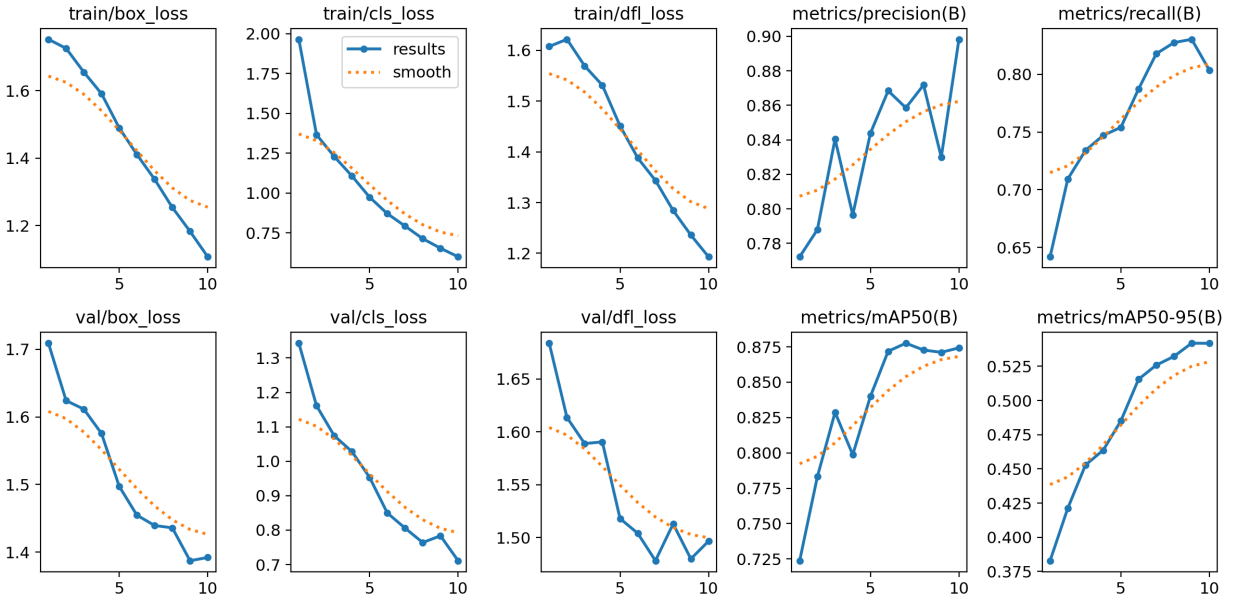


Figure 7.4: Training progress for **YOLOv12** across 10 epochs.

YOLOv10

Training Losses:

- **Box Loss:** Decreased from 3.498 to 2.361, showing better bounding box localization.
- **Classification Loss:** Dropped from 6.097 to 1.327, indicating substantial improvement in fire classification.

- **Distribution Focal Loss (DFL):** Reduced from 3.150 to 2.407, reflecting enhanced precision in bounding box regression.

Validation Losses:

- **Box Loss:** Decreased from 3.498 to 2.361
- **Cls Loss:** Decreased from 6.097 to 1.327
- **DFL Loss:** Decreased from 3.150 to 2.407

Evaluation Metrics (Validation set, 521 images, 846 instances):

- **Precision:** Improved from 0.596 (Epoch 1) to **0.868** (Epoch 10)
- **Recall:** Increased from 0.560 to **0.806**
- **mAP@0.5:** Increased from 0.575 to **0.870**
- **mAP@0.5:0.95:** Rose from 0.260 to **0.536**

YOLOv11

Training Losses:

- **Box Loss:** Reduced from 1.856 to 1.421, indicating improved bounding box accuracy.
- **Classification Loss:** Decreased from 2.608 to 0.9251, reflecting the model's improved ability to correctly classify fire instances.
- **Distribution Focal Loss (DFL):** Dropped from 1.698 to 1.378, showing better precision in bounding box regression.

Validation Losses:

- **Box Loss:** Decreased from 2.0 to 1.48
- **Cls Loss:** Decreased from 2.24 to 1.0
- **DFL Loss:** Decreased from 2.0 to 1.48

Evaluation Metrics (Validation set, 521 images, 846 instances):

- **Precision:** Improved from 0.528 (Epoch 1) to **0.874** (Epoch 10)
- **Recall:** Improved from 0.488 to **0.792**

- **mAP@0.5:** Increased significantly from 0.508 to **0.875**
- **mAP@0.5:0.95:** Grew from 0.207 to **0.517**

YOLOv12

Training Losses:

- **Box Loss:** Reduced from 1.753 to 1.107, showing improved localization of bounding boxes.
- **Classification Loss:** Dropped from 1.965 to 0.6007, indicating better fire class discrimination.
- **Distribution Focal Loss (DFL):** Decreased from 1.608 to 1.193, reflecting more accurate bounding box regression.

Validation Losses:

- **Box Loss:** Decreased from 1.753 to 1.107
- **Cls Loss:** Decreased from 1.965 to 0.6007
- **DFL Loss:** Decreased from 1.608 to 1.193

Evaluation Metrics (Validation set, 521 images, 846 instances):

- **Precision:** Improved from 0.772 (Epoch 1) to **0.898** (Epoch 10)
- **Recall:** Increased from 0.642 to **0.803**
- **mAP@0.5:** Jumped from 0.724 to **0.874**
- **mAP@0.5:0.95:** Rose from 0.383 to **0.542**

7.4.4 Evaluation and Diagnostic Curves

To further evaluate the model's behavior beyond just loss and mAP metrics, several diagnostic plots were generated from the validation phase.

Confusion Matrix

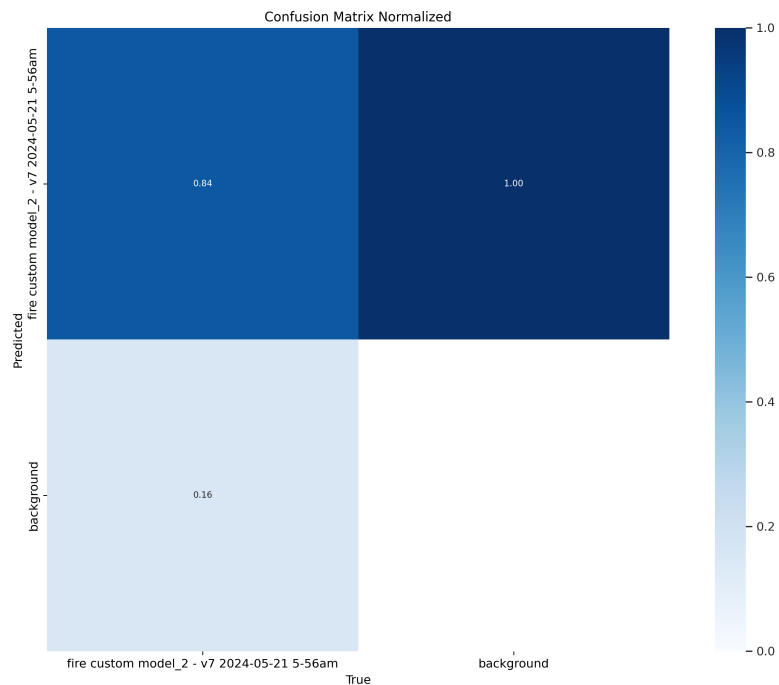


Figure 7.5: YOLOv10 — The confusion matrix shows that most fires were classified correctly, a noticeable portion of background elements were misclassified as fire, this pattern will be seen again in latter version of YOLO.

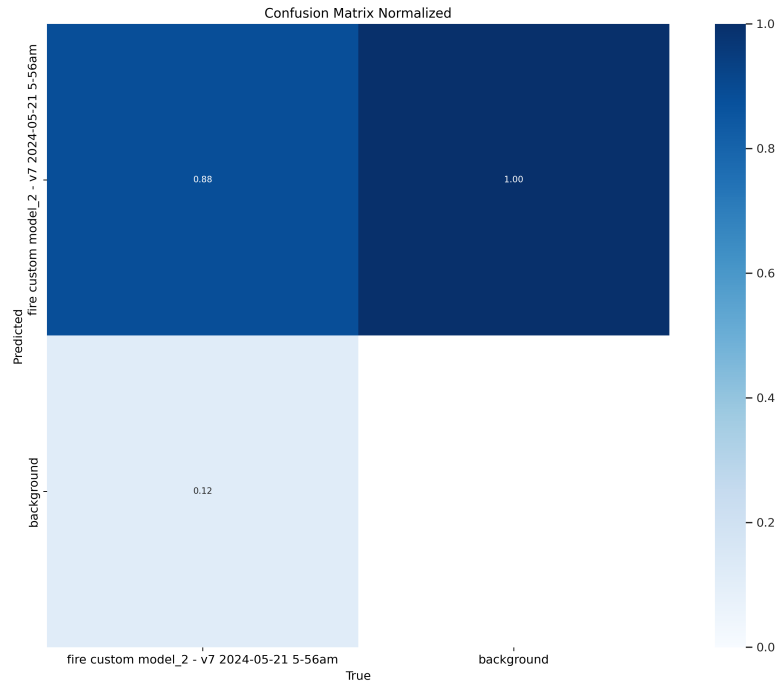


Figure 7.6: YOLOv11 - The matrix indicates that most fire instances were correctly classified. This is also the best performing model in detecting fire, disregarding false positive.

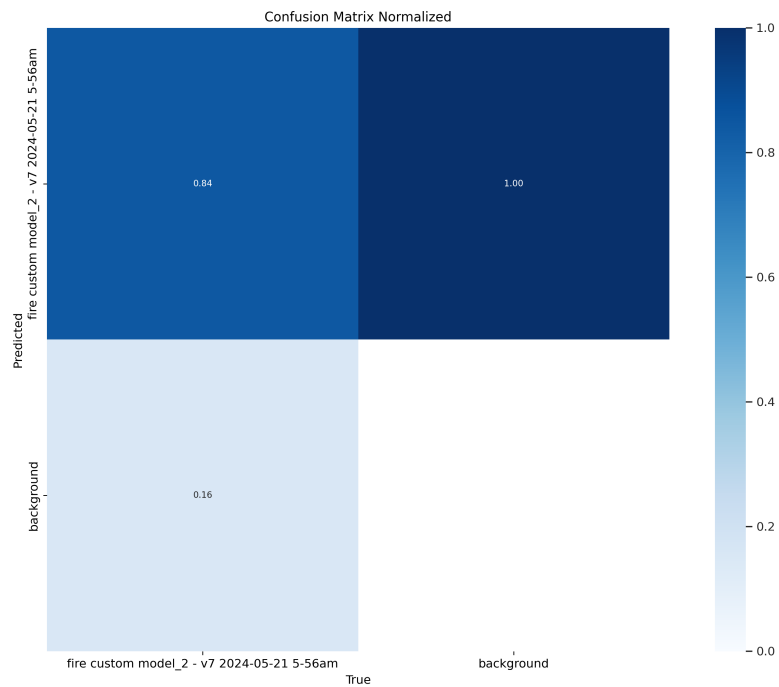


Figure 7.7: YOLOv12 — The latest YOLO model as of writing this report, however in terms of confusion matrix it is inferior compared to the confusion matrix of YOLOv11.

Curve Analysis

YOLOv10

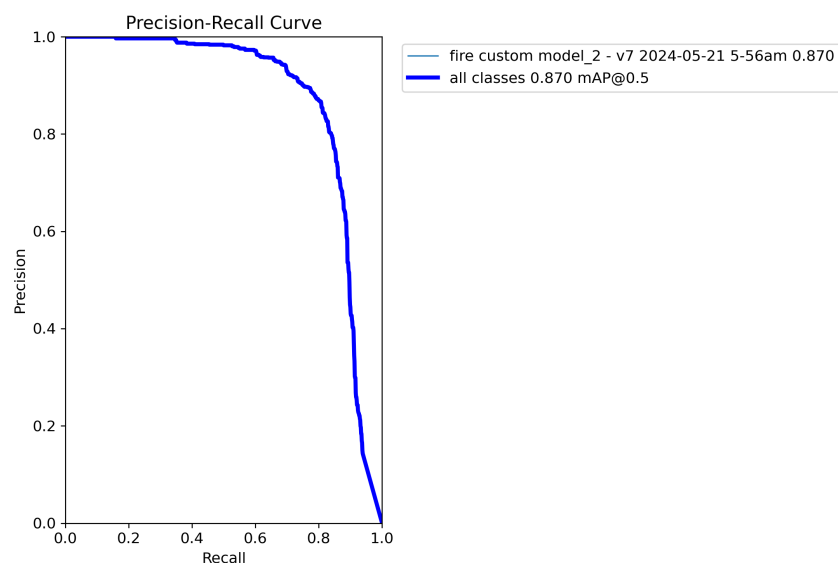


Figure 7.8: Precision–Recall (PR) Curve for YOLOv10. The model achieves a steady trade-off between precision and recall, with an $\text{mAP}@0.5$ of 0.870, showing acceptable detection performance despite being an earlier version.

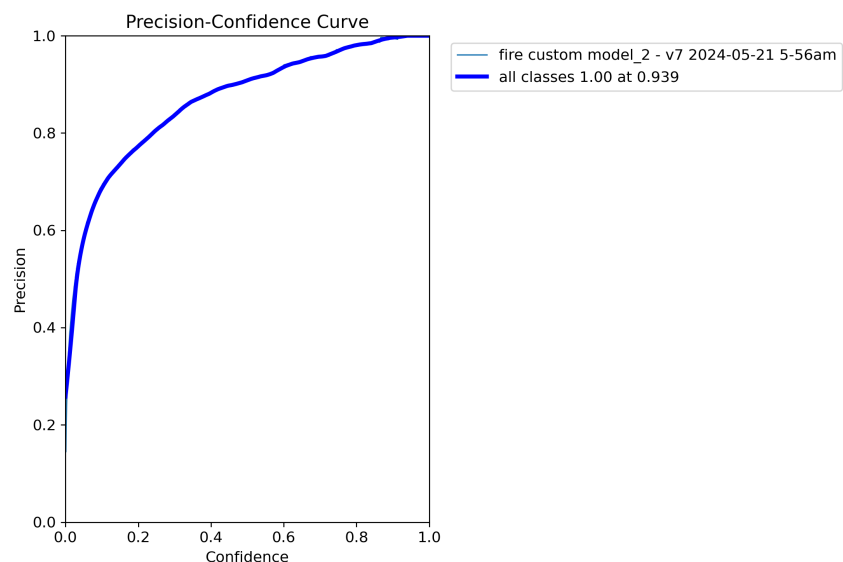


Figure 7.9: Precision vs. Confidence Curve. Precision increases as confidence grows, reaching 1.0 at a threshold near 0.939. This shows better reliability occurs when using stricter confidence thresholds.

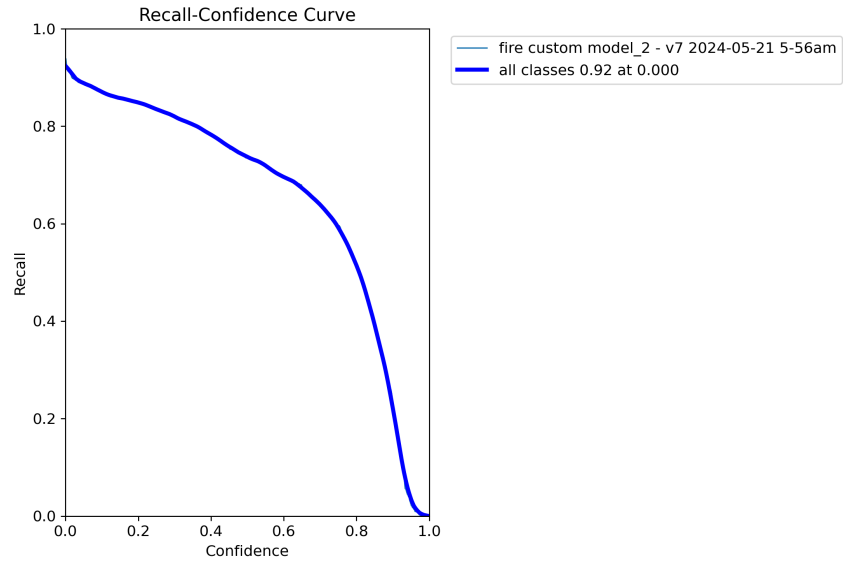


Figure 7.10: Recall vs. Confidence Curve. Recall starts high and drops with increasing confidence thresholds showing normal behavior

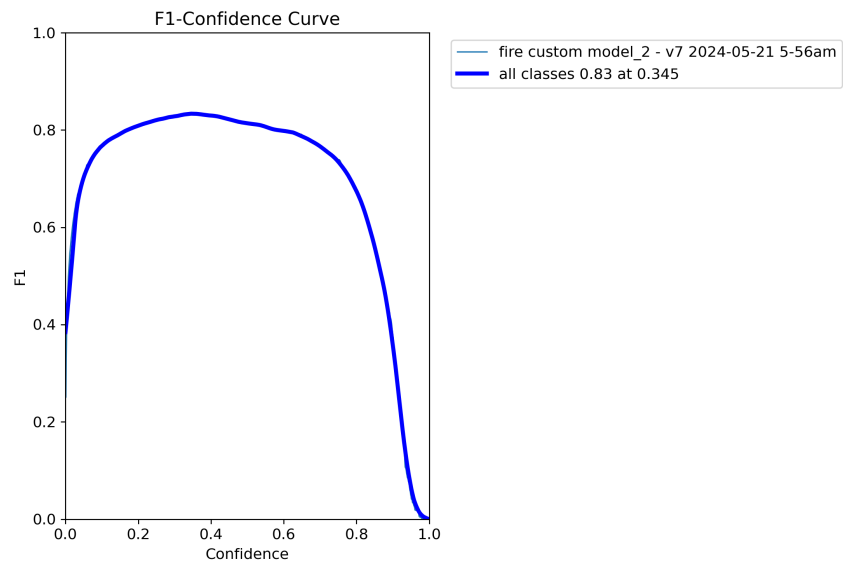


Figure 7.11: F1 Score vs. Confidence Curve. YOLOv10 achieves a maximum F1 score of 0.83 at a confidence threshold of approximately 0.345.

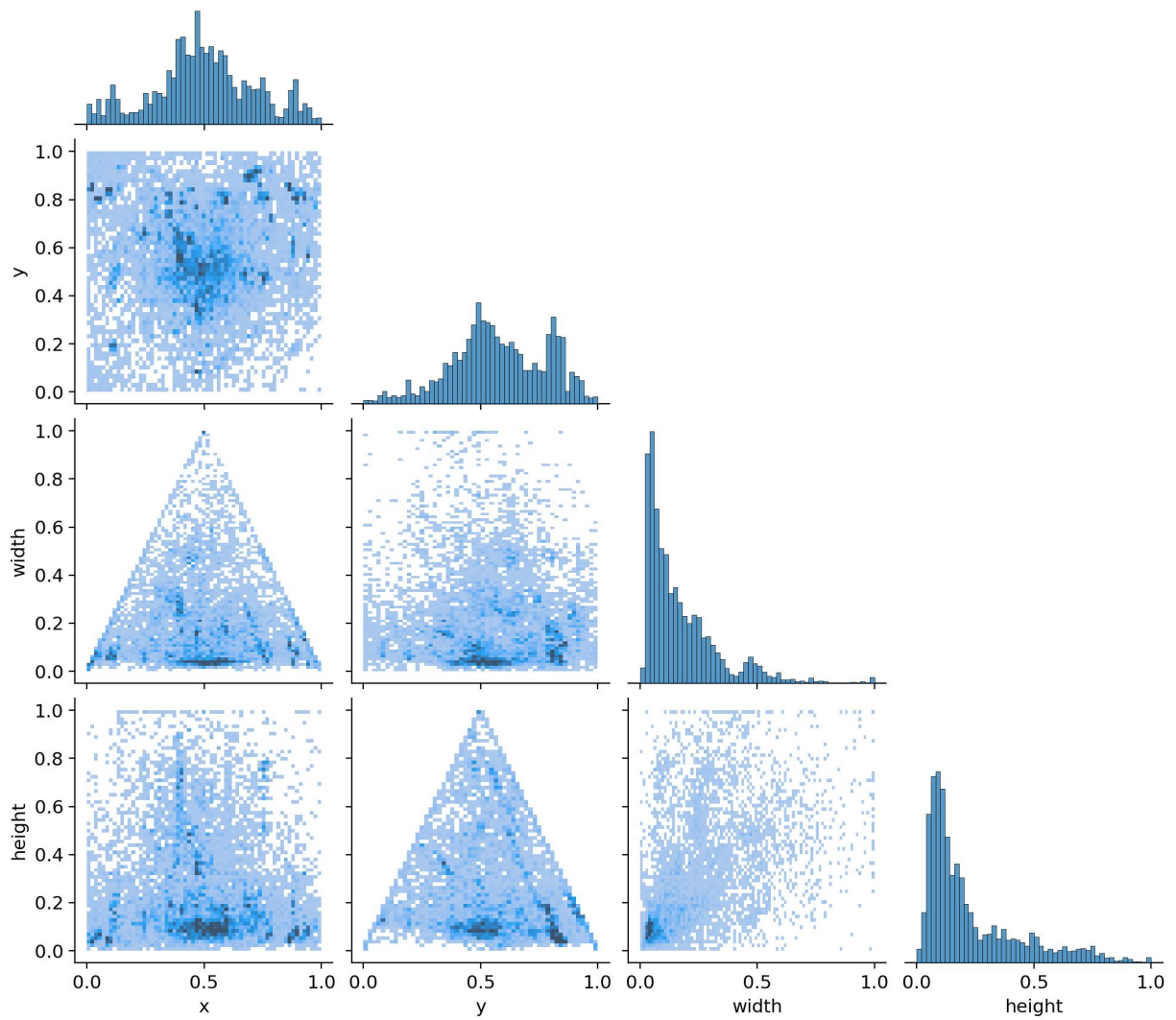


Figure 7.12: Label correlogram for YOLOv10, confirming consistent bounding box distribution for the fire class. The plot shows uniform labeling without noticeable outliers.

YOLOv11

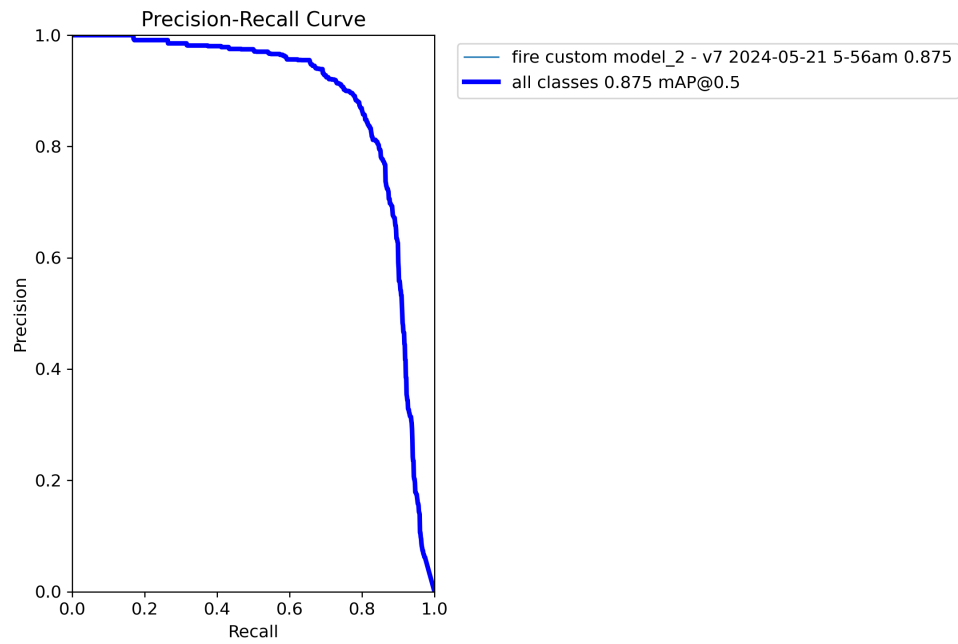


Figure 7.13: Precision–Recall (PR) Curve for the fire detection class. The model achieves high precision and recall across most thresholds, with an average mAP@0.5 of 0.875, indicating a reliable detection capability.

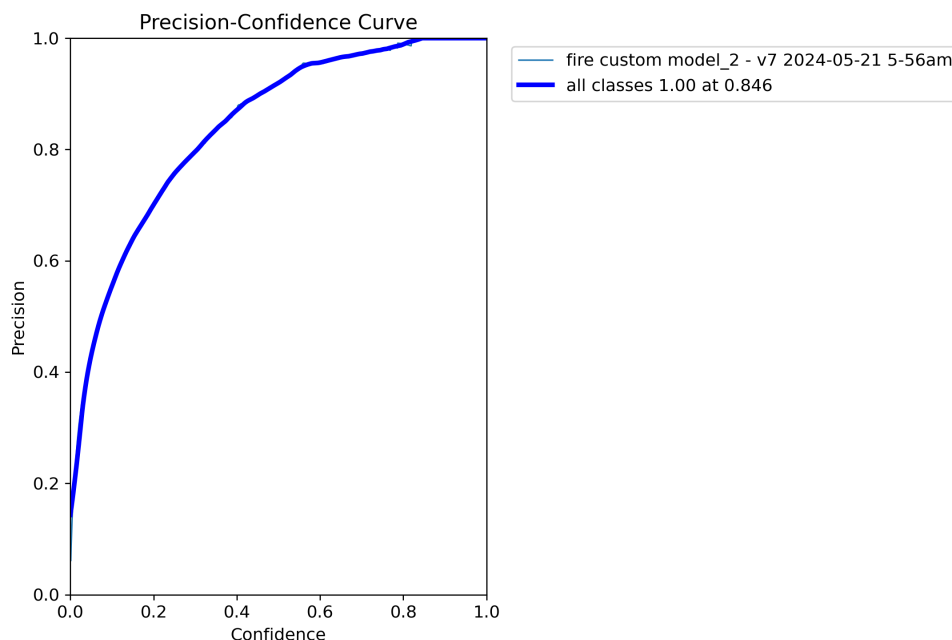


Figure 7.14: Precision vs. Confidence Curve. Precision improves consistently as the confidence threshold increases, reaching 1.0 precision at a threshold of 0.846. This is useful for setting a confidence cutoff during deployment.

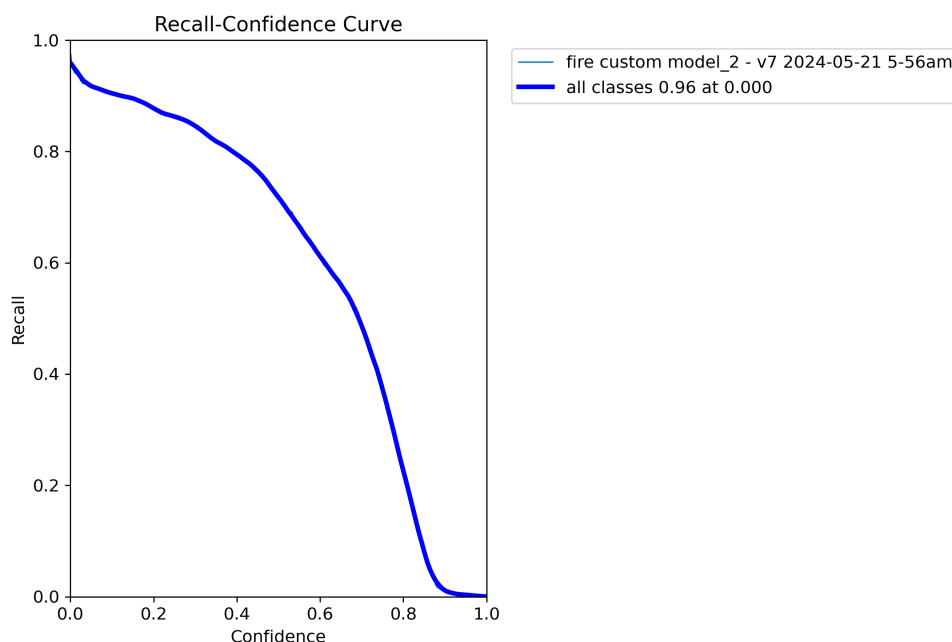


Figure 7.15: Recall vs. Confidence Curve. Recall decreases as confidence increases, which is expected. The curve helps understand the recall drop-off and optimal thresholds for maximizing recall in safety-critical applications.

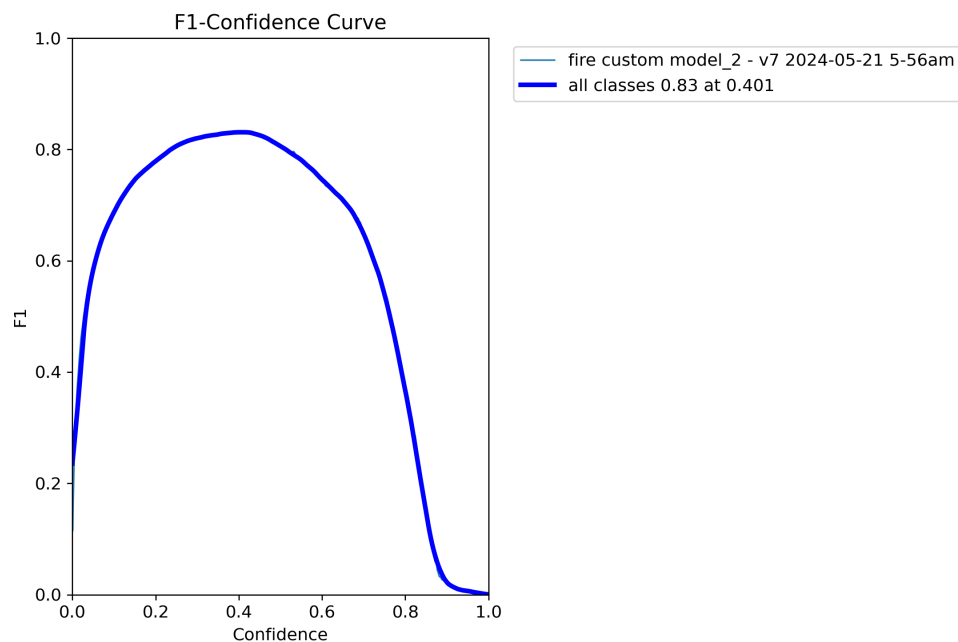


Figure 7.16: F1 Score vs. Confidence Curve. The optimal confidence threshold for the best F1 score (0.83) is approximately 0.401. This balance point reflects the best trade-off between precision and recall.

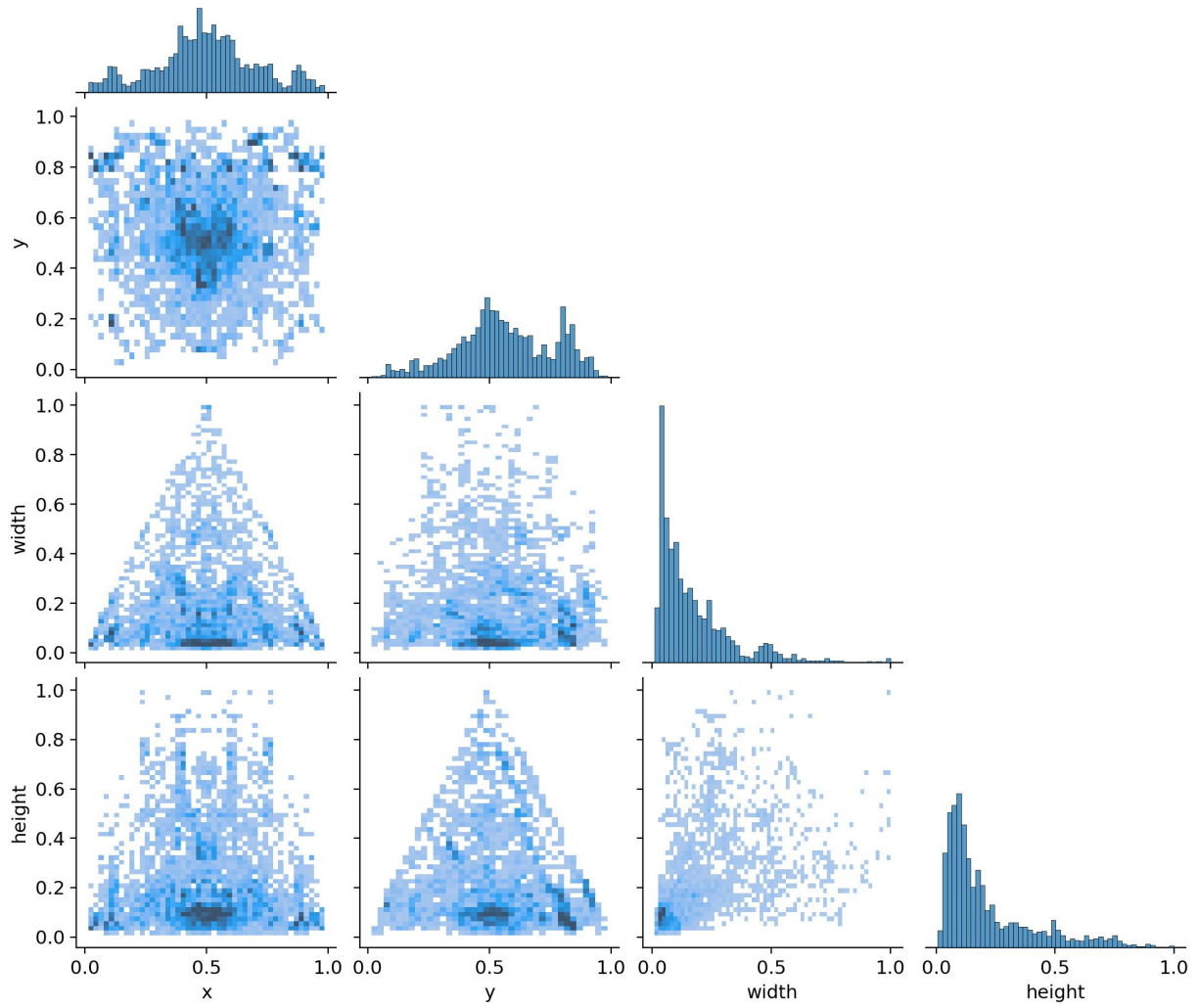


Figure 7.17: Label correlogram showing the frequency and co-occurrence of labels in the dataset. In this case, since we are detecting a single fire class, the plot confirms label consistency and absence of label noise.

YOLOv12

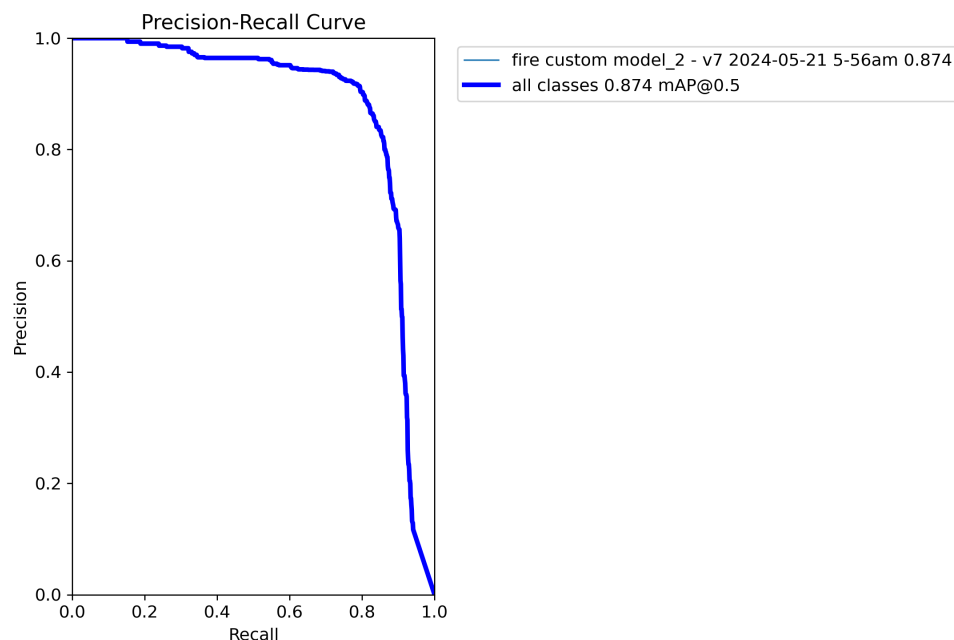


Figure 7.18: Precision–Recall (PR) Curve for YOLOv12. The model maintains high precision and recall for different thresholds, with a strong mAP@0.5 of 0.874, showing balanced accuracy.

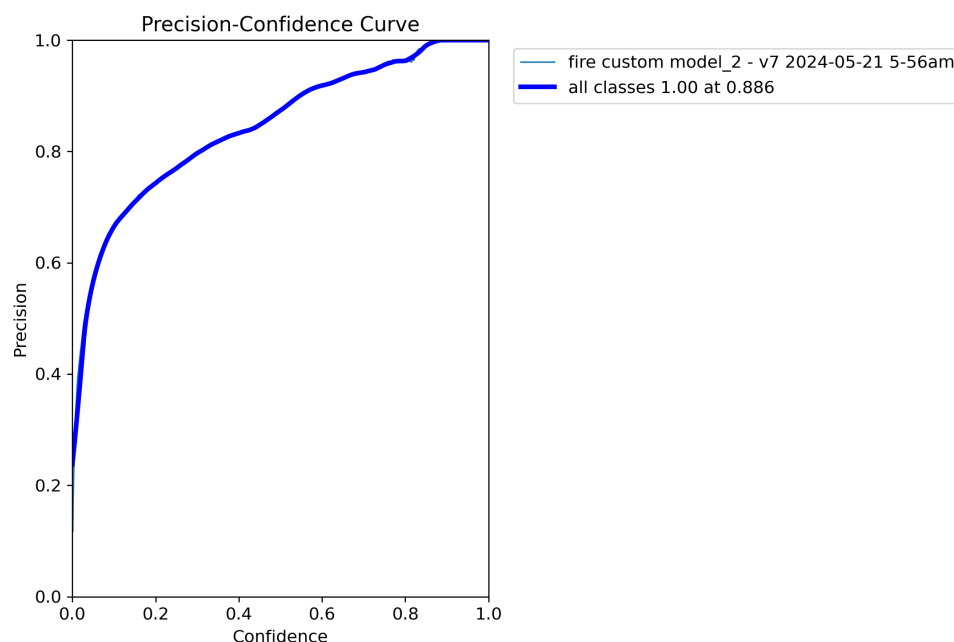


Figure 7.19: Precision vs. Confidence Curve. YOLOv12 achieves its absolute maximum precision at a confidence threshold of 0.886, showing its reliability under high confidence predictions.

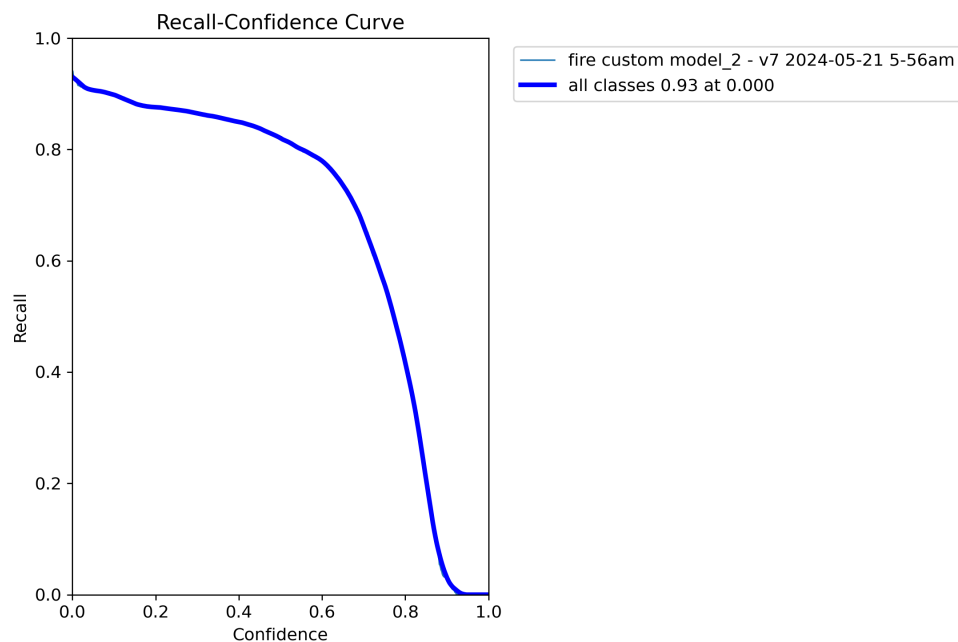


Figure 7.20: Recall vs. Confidence Curve. Recall topped out at around 0.93 at lower confidence levels, then slowly declines.

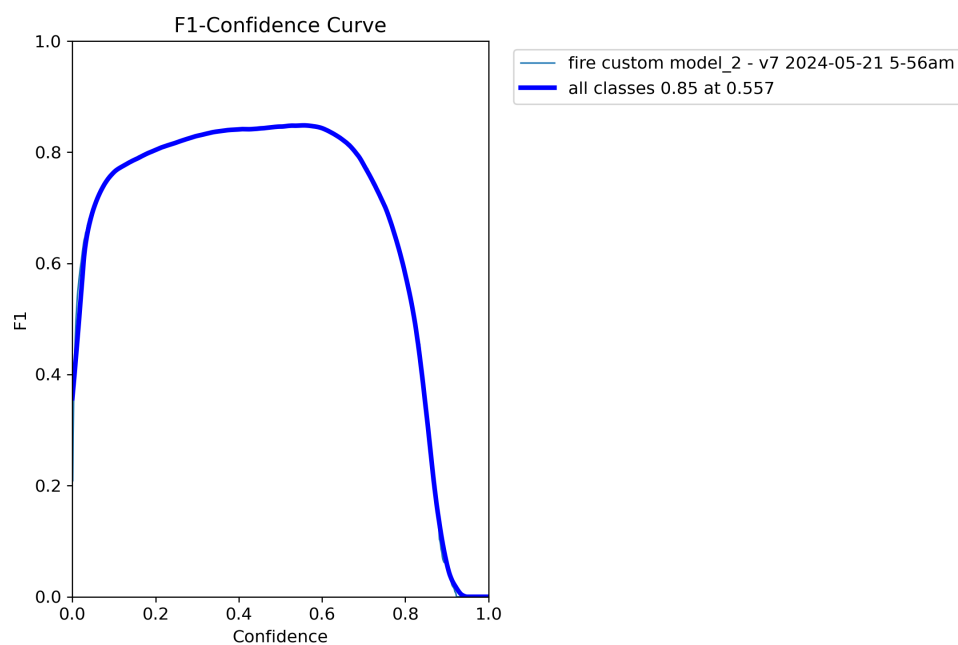


Figure 7.21: F1 Score vs. Confidence Curve. YOLOv12 achieves an great F1 score of 0.85 at a confidence threshold of 0.557.

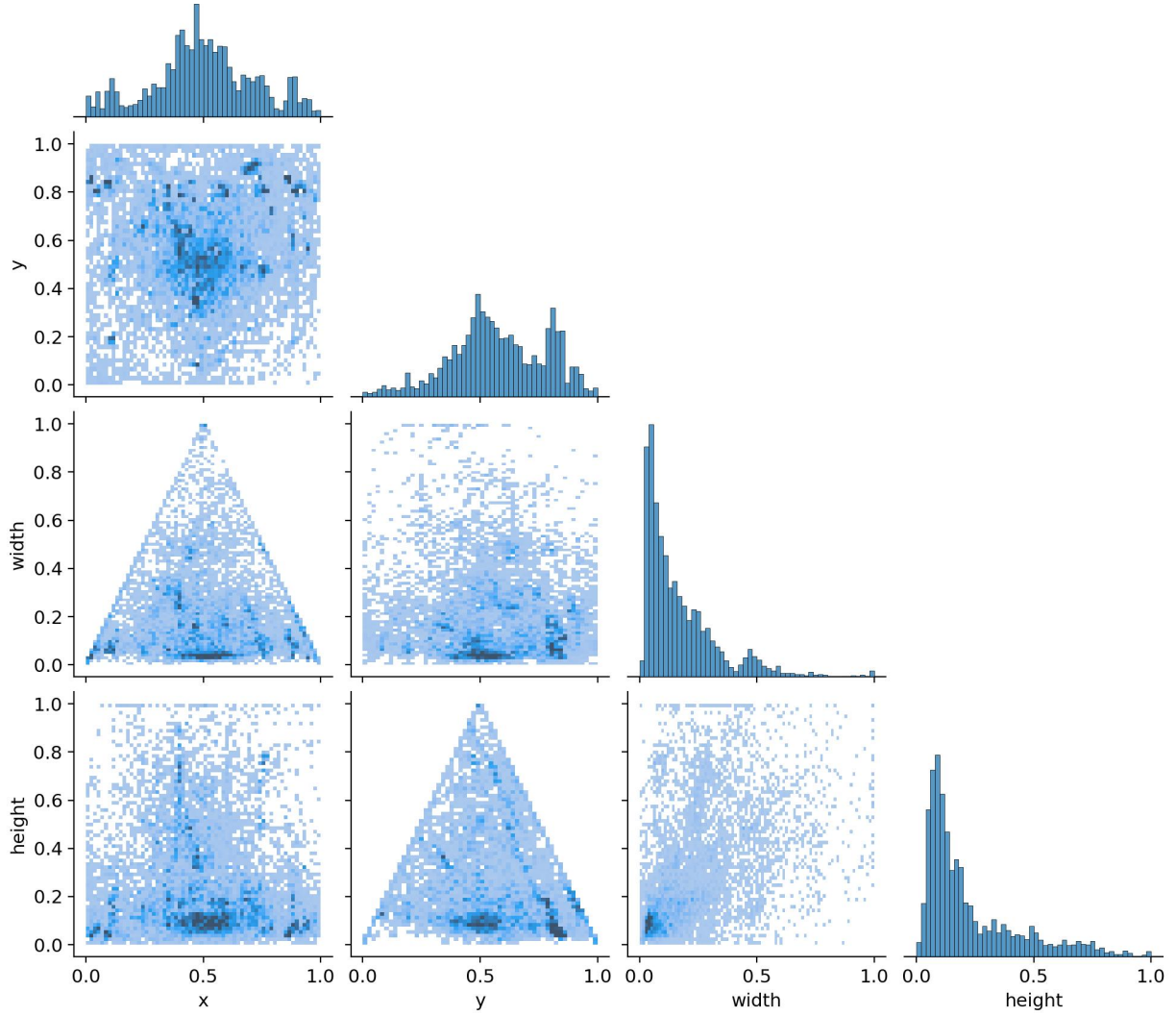


Figure 7.22: Label correlogram for YOLOv12. This visualization show consistent label distribution for the fire class.

7.4.5 Conclusion

The YOLOv11n model demonstrates excellent performance on the fire detection dataset, achieving a **mAP@0.5 of 87.5** and **mAP@0.5:0.95 of 51.7** in only 10 epochs. The training and validation loss curves suggest smooth convergence without overfitting. Diagnostic plots further confirm that the model generalizes well and is suitable for deployment in real-time fire detection systems.

7.5 Testing and Inference

After completing training of each Yolo Models (v10, v11, and v12), their best-performing model checkpoint (`best.pt`) was evaluated on the entire test set. Inference was performed using the official Ultralytics API for YOLO-based models. The model was loaded onto the available device and tested on the first 10 test images. The inference process involved loading images, applying the trained model, extracting predictions, drawing bounding boxes, and saving the visual outputs.

7.5.1 Model YOLOv10

7.5.1.1 Quantitative Evaluation

Following a comprehensive evaluation on the test set, our YOLOv10 model demonstrated strong performance across all key metrics, summarized as follows:

- **Precision:** 0.8498
- **Recall:** 0.8333
- **F1-Score:** 0.8415
- **mAP@0.5:** 0.9935

The high precision score of 0.8498 indicates the model’s ability to accurately detect fire instances with minimal false positives. The recall score of 0.8333 shows that the model successfully identifies a majority of true fire instances present in the test data. With an F1-score of 0.8415, the model achieves a favorable balance between precision and recall, suggesting it effectively detects fire regions without excessive false positives or false negatives.

Moreover, the mAP@0.5 score of 0.9935 highlights the model’s exceptional localization accuracy, confirming its robustness in accurately detecting fire instances regardless of scale, background complexity, or lighting conditions. Overall, the YOLOv10 model is both reliable and efficient for fire detection across diverse scenarios.

7.5.1.2 Qualitative Inference Results

The following figure shows qualitative detection examples of the model on unseen test images. The bounding boxes display the model’s predictions for fire locations, along with associated confidence scores.



Figure 7.23: Qualitative inference results. Each quadrant displays one example of fire detection from the test set. The model accurately detects both large and small fire regions, even under different lighting and background conditions.

7.5.2 Model Yolov11

7.5.2.1 Quantitative Evaluation

Following a comprehensive evaluation on the test set, our YOLOv11 model demonstrated strong performance across all key metrics, summarized as follows:

- **Precision:** 0.8254
- **Recall:** 0.8460
- **F1-Score:** 0.8355
- **mAP@0.5:** 0.9935

These metrics indicate that the YOLOv11 model is **highly accurate** and **robust** in detecting fire across diverse scenarios. A precision of 0.8254 minimizes false positives, while a recall of 0.8460 captures most fire instances. The F1-score of 0.8355 maintains a good balance between precision and recall, and the mAP@0.5 of 0.9935 confirms exceptional localization accuracy under various conditions.

7.5.2.2 Qualitative Inference Results

The following figure shows qualitative detection examples of the model on unseen test images. The bounding boxes display the model's predictions for fire locations, along with associated confidence scores.



Figure 7.24: Qualitative inference results. Each quadrant displays one example of fire detection from the test set. The model accurately detects both large and small fire regions, even under different lighting and background conditions.

7.5.3 Model Yolov12

The training process for the YOLOv12 model concluded with the selection of the most effective checkpoint (`best.pt`) following evaluation on the entire test dataset. Inference was carried out utilizing the Ultralytics API, designed specifically for YOLO models. The trained model was deployed on the available hardware and tested on the first 10 test images. This procedure involved loading the images, executing predictions via the trained model, generating bounding boxes, and saving the resulting visual outputs for analysis.

7.5.3.1 Quantitative Evaluation

Upon thoroughly evaluating the test set, the YOLOv12 model yielded the following performance metrics:

- **Precision:** 0.8156
- **Recall:** 0.8291
- **F1-Score:** 0.8223
- **mAP@0.5:** 0.9944

The YOLOv12 model demonstrates strong detection capabilities, achieving a precision of 0.8156 and a recall of 0.8291, effectively balancing false positives and true detections. The F1-score of 0.8223 indicates robustness across various scenarios. Its high mAP@0.5 of 0.9944 confirms excellent localization accuracy, even under challenging conditions such as varying lighting, scene complexity, and fire sizes.

7.5.3.2 Qualitative Inference Results

The following figure displays qualitative inference results produced by the YOLOv12 model when applied to a subset of unseen test images. The bounding boxes illustrate predicted fire locations, annotated with corresponding confidence scores.



Figure 7.25: Qualitative inference results. Each panel presents a distinct fire detection scenario. The YOLOv12 model successfully identifies both large-scale and small-scale fire regions across diverse settings and conditions.

7.5.4 Comparison of YOLOv10, YOLOv11, and YOLOv12

Table 7.1: Comparison of Performance Metrics for YOLOv10, YOLOv11, and YOLOv12

Model	Precision	Recall	F1-Score	mAP@0.5
YOLOv10	<u>0.8498</u>	0.8333	<u>0.8415</u>	0.9935
YOLOv11	0.8254	<u>0.8460</u>	0.8355	0.9935
YOLOv12	0.8156	0.8291	0.8223	<u>0.9944</u>

7.5.4.1 Detailed Comparison

The performance metrics across the three models indicate distinct strengths and weaknesses:

- **Precision:** YOLOv10 achieves the highest precision at 0.8498, indicating superior performance in minimizing false positives. This suggests that YOLOv10 is better at accurately identifying actual fire instances without mistakenly flagging non-fire regions.
- **Recall:** YOLOv11 excels in recall with a score of 0.8460, demonstrating its ability to detect a broader range of fire instances. This model is slightly better at ensuring that no fire is missed compared to the other models.
- **F1-Score:** The highest F1-score is achieved by YOLOv10 at 0.8415, highlighting its well-balanced trade-off between precision and recall. YOLOv11 follows closely with 0.8355, while YOLOv12 lags behind with 0.8223, indicating a slight decline in overall performance balance.
- **mAP@0.5:** All models achieve very high localization accuracy, with YOLOv12 having the highest mAP@0.5 of 0.9944. This indicates exceptional precision in detecting and localizing fire regions accurately.

7.5.4.2 Conclusion and Recommendation

Considering the performance metrics, **YOLOv10** emerges as the best-suited model for the fire detection problem due to its highest precision and F1-score, which ensures accurate detections while maintaining a good balance between precision and recall. Although YOLOv12 slightly surpasses the others in mAP@0.5, its lower F1-score indicates it may not be as reliable in maintaining precision-recall trade-offs. Therefore, for practical fire detection tasks where minimizing false positives and maximizing detection rates are essential, YOLOv10 is the optimal choice.

Chapter 8

System detailed design

8.1 System Interaction Overview

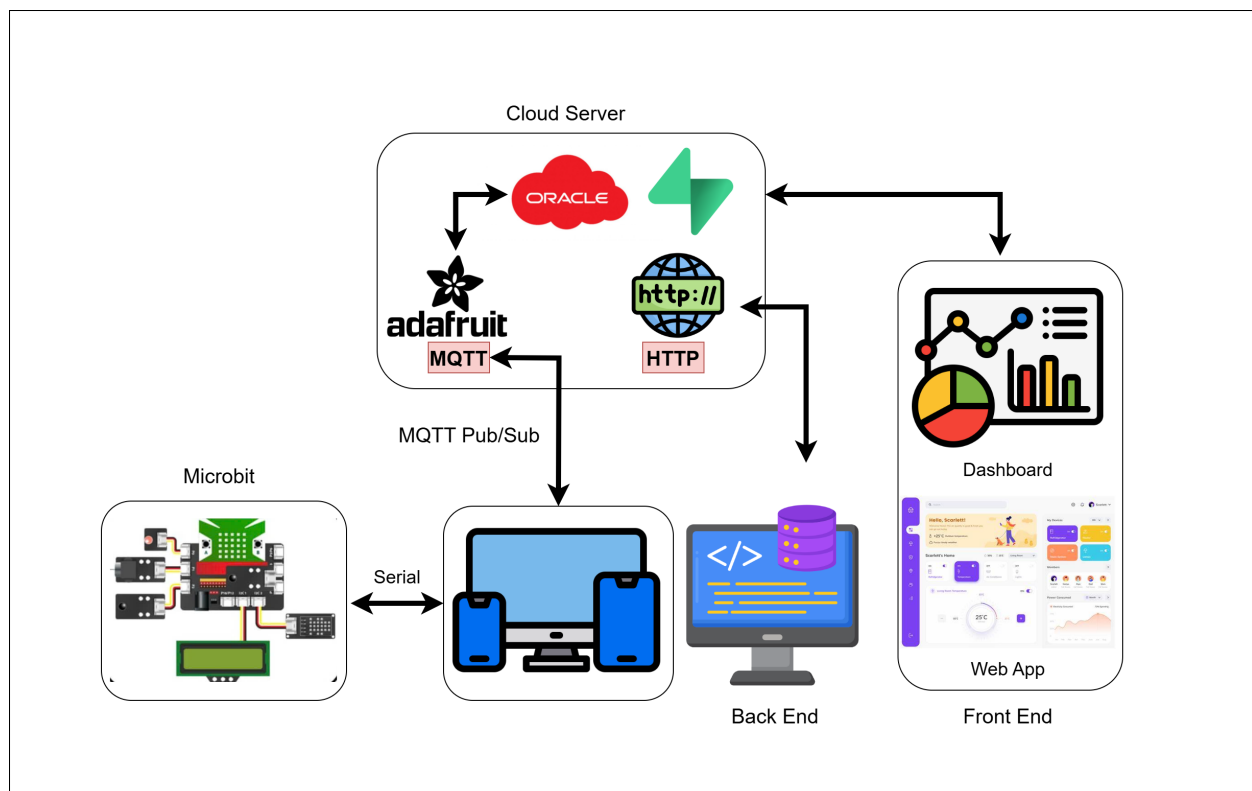


Figure 8.1: Detailed System Architecture: Microbit to Cloud Dashboard via MQTT and Adafruit IO

The Smart Home system integrates multiple layers of technologies and services to achieve real-time sensing, intelligent control, and responsive visualization. Figure 8.1 provides a visual representation of how hardware components, cloud services, backend APIs, and the

frontend interface interact.

- **Device Layer (Micro:Bit-based Hardware):** Sensors for temperature, humidity, light, and gas are connected to a Micro:Bit microcontroller. This controller gathers environmental data and sends it via **Serial communication** to the IoT Gateway (e.g., a Raspberry Pi).
- **IoT Gateway Layer:** The gateway performs several crucial tasks:
 - Receives sensor data via serial.
 - Publishes data to **Adafruit IO feeds** via MQTT.
 - Optionally executes **local AI inference** using Python/Java services when cloud inference is not needed or offline.
- **Cloud Platform and Backend Server:**
 - A FastAPI backend subscribes to the Adafruit MQTT feed to receive sensor updates.
 - The backend **saves the data to Supabase Cloud Database** for long-term storage and historical queries.
 - For intelligent decision-making (e.g., fire detection), the frontend forwards input (be it an image or a video stream) to a **model hosted on Oracle Cloud**. The model's inference result (bounding boxes for our fire-detection model) is sent back to the frontend to draw the bounding boxes.
 - A MySQL database is used for user authentication and session tracking.
- **Frontend Layer (Web App and Dashboard):**
 - The React + Next.js frontend consumes RESTful APIs exposed by the backend.
 - Real-time and historical sensor data is visualized using Chart.js and Recharts.
 - The dashboard also allows users to send control signals (e.g., turn on/off fan/light), which are forwarded to the backend and then published to the Adafruit MQTT topic.
- **Cloud Services:**
 - **Adafruit IO** acts as the communication middleware between IoT devices and the cloud via MQTT and HTTP.
 - **Supabase** stores structured time-series sensor data for future analytics.
 - **Oracle Cloud** hosts the AI model used for complex inference tasks.

Overall, the system ensures a seamless end-to-end flow from physical sensor data acquisition to intelligent decision-making and user feedback via dashboards — all designed to support real-time responsiveness and modular integration.

8.2 Cloud-Based AI Inference with Oracle Cloud

To enable real-time AI inference while minimizing latency, our system leverages **Oracle Cloud Infrastructure (OCI)**. Oracle offers a *free tier instance*, which our group utilized by registering for a developer account. This instance includes a flexible compute resource (VM.Standard.A1.Flex) that we configured to host our AI services.

To reduce communication latency between the backend hosted in Vietnam and the cloud AI server, we strategically selected the **Singapore region** — the nearest available OCI data center to Vietnam. This ensures low-latency model inference and faster control feedback for latency-sensitive operations like fire detection. By doing so, we minimize the communication delays between our service and the homeowner, making sure that the moment our model detect something critical, an alert will be sent instantly.

Upon instance creation, we configured the **Virtual Cloud Network (VCN)** to allow TCP and UDP traffic across *all ports*. While it is generally advisable to limit access to specific ports, this design choice simplified integration and reduced complexity during the rapid prototyping phase.

The server runs on the **Ubuntu** operating system. For firewall management and secure access, we used **iptables** rules to allow necessary inbound and outbound connections to our FastAPI model inference service. This OCI-hosted model is invoked by the frontend when the user choose the upload an image to test our fire detection models or turn on their camera to stream a live feed for analysis, the inference results are sent back the the frontend where the bounding boxes are drawn based of the detection results.

8.3 Supabase Cloud Database Integration

The system employs **Supabase** — a cloud-native platform based on PostgreSQL — to store and back up all sensor data. The data flow is structured as follows:

1. Sensor data (e.g., temperature, humidity, light) is transmitted from the Micro:Bit hardware to the IoT Gateway (Raspberry Pi).
2. The gateway publishes the data to the corresponding **Adafruit IO feed** using MQTT.
3. The backend service, implemented in FastAPI, subscribes to Adafruit IO feeds and listens for incoming data.
4. Upon receiving new sensor data, the backend **stores the values into Supabase** using HTTP requests authenticated via Supabase’s secure API keys.

This integration ensures that sensor readings are **persistently stored** in the cloud for real-time monitoring, analytics, and retrospective inspection via the frontend dashboard. Supabase was chosen for its easy setup, scalability, built-in RESTful API layer, and compatibility with modern JavaScript-based frontend frameworks.

8.4 Frontend Technologies and Design Rationale

The frontend of our system is built on modern web technologies to ensure a responsive and interactive user interface. The entire application is implemented using the **Next.js** framework (v15.3.1), which provides server-side rendering, fast navigation, and seamless API integration.

Key libraries and their purposes:

- **React** (v19.0.0): Powers the UI as a component-based architecture, enabling efficient re-rendering and modular design.
- **TypeScript** (v5): Ensures strict type checking and better maintainability.
- **Tailwind CSS** (v4): Provides a utility-first styling framework for rapid UI development, ensuring consistency and responsiveness.
- **Chart.js** (v4.4.9) and **react-chartjs-2** (v5.3.0): Used for visualizing real-time and historical sensor data in the dashboard (line charts, bar charts, etc.).
- **Recharts** (v2.15.3): Offers alternative chart components optimized for responsive layouts and customizable SVG-based rendering.
- **@iconify/react** (v5.2.1): Provides scalable vector icons used across dashboard controls for fans, lights, and sensors.

Development tools and configuration:

- **ESLint** (v9) and **@eslint/eslintrc** (v3): Enforce coding best practices and maintain code hygiene.
- **PostCSS with Tailwind Plugin**: Enables CSS post-processing and theme customization using Tailwind utilities.
- **NextConfig and TypeScript Path Aliases**: Defined for better import resolution and future scalability via '@/*' aliases.

This stack empowers us to build a robust and visually rich frontend dashboard that supports:

- Real-time display of temperature, humidity, and light data.
- Toggle-based fan and light controls.
- Time-series analysis across different time frames (day, week, month, year).
- Fast navigation between modules like dashboard, fan, and sensor monitoring.

8.5 Backend Technologies and Architecture

The backend is implemented using **FastAPI** (Python), chosen for its asynchronous capabilities, performance, and compatibility with RESTful API standards. It serves as the middleware between the frontend, the Adafruit IO MQTT feed, Supabase DB, and the cloud AI model.

Key Components and Functionalities:

- **Sensor API Endpoints:**
 - GET `/sensor/temp/latest`, `/humid/latest`, `/light/latest`
 - GET `/sensor/temp/history1000`, etc., for querying historical data
- **Device Control Endpoints:**
 - POST `/fan/fan/on`, POST `/light/switch/on`
 - Triggered by user commands from the frontend dashboard.
- **Authentication API:**
 - POST `/login/authentication`: Verifies user credentials against a local MySQL database.
- **Adafruit IO Integration:**
 - Uses the `Adafruit MQTT Client` to subscribe and publish to sensor/device feeds.
 - Functions as both a listener for sensor data and a publisher for actuator control signals.

Design Highlights:

- CORS middleware is configured to allow secure communication between the Next.js frontend and FastAPI backend.
- Asynchronous I/O ensures that sensor polling and feed publishing do not block API responsiveness.
- Adafruit MQTT communication is encapsulated in a dedicated module, supporting both Pub/Sub mechanisms.

Overall, the backend acts as the **central orchestrator** that synchronizes data flow from physical sensors to cloud storage, invokes inference models hosted on Oracle, and facilitates real-time control via frontend commands.

8.6 Design Justification and Architectural Fit

Our system design not only ensures technical feasibility but also aligns well with the principles of modularity, separation of concerns, and scalability. One of the core software engineering patterns that guided our implementation is the **Model-View-Controller (MVC)** architecture, which is naturally supported by our selected technology stack.

Advantages of Next.js and the MVC Paradigm

- **Model:** Data is fetched and updated via APIs and synchronized with external sources such as Adafruit IO and Supabase. Supabase acts as our long-term data store, while Adafruit provides real-time device-state streams.
- **View:** Built with **React 19** and **Tailwind CSS**, the View layer renders data responsively and supports real-time interactivity through reusable components.
- **Controller:** Next.js routing mechanisms and custom React hooks act as controllers, handling user interactions and API calls. The controller logic is cleanly separated from the UI and data layers.

This separation not only improves maintainability and testability but also allows collaborative development among team members working on different layers independently.

Design Appropriateness and Evaluation

We evaluate our system design across several key criteria:

- **Scalability:** By leveraging cloud services (Oracle Cloud, Supabase) and stateless backend APIs via FastAPI, the system can scale horizontally. Additional frontend or backend instances can be deployed without disrupting the architecture.
- **Responsiveness:** MQTT and HTTP-based communication ensure that device control and data updates are pushed or pulled in near real-time. The dashboard reflects sensor changes and control feedback with minimal delay.
- **Security and Configurability:** Authentication is managed using a MySQL database, and secure API communication is facilitated via HTTPS. The backend allows safe handling of device commands with role-based access extensibility.
- **Maintainability:** With Next.js + TypeScript in the frontend and FastAPI + modular routing in the backend, each part of the system can evolve independently. The clear separation of hardware, logic, and presentation makes debugging and future upgrades more manageable.

- **Flexibility and Extensibility:** Additional modules (e.g., image recognition, energy usage tracking) can be integrated with minimal disruption, as the architecture supports modular feature expansion.
- **User-Centric Experience:** The frontend provides an intuitive and responsive UI, accessible from desktops or tablets. Real-time graphs and toggles create an effective user experience for monitoring and control.

In conclusion, the combination of FastAPI, MQTT, Supabase, and Next.js forms a cohesive and well-aligned architecture that supports real-time smart home applications while remaining scalable, secure, and developer-friendly.

Chapter 9

Deployment

9.1 Github Repository, Ohstem Code and Video Demo

9.1.1 Part 1: Yolo:Bit Module

Link to the Yolo:Bit program:

<https://app.ohstem.vn/#!/share/yolobit/2wJ4QNxINW7ypVpHXpSp6DlXwfS>

9.1.2 Part 2: Web Application Module

Link to the web application repository:

https://github.com/minhhieutaro/DoAnDaNganh_Group1

9.1.3 Part 3: Fire and Smoke Detection Model with AI (YOLOv10, YOLOv11, YOLOv12)

Links to the training notebooks on Kaggle:

- YOLOv10: <https://www.kaggle.com/code/phunguyenquang/dadn-yolov10>
- YOLOv11: <https://www.kaggle.com/code/zphudzz/dadn-yolov11>
- YOLOv12: <https://www.kaggle.com/code/phunguyenquang/dadn-yolov12>

9.1.4 Part 4: Demonstration

Link to the demonstration videos of our project:

<https://drive.google.com/drive/u/3/folders/1Gca9v4jIapleQGKFx32boxOEGag5bx9f>

9.2 YoloBit module

9.2.1 Monitoring Air Quality

A core component of the smart home system is its ability to persistently monitor the indoor environment. This system gathers essential data about the living space at regular intervals, such as every five minutes, allowing for seamless real-time tracking and remote management of household conditions.

This project emphasizes the observation of air quality and lighting within the home. To accomplish this, we rely on the following carefully chosen sensors:

- The DHT20 sensor is employed to measure temperature and humidity, delivering reliable and accurate readings of these critical indoor parameters.
- Lighting levels are evaluated using an external light sensor connected to the expansion board. Although the Yolo:Bit includes an onboard light sensor, this external option provides superior precision and ease of setup.

The information collected is displayed on an LCD screen with a 16-row by 2-column configuration. Crafting a clear and attractive layout for this display is essential for user satisfaction. Our design organizes the data as follows:

- Line 1 presents real-time values for temperature, humidity, and light intensity, offering an immediate overview of the environment.
- Line 2 provides the current date (day, month, year) and time (hour, minute), ensuring users have both spatial and temporal context.

Since each row can only display a maximum of 16 characters, we'll try to sort the believe to fit the screen. We recommend that you read how to buy information such as follows:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	2	5	.	7	*	C			6	3	%			2	1	%
1	2	1	/	1	0	/	2	0	2	2		1	3	:	3	8

Figure 9.1: Display information on the LCD character screen

The AIOT-KIT programming library facilitates the straightforward implementation of this display. Nonetheless, the success of the interface hinges on a well-thought-out screen design that balances functionality with visual appeal, enhancing the overall experience of the smart home system.

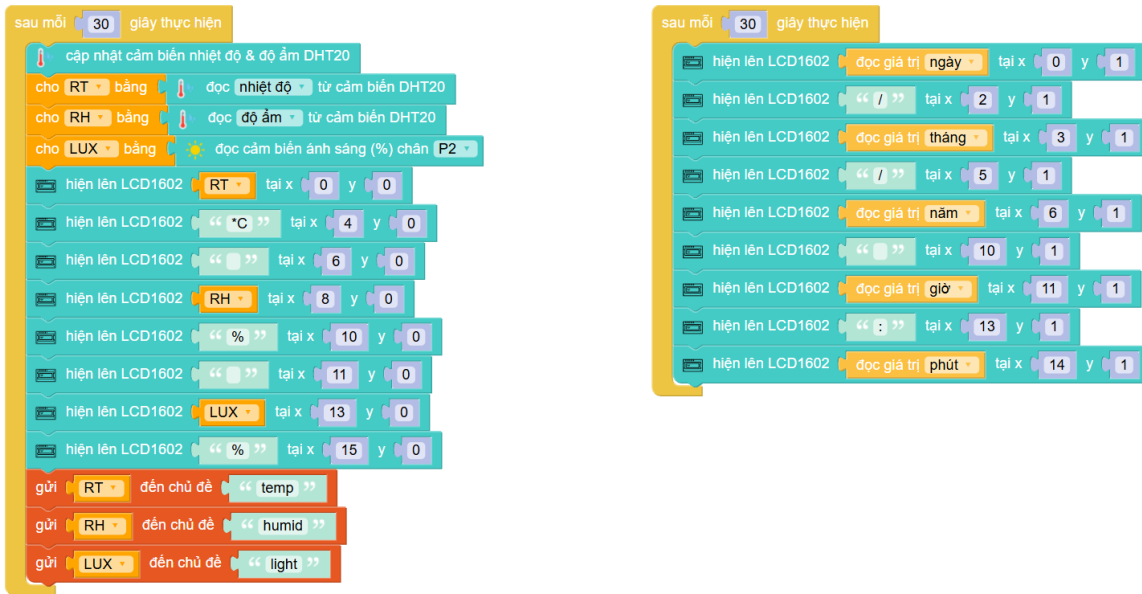


Figure 9.2: Display information on the LCD character screen

9.2.2 Cipher Lock Door

The cipher lock door system integrates two essential features: validating an access code and allowing the code to be updated when necessary. Interaction with the system is handled through an infrared remote control, enabling wireless password entry and control commands.

For regular operation, users must input a predefined sequence of digits via the IR remote. This input is then checked against a stored access code. If the entered code matches the stored one, the door will unlock momentarily, remaining open for approximately 2 seconds before automatically relocking. If the entered code is incorrect, the door stays locked, and the system will notify the user with a signal indicating authentication failure.

The system also supports secure password modification. To initiate the password update process, the correct password must first be entered twice in a row. After this confirmation, pressing the “F” key activates the password update mode, during which the system sets an internal flag variable **STATUS** to 1. In this mode, users are allowed to define and store a new access code. Once the new password is successfully stored, the system returns to the standard verification mode by resetting **STATUS** to 0. If the user decides not to change the password, they may exit the password update process at any time by pressing “F” again, which cancels the operation and resumes normal code verification.

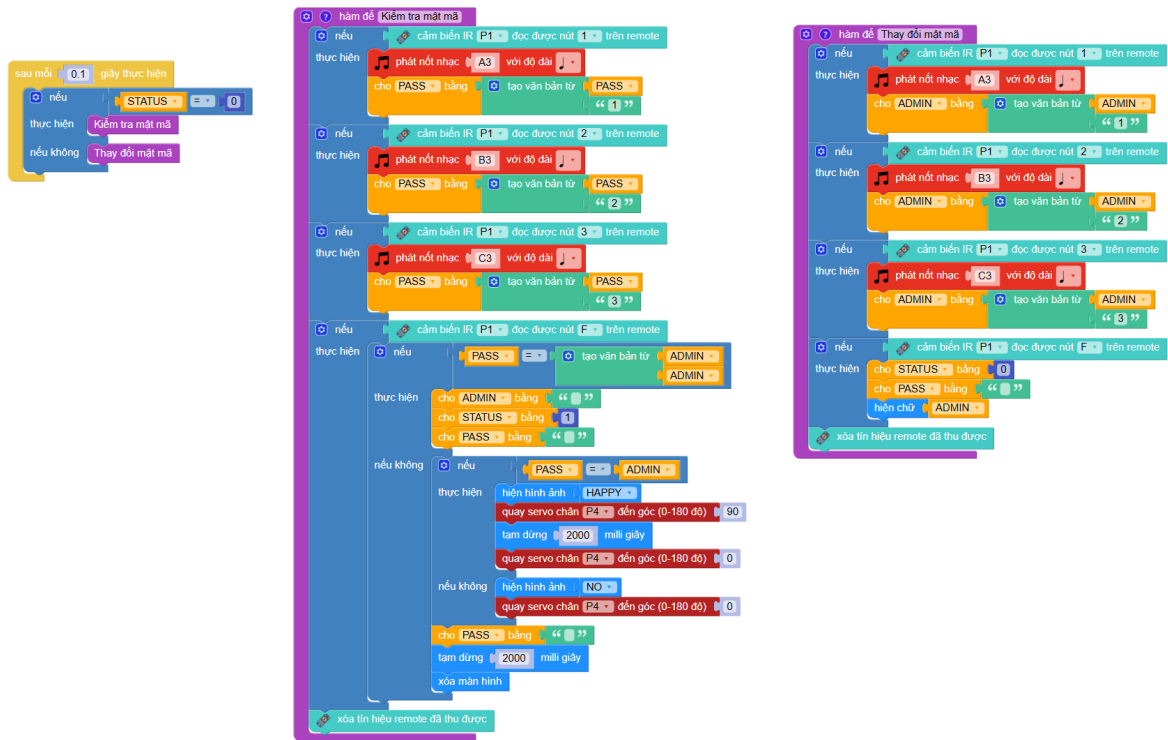


Figure 9.3: Cipher Lock Door

9.2.3 IOT Dashboard

The dashboard based on MQTT protocol will be implemented on Adafruit's Dashboard. This is where we exchanged data between the MCU with web application.

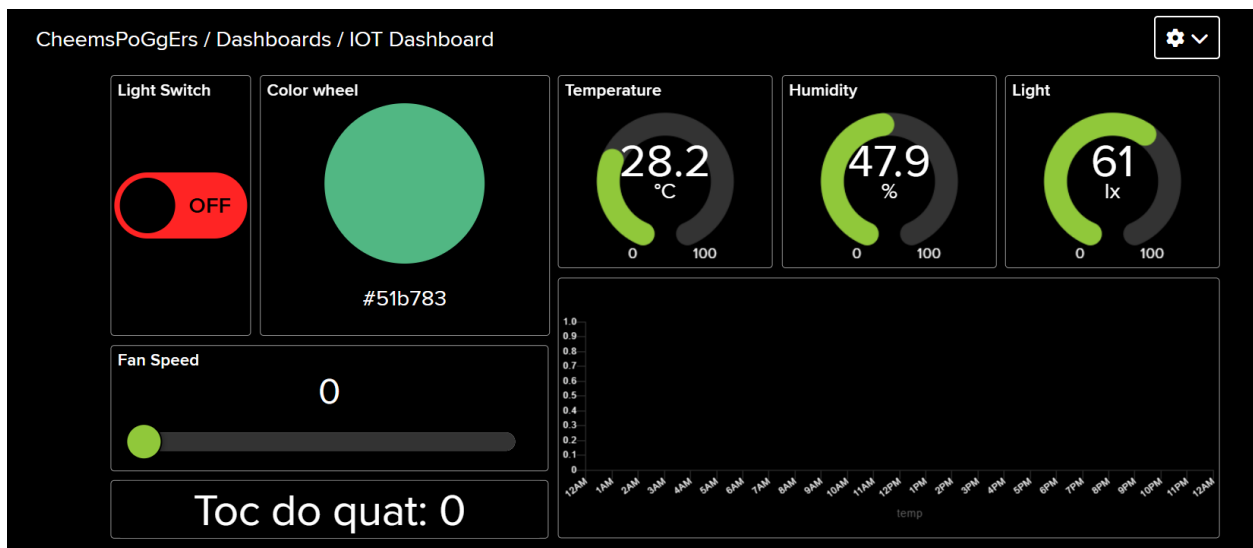


Figure 9.4: IOT Dashboard

This section describes the communication structure and topic interactions between the hardware devices and the Adafruit server through MQTT protocol.

The power control for the RGB LED module is handled by a toggle button, which sends a message of either “1” (to turn on) or “0” (to turn off) to the MQTT topic **switch**. Alongside this, the LED color is dynamically updated based on a selected color palette, which publishes a hexadecimal color code to topic **color**. The supported color set includes red, orange, yellow, green, blue, indigo, and violet, while black is used to represent the “off” state.

Environmental sensors relay real-time data to the system: the temperature, humidity, and light intensity readings are received via MQTT subscriptions to topics **temp**, **humid**, and **light** respectively. These values are continuously updated and displayed in dedicated on-screen information boxes.

Additionally, a temperature history graph subscribes to topic **temp**, where it captures and plots temperature data over time to provide a visual timeline of thermal conditions.

The fan is controlled through a slider component that publishes the desired speed value to topic **fan**. This value determines and adjusts the fan’s operating speed accordingly.

9.2.4 Voice AI

In this section, we implement an AI module using the Ohstem application to interpret speech input and perform corresponding actions within the IoT system.

The first step involves establishing a connection to the Ohstem server by logging in through the user’s account credentials. Once connected, the application initiates the voice recognition process. In this project, Vietnamese is used as the input language for voice commands.

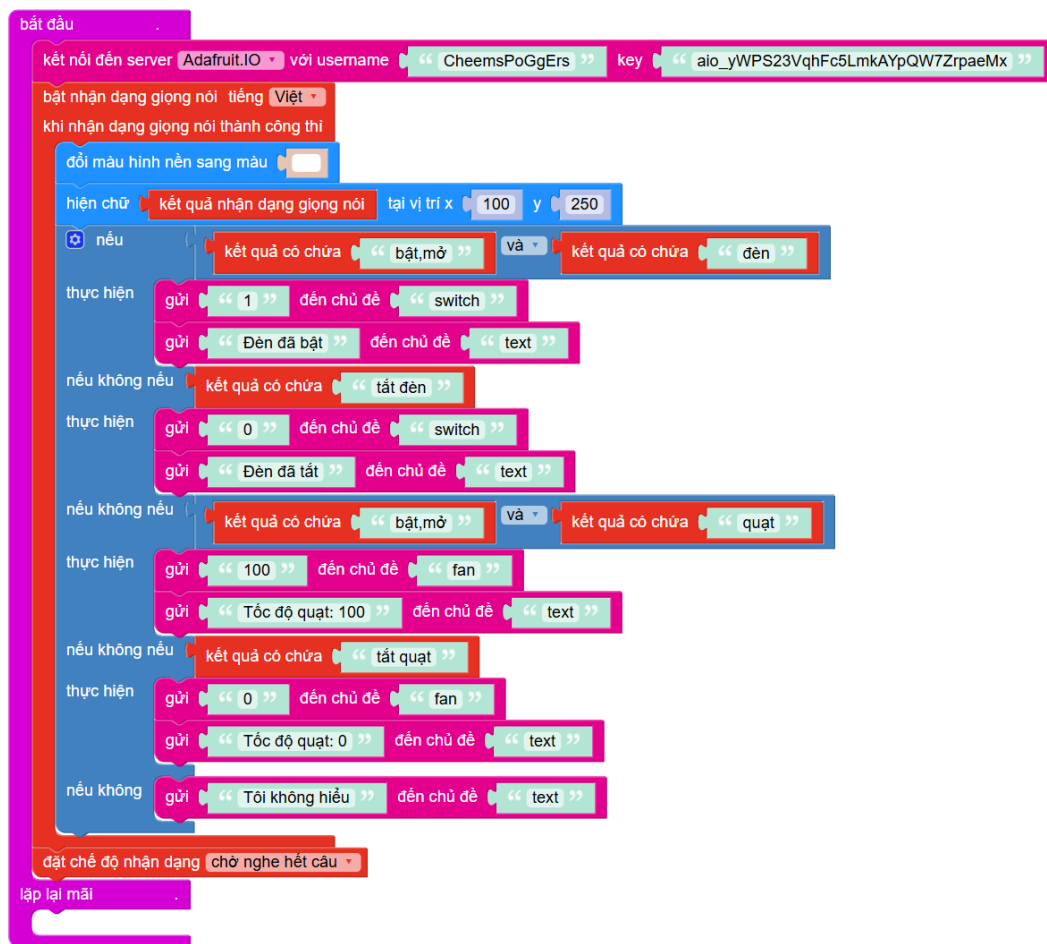


Figure 9.5: YoloBit code for Voice AI

Upon successfully receiving and converting a spoken phrase to text, the system analyzes the message to identify specific keywords. If the interpreted text includes the phrase “bật đèn” or “mở đèn” (meaning “turn on the light”), the system publishes a signal “1” to topic **switch**, which triggers the light to switch on. Conversely, if the phrase “tắt đèn” (meaning “turn off the light”) is detected, a signal “0” is sent to the same topic to turn the light off. The same goes for the fan. If the text includes the phrase “bật quạt” or “mở quạt” (meaning “turn on the fan”), the system sends the signal “100” to the topic **fan** which turns on the fan at max speed. The fan will stop if the text includes the phrase “tắt quạt”.

In cases where the voice input does not contain either of the predefined keywords, the system defaults to publishing the message “Tôi không hiểu” (translated as “I don’t understand”) to topic **text**. This message is then displayed in the Information panel of the IoT dashboard to inform the user of the unrecognized command.

9.2.5 Face AI

The face recognition functionality utilized in this project is powered by a pre-trained model available through the YOLOHOME platform. This model operates as a supervised image classifier, meaning it requires labeled training data for each individual the system is expected to recognize. For instance, if the application needs to identify five unique individuals, it must be trained using facial images of those five specific people. Consequently, the system will only be able to identify individuals from this predefined group.

During inference, the model analyzes the input image and returns a confidence score representing the likelihood that the face matches one of the trained classes. If this score exceeds a predefined threshold—set to 0.7 in our case—the system considers the recognition to be valid and associates it with the corresponding individual’s label.

To improve reliability in real-time scenarios, the system includes a verification mechanism based on repetition. A counter is maintained to monitor how many consecutive frames a person has been correctly recognized. Once the counter surpasses a threshold of 3, the identification is deemed consistent and is accepted for further system actions.

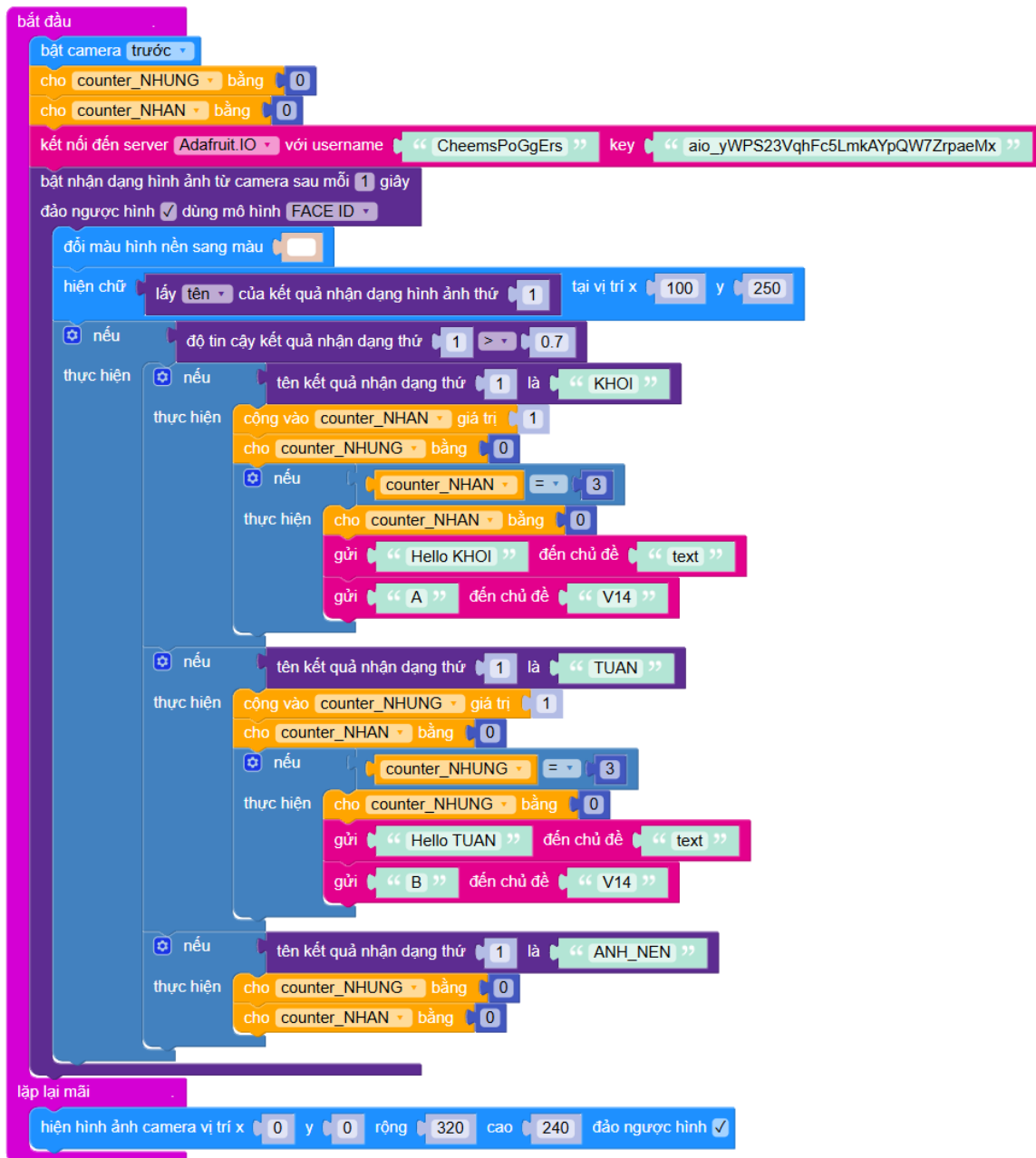


Figure 9.6: YoloBit code for Voice AI

9.3 Web Application

9.3.1 Home Page (Landing Page)

Description

The Home Page serves as the main landing page for the Smart Home System application, providing an introduction to the service and encouraging new users to sign up.

The main components of this page include:

- **Welcome Section:** Features the heading "Welcome to Smart Home System" with a brief introductory text.
- **Team Showcase:** Displays profile cards for five team members (**Phu**, **Hieu**, **Khoa**, **Binh**, and **Khoi**) with their roles.
- **Testimonials:** Features three customer quotes with usernames to build credibility.
- **Impact Statistics:** Displays key metrics (10,000+ users served, 24+ cities, 98% customer satisfaction).
- **Call-to-Action:** Contains a "Ready to Make Your Home Smarter?" heading with a prominent "Get Started Now" button for login functionality.

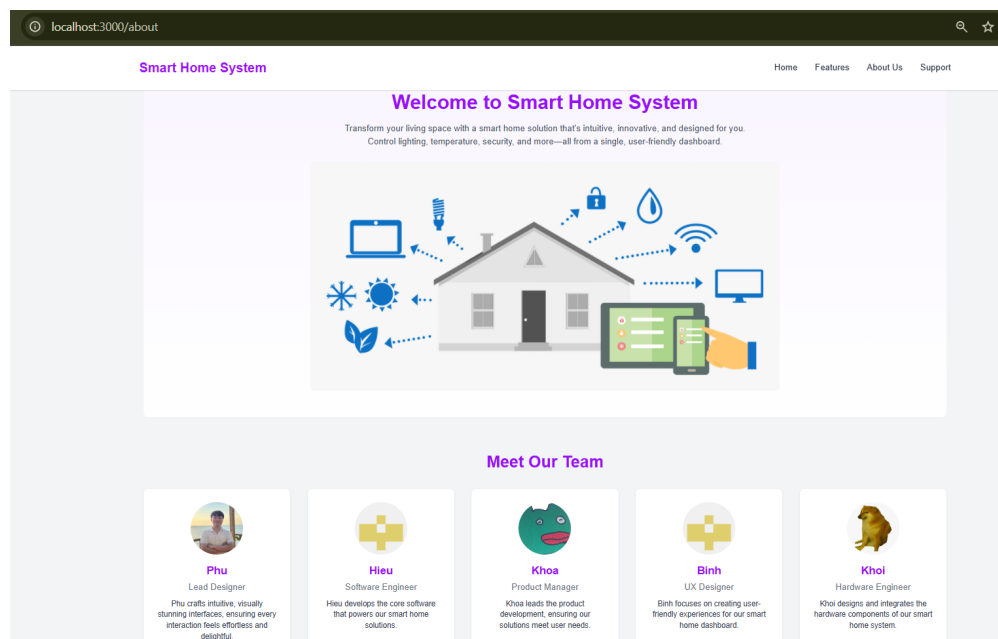


Figure 9.7: Home page: introduction, team profiles, testimonials

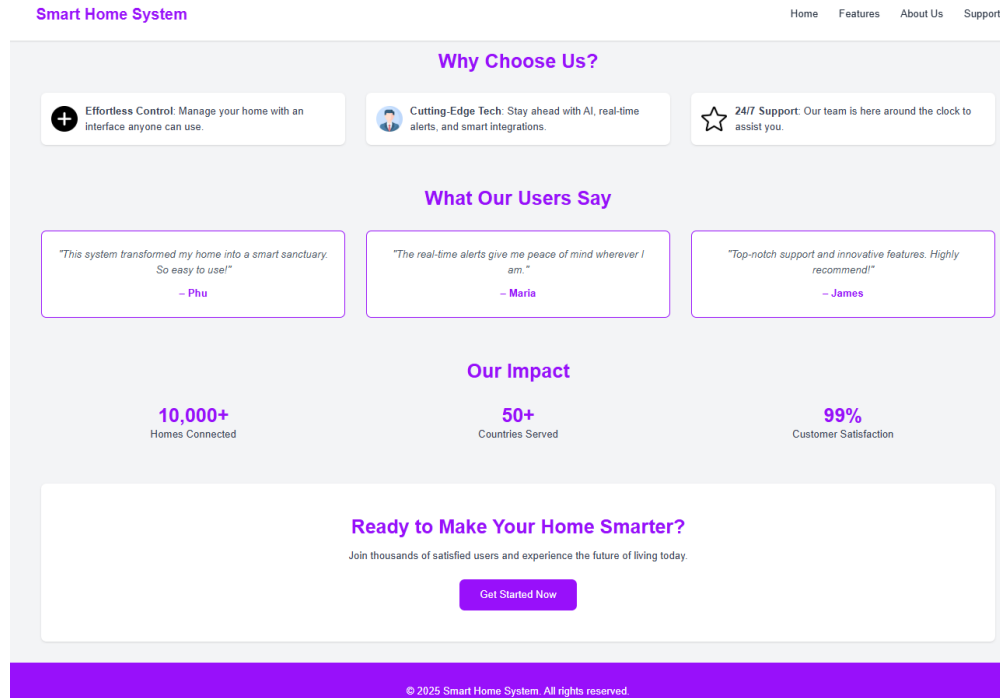


Figure 9.8: Home page: call-to-action.

Future Improvements

- Implement user authentication system to replace the temporary login.
- Add interactive demonstrations of key system features.
- Create a responsive design for better mobile experience.

9.3.2 Authentication Pages

Description

The Authentication system provides secure access to the Smart Home Dashboard through separate login and registration interfaces. These pages serve as the entry point to the system, ensuring only authorized users can access the dashboard features.

The main components of this system include:

- **Login Page:** Features a clean, centered form with "Smart Home Dashboard" heading and "Sign in to your account" subheading. Contains username and password input fields, "Remember me" checkbox, and a prominent purple "Sign In" button. Additional links for password recovery and account creation are provided below the form.

- **Registration Page:** Presents a user-friendly form with "Create your account" sub-heading. Includes fields for username, email address, date of birth (with calendar picker), password, and password confirmation. A purple "Create account" button submits the registration, while a link redirects existing users to the login page.
- **Date Selection:** Features an interactive calendar component that appears when the date field is active, allowing users to select their birthdate with a visual monthly calendar interface.

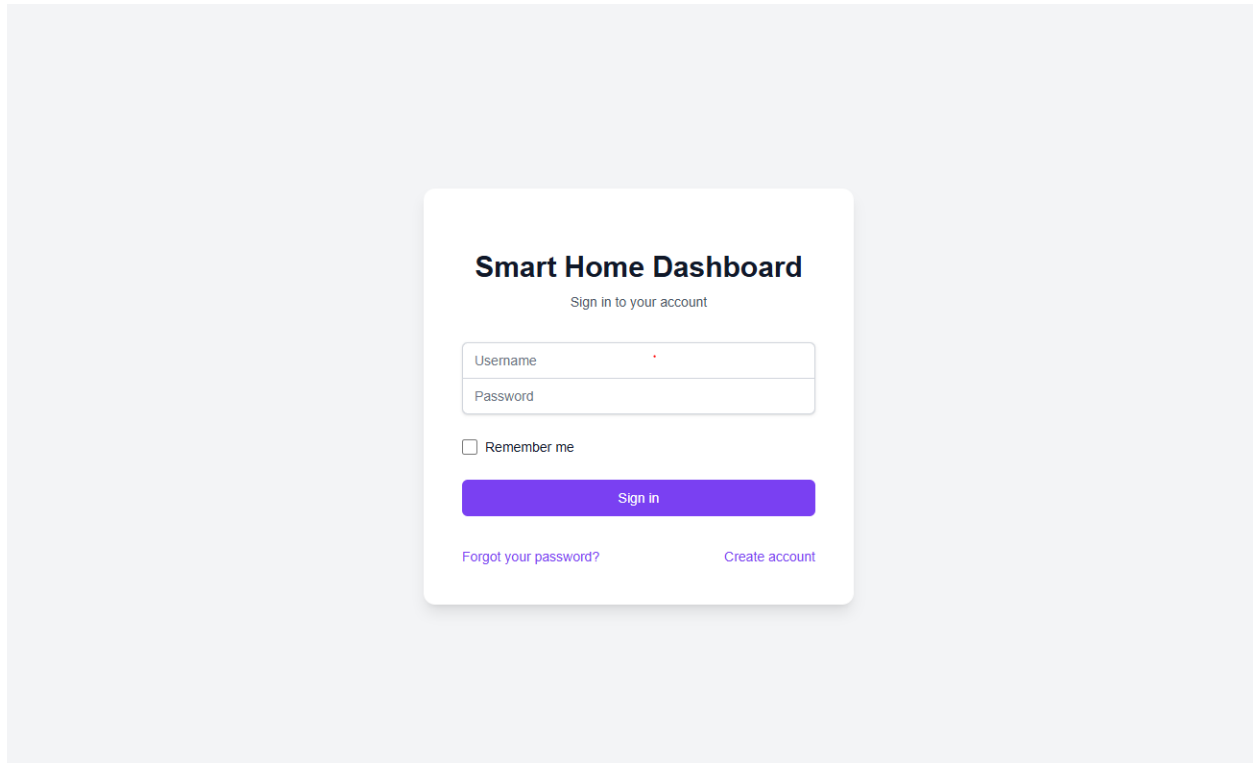


Figure 9.9: Authentication pages: secure login with interactive components.

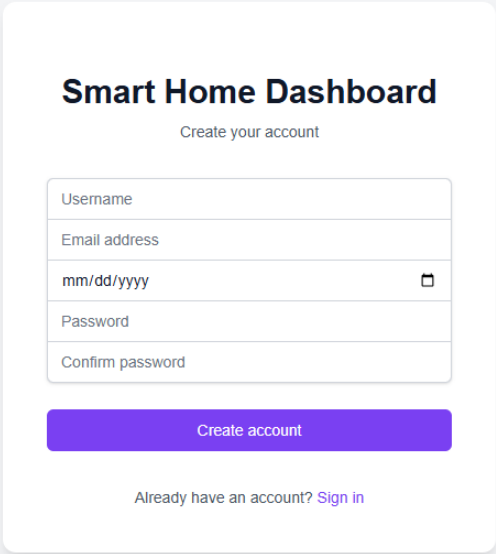
A registration form for a 'Smart Home Dashboard'. The form is centered on a light gray background. It has a title 'Smart Home Dashboard' in bold black text, followed by the subtitle 'Create your account' in a smaller font. Below the subtitle is a form with five input fields: 'Username', 'Email address', a date field with a placeholder 'mm/dd/yyyy' and a calendar icon, 'Password', and 'Confirm password'. A prominent blue button labeled 'Create account' is positioned below the form fields. At the bottom of the form, there is a link that says 'Already have an account? Sign in'.

Figure 9.10: Authentication pages: user registration with interactive components.

Future Improvements

- Implement social media authentication options for faster sign-in.
- Add two-factor authentication for enhanced security.
- Include password strength indicator during registration.
- Integrate email verification for new account creation.
- Store user preferences and settings in secure user profiles.

9.3.3 Home Page (Dashboard)

Description

The Home Page serves as the main dashboard interface after authentication, providing users with a comprehensive overview of their smart home environment. It combines real-time environmental data, sensor trends, and device controls in a single, intuitive interface.

The main components of this page include:

- **Welcome Banner:** A yellow highlight section that greets the user by name ("Hello, Phu!"), provides a welcome message, and offers quick environmental status updates including current temperature (+25°C) and weather conditions ("Fuzzy cloudy weather").
- **Multi-sensor Chart:** A central interactive graph displaying historical trends for Humidity (blue), Light Intensity (pink), Temperature (purple), and Air Quality (green) with selectable time ranges (Day, Week, Month, Year).
- **Notification Panel:** Displays critical alerts such as "Phát hiện cháy tại Phòng chính!" with timestamps to alert users of urgent conditions.
- **Control Panel:** The right section offers toggle controls for Humidity and Temperature, LED color selection with dropdown menu (currently set to "RED"), Light Intensity adjustment, and Fan speed control with an interactive slider.

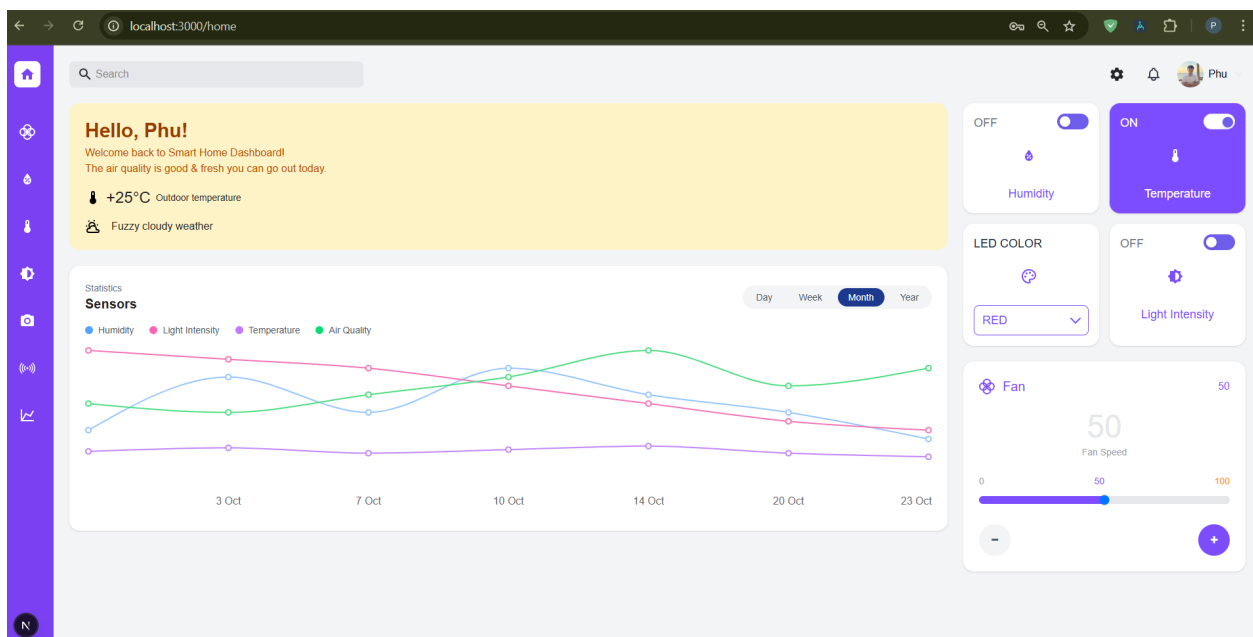


Figure 9.11: Home Page: comprehensive dashboard with device controls.

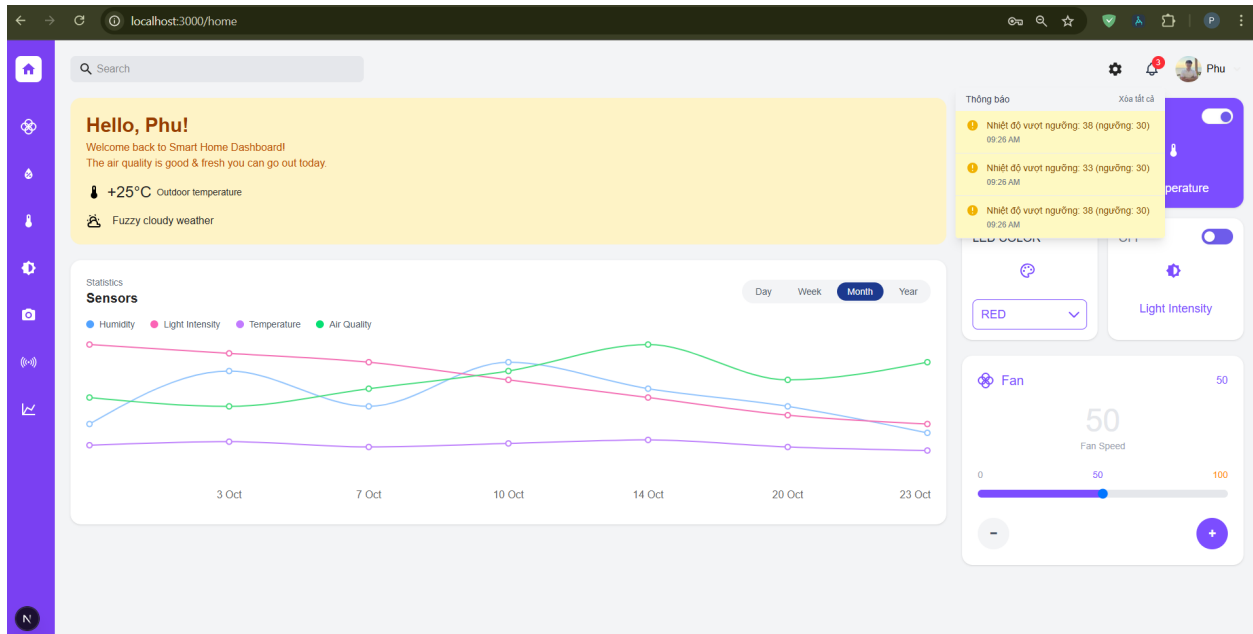


Figure 9.12: Home Page: comprehensive dashboard with environmental monitoring.

Future Improvements

- Implement customizable dashboard layouts to allow users to prioritize specific sensors or controls.
- Add voice control capabilities for hands-free operation of smart home features.
- Integrate machine learning algorithms to provide predictive insights based on usage patterns.
- Enable user-defined automation rules based on sensor threshold values.
- Add multi-language support for international users.

9.3.4 Fan and Light Control Page

Description

The Fan and Light Control page allows remote management of both the fan system and the LED lighting system through MQTT-based communication. This page provides direct interaction for managing device statuses and settings.

The main components of this page include:

- **Fan On/Off Control Button:** A toggle switch to turn the fan on or off.

- **Fan Speed Adjustment Slider:** A slider control that adjusts the fan speed from 0% to 100%, with the current fan speed value displayed numerically.
- **LED Light On/Off Control Button:** A toggle switch to turn the LED light on or off.
- **LED Color Selection Dropdown:** A dropdown menu allowing the user to select the desired LED color (e.g., Black).

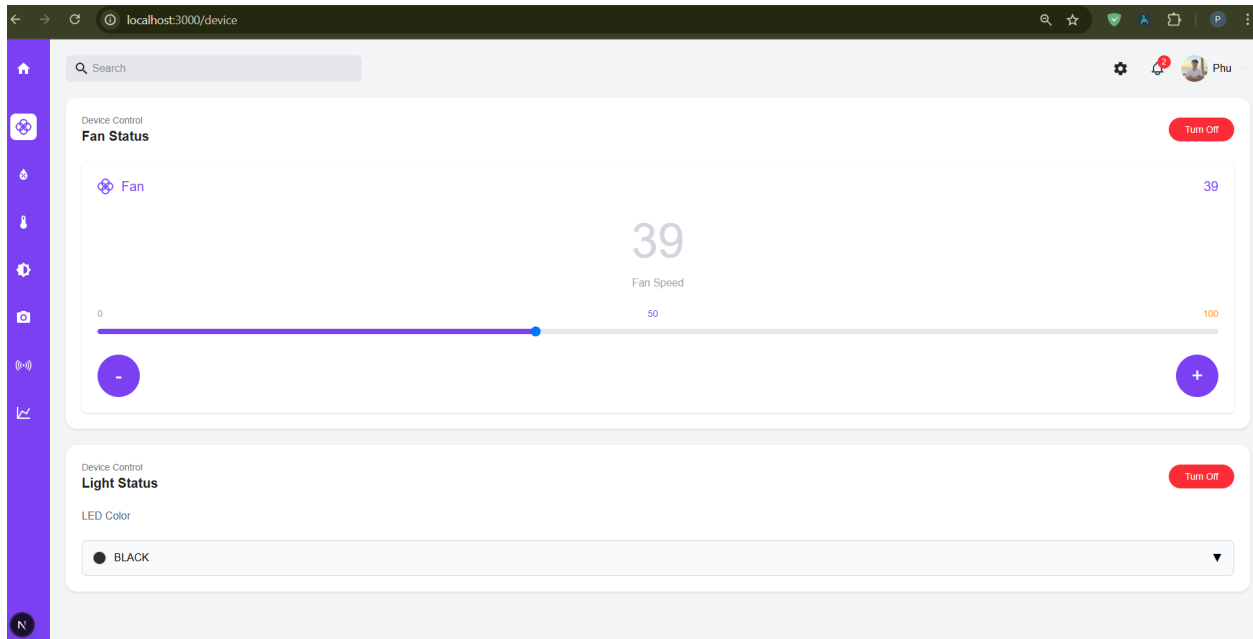


Figure 9.13: Fan and Light Control page: management of fan operation, speed, and LED light color.

Future Improvements

- Implement real-time feedback from the fan hardware (e.g., actual RPM, power consumption).
- Enable scheduling of fan and light operation based on time or environmental conditions.
- Add graphical animations to visualize the fan's current speed and light status.
- Introduce safety features like automatic shutdown if abnormal behavior is detected.
- Expand LED color options with a full-color picker and dynamic lighting effects.

9.3.5 Temperature Page

Description

The Temperature page focuses on monitoring and analyzing temperature data over time. It provides both current status and statistical analysis of temperature readings.

The main components of this page include:

- **Current Temperature Indicator:** Displays the real-time temperature value with warning notifications (e.g., "Warning - Low Temperature").
- **Statistics Panel:** Provides key statistical data including minimum, maximum, average, median, and standard deviation.
- **Temperature History Chart:** A line graph that visualizes the trend of temperature changes over the selected time range (Day, Week, Month, Year).

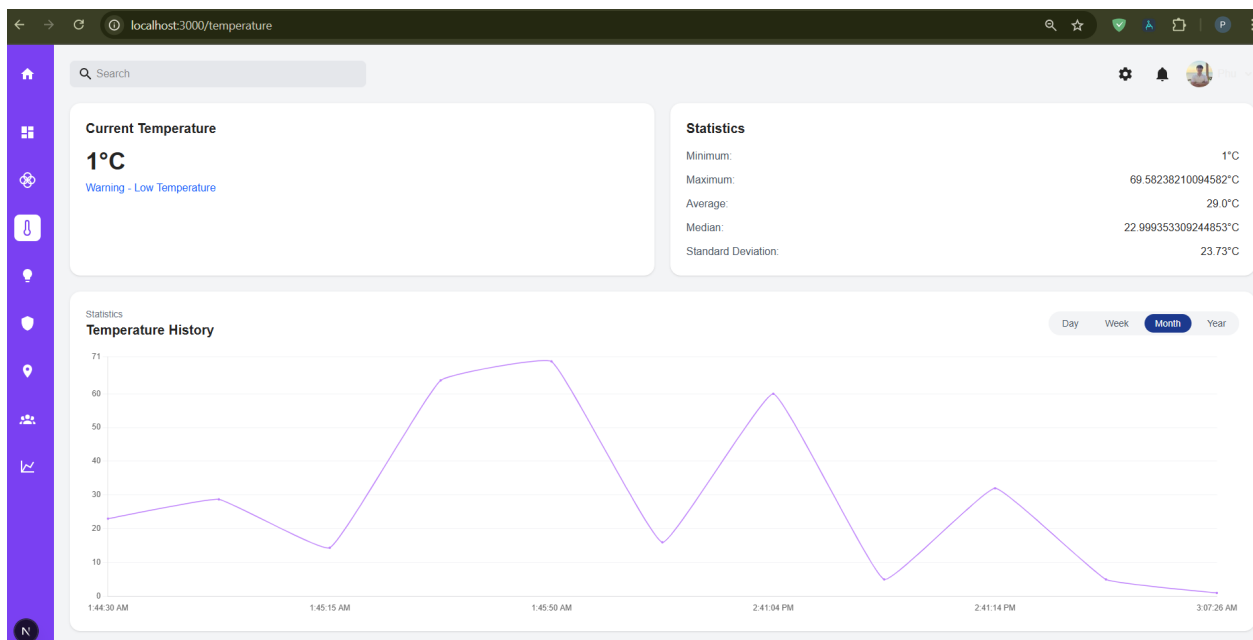


Figure 9.14: Temperature page: real-time monitoring with detailed statistical insights.

Future Improvements

- Integrate humidity data on the same page for combined environmental monitoring.
- Add predictive temperature forecasting based on historical data and trend analysis.
- Include threshold-based alerts with email or push notifications.
- Enhance statistical insights with additional metrics like quartiles and interquartile ranges.

9.3.6 Humidity Page

Description

The Humidity page focuses on monitoring moisture levels and providing users feedback on whether conditions are in a healthy range.

- **Current Humidity Indicator:** Shows real-time humidity as a percentage, with contextual warnings such as "Too Humid" or "Too Dry" when thresholds are exceeded.
- **Statistics Panel:** Lists the minimum, maximum, average, median and standard deviation for the selected timeframe.
- **Humidity History Chart:** An interactive line graph that visualizes historical humidity data over Day, Week, Month or Year ranges.

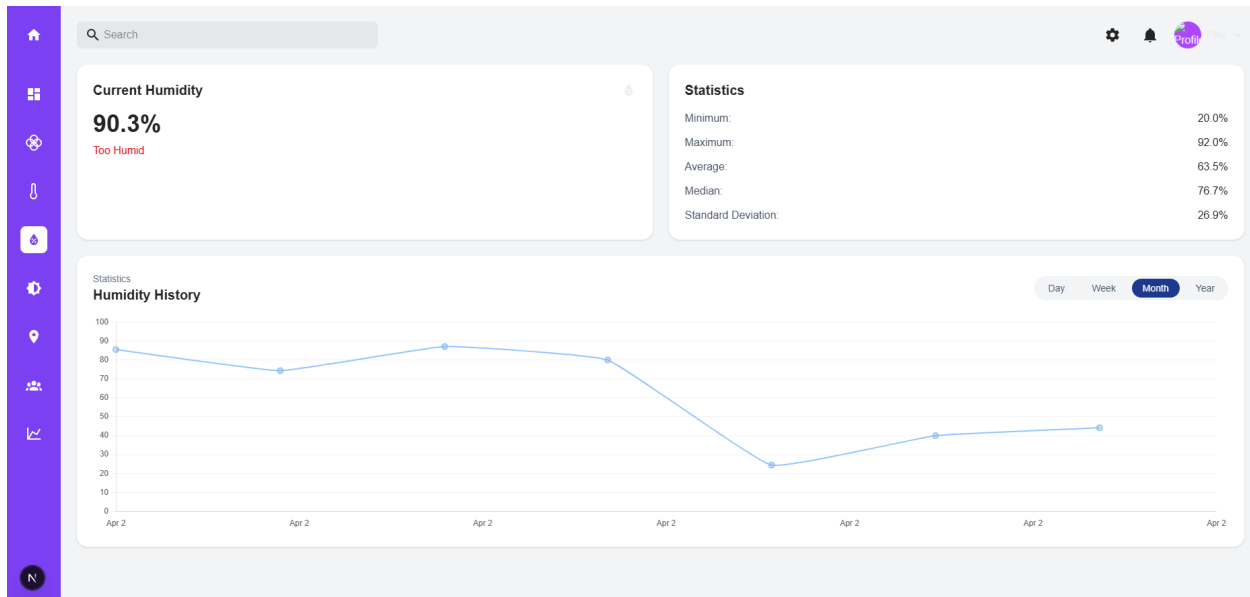


Figure 9.15: Humidity page: real-time moisture readings and historical trend analysis.

Future Improvements

- Implement user-defined humidity thresholds.
- Provide suggestions for dehumidifier or humidifier use based on long-term trends.

9.3.7 Light Intensity Page

Description

The Light Intensity page (a.k.a. lighting page) tracks ambient brightness and helps users optimise lighting for comfort and energy efficiency.

- **Current Light Intensity Indicator:** Displays live value (or percentage of a calibrated maximum) with friendly warnings such as "Very Bright" or "Too Dim".
- **Statistics Panel:** Shows minimum, maximum, average, median and standard deviation of light levels for the selected period.
- **Light Intensity History Chart:** Plots historical lighting readings, allowing quick comparison across Day, Week, Month and Year views.

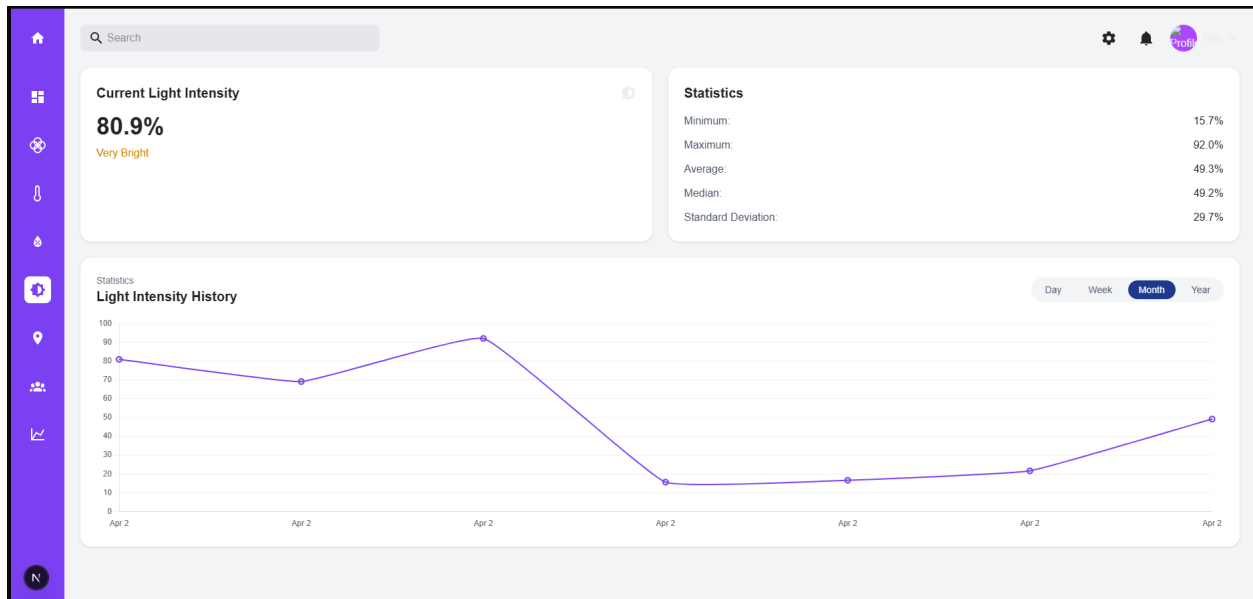


Figure 9.16: Light Intensity page: monitoring ambient brightness with rich statistics.

Future Improvements

- Implement lighting recommendations based on time of day at different seasons.
- Add integration with smart bulbs for automatic brightness tuning.

9.3.8 Data Logging Page

Description

The Data Logging page is designed to provide a centralized view of all recorded sensor activities and events. It offers detailed logs for different types of sensors, allowing users to monitor, review, and manage historical sensor data effectively.

The main components of this page include:

- **Status Indicator:** Displays the severity level of each event (e.g., *Warning*, *Info*) for quick identification and prioritization.
- **Date and Time Logging:** Records the exact date for each sensor event, helping with chronological tracking.
- **Sensor Information:** Specifies which sensor (e.g., Humidity Sensor, Temperature Sensor, Smoke Detector) generated the event.
- **Problem Description:** Briefly describes the detected issue or status update (e.g., "Humidity level above 80%").
- **Notes Section:** Provides additional details or actions taken in response to the event (e.g., "Activated dehumidifier").
- **Actions:** Enables users to download individual logs, edit event details, or delete entries.
- **Filter and Search Tools:** Allows filtering by status, date range, and a free-text search to quickly find specific events.

Data Logging

Filter

All Statuses mm/dd/yyyy mm/dd/yyyy Search data...

STATUS	DATE	ID	SENSOR	PROBLEM	NOTE	ACTIONS
Warning	2023-09-28	1	Humidity Sensor	Humidity level above 80%	Activated dehumidifier	Download  
Info	2023-09-29	2	Light Sensor	Light levels normal	No action needed	Download  
Warning	2023-09-30	3	Temperature Sensor	Temperature below 18°C	Turned on heating	Download  
Info	2023-10-01	4	Motion Sensor	Motion detected in living room	Checked camera feed, no issues	Download  
Warning	2023-10-02	5	Smoke Detector	Smoke detected in kitchen	False alarm, cooking smoke	Download  
Info	2023-10-03	6	Temperature Sensor	Temperature stable at 22°C	Optimal conditions	Download  
Warning	2023-10-04	7	Humidity Sensor	Humidity level below 30%	Activated humidifier	Download  
Info	2023-10-05	8	Light Sensor	Light levels low	Turned on lights	Download  
Warning	2023-10-06	9	Motion Sensor	Motion detected in bedroom	Checked camera, pet movement	Download  
Info	2023-10-07	10	Smoke Detector	No smoke detected	Routine check	Download  
Warning	2023-10-08	11	Temperature Sensor	Temperature above 28°C	Turned on air conditioning	Download  
Info	2023-10-09	12	Humidity Sensor	Humidity level at 50%	Ideal conditions	Download  
Warning	2023-10-10	13	Light Sensor	Light levels too high	Adjusted blinds	Download  
Info	2023-10-11	14	Motion Sensor	No motion detected	All quiet	Download  
Warning	2023-10-12	15	Smoke Detector	Smoke detected in garage	Investigated, found exhaust fumes	Download  

Prev Page 1 of 1 Next

Figure 9.17: Data Logging page: centralized management of all sensor events and activities.

Future Improvements

- Implement batch operations (e.g., download or delete multiple logs at once).
- Introduce log export formats such as CSV and PDF for external reporting.
- Add automated anomaly detection to highlight unusual patterns in the log entries.
- Include role-based access control to restrict editing and deletion permissions.
- Enhance filtering options with advanced queries (e.g., by sensor type or problem severity).

9.3.9 Fire Detection Page

Description

The Fire Detection page enables users to detect fire hazards either by uploading an image or by using real-time video streaming through a connected camera device. This functionality ensures early warning and timely intervention for fire risks.

The main components of this page include:

- **Image Detection Module:** Allows users to upload a static image and analyzes it for fire hazards.
- **Video Stream Detection Module:** Enables real-time fire detection by capturing frames from a device camera and streaming them to the server for analysis.

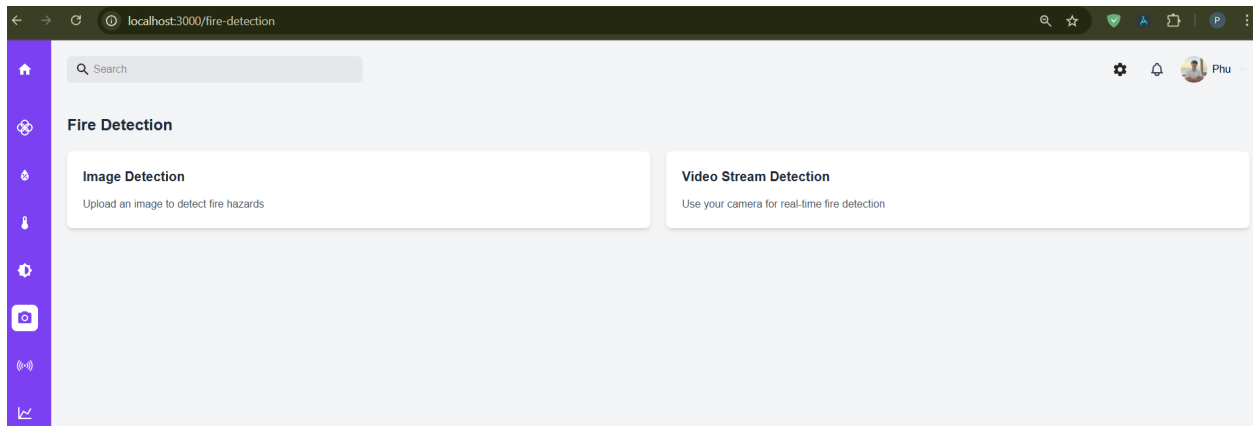


Figure 9.18: Fire Detection page: selection between Image Detection and Video Stream Detection functionalities.

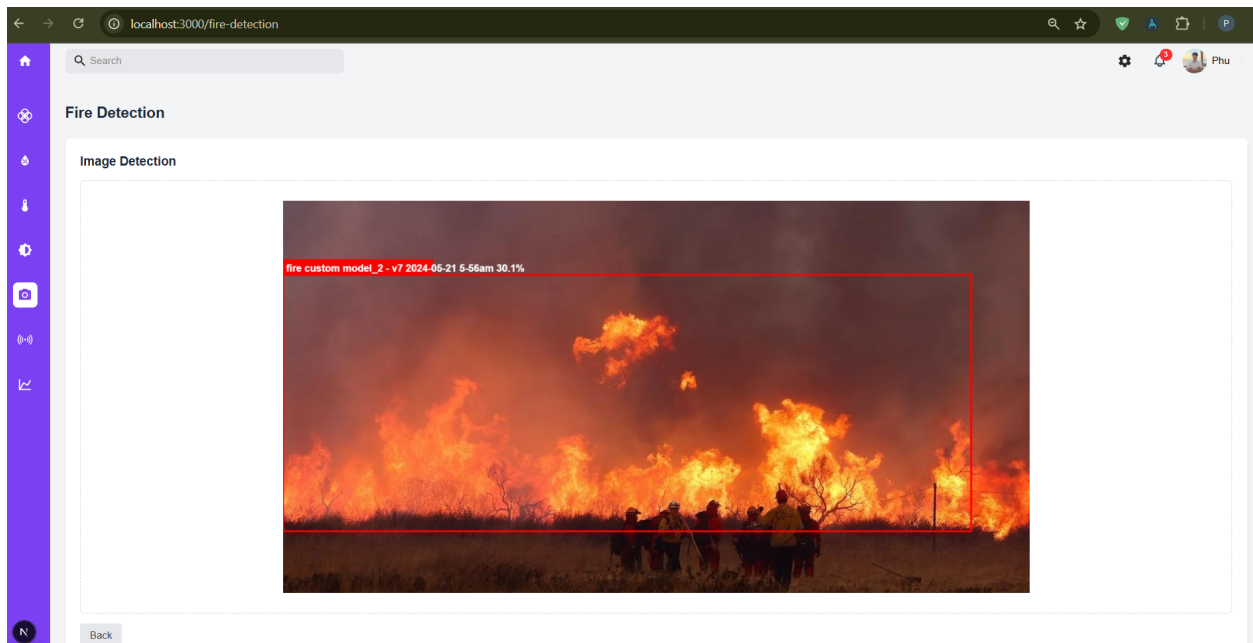


Figure 9.19: Image Detection: detecting fire by uploading an image and analyzing it with the remote detection server.

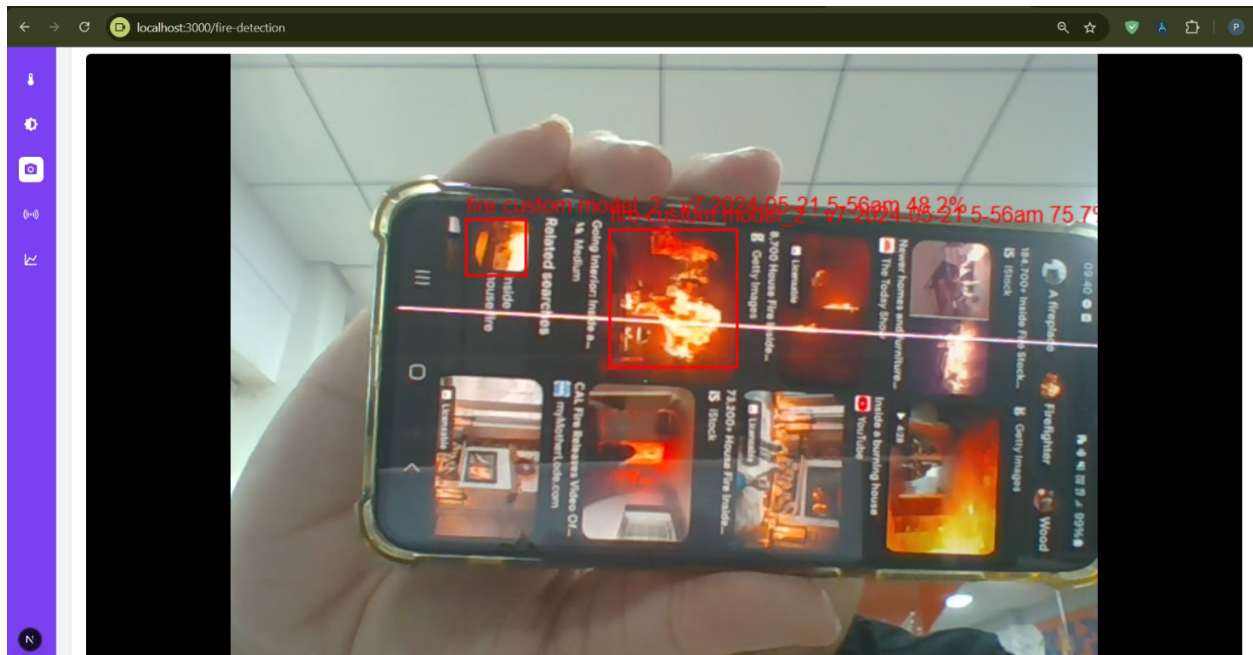


Figure 9.20: Video Stream Detection: using a laptop camera to simulate real-time fire detection via video streaming.

Mechanism of Detection System

The Fire Detection system is designed with a **frontend-server-model** architecture:

- **Frontend (Web App):** Provides the user interface for uploading images or streaming video data.
- **Server API:** Hosts pre-trained fire detection models and provides REST and Web-Socket APIs for communication.
- **Detection Model:** Processes incoming image or video frames, performs inference, and returns detection results (bounding boxes, classes, confidence scores).

Workflow:

1. For **Image Detection**:

- The user uploads an image through the frontend.
- The image is sent via a POST request to the remote server API (*/detect_images*).
- The server processes the image and responds with a list of detected fires and their bounding boxes.
- The frontend visualizes these detections by drawing bounding boxes on the uploaded image.

2. For **Video Stream Detection**:

- The frontend captures video frames using the device's camera.
- Each frame is resized and encoded, then sent to the server through a WebSocket connection ('/ws').
- The server analyzes the incoming frames and sends back real-time detection results.
- The frontend draws the detected fire bounding boxes on the video overlay in real-time.

Simplified Pseudocode for Detection:

```
// Image Detection Pseudocode
function detectFireInImage(file):
    upload file to remote_server/detect_images
    receive list of detections
    draw bounding boxes on uploaded image

// Video Stream Detection Pseudocode
class FireDetectionStream:
    open WebSocket connection to remote_server/ws
    for each video frame:
        resize and encode frame
        send frame via WebSocket
        receive detections
        draw bounding boxes on live video feed
```

Future Improvements

- Add a notification system to alert users immediately upon fire detection.
- Implement recording functionality for detected fire events in video stream mode.
- Improve detection speed and reduce latency through model optimization.
- Enable switching between multiple cameras or video sources for more flexible monitoring.
- Integrate an offline fallback mode for local fire detection if the server is unreachable.

Chapter 10

Conclusion

Throughout the development of the **Yolo:Home** project, our team has gained not only technical skills but also valuable insights into the practical challenges of building an AI-based smart home system. From initial requirement analysis to real-time testing and evaluation, each phase has contributed significantly to our overall understanding of both hardware and software integration in an IoT ecosystem.

One of the key milestones was the successful implementation of the **YOLO-based fire detection system**, which we trained, tested, and optimized across different versions (YOLOv10, YOLOv11, YOLOv12). This allowed us to evaluate improvements quantitatively and qualitatively and helped us understand how architectural changes impact performance.

In addition, working with various sensors and modules such as the **PIR sensor, IR sensor, DHT20, RC Servo, and light sensors** taught us how to interface hardware components effectively and how to handle real-time data acquisition and actuation within the Yolo:Bit environment. This experience strengthened our knowledge of embedded systems, sensor calibration, and system latency issues.

From a system design perspective, the construction of **modular use-cases and sequence diagrams** ensured that we followed a structured approach to software development, with clarity in responsibilities and logical workflows. These practices enhanced team collaboration and made debugging and scaling much more manageable.

Furthermore, a critical lesson emerged during the **real-time inference phase**. While testing our fire detection model in a real environment, we observed that the model tends to be **overly sensitive to objects with red or orange hues**, occasionally misclassifying such elements as fire. This behavior underlines the importance of real-world testing and opens avenues for future improvement, especially in terms of dataset refinement and model generalization.

In conclusion, this project not only helped us reinforce theoretical knowledge from our coursework but also pushed us to think critically and creatively when solving practical problems. We have grown in our understanding of AI model development, IoT integration, system evaluation, and collaborative engineering. These experiences will undoubtedly benefit us in future academic or industrial pursuits.

Chapter 11

Future Developments

To further improve the **Yolo:Home** system and address its current limitations, we have identified several future development directions. These improvements are aimed at enhancing usability, performance, scalability, and security of the platform:

- **Improved AI Models:** We will continue enhancing our AI models for fire detection and face recognition by expanding the dataset and refining the training process. The goal is to achieve higher accuracy and robustness in real-world deployment scenarios.
- **Mobile Application Development:** A dedicated mobile application will be developed to complement the current web interface. It will integrate all existing features while introducing push notifications and on-the-go system control, improving overall user convenience.
- **Broader Smart Device Integration:** As the IoT ecosystem grows, we plan to increase compatibility with a wider variety of smart devices. This will enhance the flexibility and extensibility of **Yolo:Home**, making it suitable for diverse environments and use cases.
- **Energy Efficiency Enhancements:** Future versions will implement smarter control algorithms that reduce energy usage without sacrificing responsiveness or user experience. This includes dynamic scheduling and adaptive response systems.
- **Advanced Security Measures:** To ensure data privacy and protect against potential cyber threats, we aim to adopt multi-factor authentication, end-to-end encryption, and secure communication protocols in all modules.

By implementing these improvements, **Yolo:Home** is expected to become a more reliable, user-friendly, and adaptable smart home platform that aligns with the evolving landscape of AIoT technology.

Chapter 12

Special Thanks

We would like to extend our deepest gratitude to our advisor, Dr. Nguyen Le Duy Lai, for his invaluable guidance throughout our journey in learning machine learning. His mentorship has not only helped us master complex machine learning concepts but has also provided us with a broader perspective on academic and career growth.

Beyond academic guidance, Dr. Nguyen Le Duy Lai has played a pivotal role in offering career advice, helping us navigate our professional aspirations with confidence and clarity. He has also encouraged us to build meaningful academic relationships with senior colleagues, broadening our perspectives and creating opportunities for future collaborations. His encouragement and dedication have been instrumental in shaping our academic growth and preparing us for future challenges.

We are truly grateful for his unwavering support and inspiration throughout this project.

Bibliography

- [1] Abu-Mostafa, Yaser S., Malik Magdon-Ismail, and Hsuan-Tien Lin, *Learning from Data*. Vol. 4. New York, NY, USA: AMLBook, 2012. [Online]. Available: <http://work.caltech.edu/telecourse.html>
- [2] Bennett, K. P., *Robust Linear Programming Discrimination of Two Linearly Separable Sets*. Optimization Methods and Software, Vol. 1, 1992. [Online]. Available: <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.23.3307&rep=rep1&type=pdf>
- [3] Cristhian Guerrero, *Amenazas Dataset*, Roboflow Universe, Roboflow, Jul. 2024. [Online]. Available: <https://universe.roboflow.com/cristhian-guerrero-eltw1/amenazas> (accessed: 2025-03-22).
- [4] Vercel, *Next.js Documentation*. [Online]. Available: <https://nextjs.org/docs> (accessed: 2025-04-21).
- [5] Sebastián Ramírez, *FastAPI Documentation*. [Online]. Available: <https://fastapi.tiangolo.com/> (accessed: 2025-04-21).
- [6] Evan You, et al., *Vite: Next Generation Frontend Tooling*. GitHub Repository. [Online]. Available: <https://github.com/vitejs/vite> (accessed: 2025-04-21).
- [7] Evan You, et al., *Vue.js Documentation*. [Online]. Available: <https://vuejs.org/guide/introduction.html> (accessed: 2025-04-21).
- [8] Ao Wang, et al., *YOLOv10: Real-Time End-to-End Object Detection*. arXiv preprint arXiv:2405.14458, 2024. [Online]. Available: <https://arxiv.org/abs/2405.14458>
- [9] Rizki H. G., Fadhilah A. H., Fitriyah H., *Face recognition for smart door security access with convolutional neural network*. TELKOMNIKA, Vol. 22, No. 3, pp. 637–646, Jun. 2024. [Online]. Available: <https://telkomnika.uad.ac.id/index.php/TELKOMNIKA/article/view/25946>
- [10] A. Singh, S. Sharma, A. K. Sharma, and R. Kumar, *Intelligent Door Locks with Facial Recognition*. Int. J. Precise Eng. Manag. (IJPREMS), Vol. 4, No. 11, pp. 49–54, Nov. 2024. [Online]. Available: https://www.ijprems.com/uploadedfiles/paper/issue_11_november_2024/36936/final/fin_ijprems1732013192.pdf

- [11] Jesusimo L. Dioses, Jr., *AndroiDuino-Fan: A Speech Recognition Fan-Speed Control System utilizing Filipino Voice Commands*. Int. J. Adv. Trends Comput. Sci. Eng., Vol. 9, No. 3, pp. 3042–3047, May–Jun. 2020. [Online]. Available: <https://www.warse.org/IJATCSE/static/pdf/file/ijatcse84932020.pdf>
- [12] Kazi M. F. Shahriar, *Automatic headlight controlling of vehicle using ambient light sensor based on phototransistor*. ScienceOpen Preprints, 2021. DOI: 10.14293/S2199-1006.1.SOR-.PPHTBD.v1. [Online]. Available: <https://www.scienceopen.com/document?vid=ad4cc92f-605a-4288-a920-3093e896bd4d>
- [13] S. Saravanan, S. Monisha, R. Santhosh Kumar, and V. Shanmuga Priya, *Air Quality Monitoring System*. Int. J. Eng. Res. Technol. (IJERT), Vol. 9, Issue 12, NC-CDS - 2021, pp. 68–71, 2021. [Online]. Available: <https://www.ijert.org/research/air-quality-monitoring-system-IJERTCONV9IS12017.pdf>
- [14] Shakeel Nazir, Ali Benhidour, Quoc Viet Hung Nguyen, Abdul Baseer, Waqar Hafeez, and Muhammad Adnan Khan, *Recent Advances in Sensors for Fire Detection*. Sensors, Vol. 22, No. 9, p. 3261, Apr. 2022. DOI: 10.3390/s22093261. [Online]. Available: <https://www.mdpi.com/1424-8220/22/9/3261>
- [15] Pan Shijia, Carlos Ruiz, Jun Han, Adeola Bannis, Patrick Tague, Hae Young Noh, and Pei Zhang, *UniverSense: IoT Device Pairing through Heterogeneous Sensing Signals*. Proc. 1st ACM Workshop on Device-Free Human Sensing (DeviceFree '18), pp. 27–32, Feb. 2018. DOI: 10.1145/3177102.3177108.